



天数智芯
Iluvatar CoreX

天数智芯

ixRT 推理引擎使用指南

版本：V4.3.0 (x86)|V4.3.6 (ARM)

日期：2025.6.30

适用产品：天垓 150 | 智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 修订记录	8
2 ixRT 推理引擎版本发布说明	10
2.1 新特性与功能优化	10
2.2 软件兼容性	11
2.3 已知问题与使用限制	12
2.3.1 接口功能限制	12
2.3.2 算子类型差异	16
2.3.3 算子支持限制	16
2.3.3.1 不支持的算子	16
2.3.3.2 ONNX 算子使用限制	16
2.3.3.3 算子功能限制	17
2.4 修复的问题	17
2.5 生态兼容性	17
3 ixRT 推理引擎安装指南	18
3.1 在 Docker 镜像中使用 ixRT 推理引擎	18
3.2 安装 Python 库	18
3.3 通过 C++ 库安装 ixRT 推理引擎 - .run 文件安装方式	18
3.4 通过 C++ 库安装 ixRT 推理引擎 - tar.gz 文件安装方式	19
3.5 卸载 ixRT 推理引擎	19
3.5.1 C++ 库	19
3.5.2 Python 库	20
3.6 升级 ixRT 推理引擎	20
3.6.1 C++ 库	20
3.6.2 Python 库	20
4 ixRT 推理引擎开发者文档	21
4.1 ixRT 推理引擎概览	21
4.1.1 使用示例	21
4.1.2 版本说明	21
4.1.3 C++ 和 Python API	21
4.1.4 编程模型	21
4.1.4.1 编译阶段	22
4.1.4.2 推理阶段	22
4.1.5 插件	22
4.1.6 数据类型	23
4.1.7 强类型与弱类型	23
4.1.8 量化	23
4.1.9 动态形状	24
4.1.10 条件控制流	24
4.1.11 ixrtexec 命令行工具	24
4.2 C++ API 文档	24
4.2.1 日志输出	25

4.2.2	使用 ixRT API 构建网络	25
4.2.3	使用 ONNX Parser 导入模型	26
4.2.4	构建 Engine	27
4.2.5	运行时对象	27
4.2.6	反序列化 Engine	28
4.2.7	执行推理	28
4.2.8	使用示例: ResNet18 模型推理	29
4.3	Python API 文档	31
4.3.1	构建阶段	31
4.3.1.1	创建网络	32
4.3.1.2	编译网络与序列化 Engine	33
4.3.2	运行阶段	33
4.4	高级话题	34
4.4.1	版本兼容性说明	34
4.4.2	强类型网络	35
4.4.2.1	概述	35
4.4.2.2	使用强类型网络 API	35
4.4.2.3	使用示例 - ResNet18	35
4.4.3	弱类型网络	38
4.4.3.1	概述	38
4.4.3.2	粗粒度精度控制	39
4.4.3.3	细粒度精度控制	39
4.4.3.4	使用示例	40
4.4.4	网络输入排布	40
4.4.5	空张量支持	40
4.4.6	输入数据监控	41
4.5	量化推理指南	42
4.5.1	量化原理说明	42
4.5.2	隐式量化与显式量化	42
4.5.2.1	隐式量化推理流程	43
4.5.2.2	显式量化推理流程	44
4.5.3	量化工具说明	47
4.6	动态图形推理指南	48
4.6.1	输入动态形状	48
4.6.2	输出动态形状	52
4.6.3	Shape Tensor 与 Execution Tensor	56
4.6.4	算子的形状输入	57
4.6.5	Shape Tensor I/O	58
4.6.6	形状表达总结	60
4.7	自定义插件使用指南	60
4.7.1	使用 C++ 接口添加自定义插件	61
4.7.1.1	插件接口	61
4.7.1.2	示例: 实现一个自定义插件	62
4.7.2	调用自定义插件	73
4.7.2.1	在 C++ 中使用用户自定义插件	73
4.7.2.2	在 Python 中使用用户自定义插件	77
4.7.2.3	使用 ixRT 内置插件	80

4.8	条件控制流推理指南	82
4.8.1	概述	82
4.8.2	条件控制流的结构	82
4.8.3	构建条件控制流	83
4.8.3.1	定义条件控制流	83
4.8.3.2	抽象模型结构图	84
4.8.3.3	条件控制流使用示例	84
4.8.3.4	使用限制与注意事项	85
4.8.4	执行策略	85
4.8.5	嵌套条件控制流	88
5	配套软件 CUDA Python 使用指南	90
5.1	安装 CUDA Python	90
5.2	使用示例	90
5.3	API 参考	99
6	ixRT 推理引擎辅助工具使用指南	102
6.1	插件生成工具使用指南	102
6.1.1	安装插件生成工具	102
6.1.2	使用插件生成工具	102
6.2	ixrtexec 命令行工具使用指南	103
6.2.1	参数说明	103
6.2.2	使用示例	105
6.2.2.1	示例 1: INT8 模式运行 ResNet50 模型	105
6.2.2.2	示例 2: FP16 模式运行 ResNet50 模型	105
6.2.2.3	示例 3: 运行精度对比	105
6.2.2.4	示例 4: 进行性能分析并保存结果文件	106
6.2.2.5	示例 5: 运行含插件的模型	107
6.3	ixRT Model Optimizer 图融合工具使用指南	107
6.3.1	功能说明	107
6.3.2	参数说明	108
6.3.3	运行使用示例	108
6.3.3.1	示例 1: 融合 albert/videoBERT/roberta/deberta	108
6.3.3.2	示例 2: 融合 swinL	109
6.3.3.3	示例 3: 融合 roformer	109
6.3.3.4	示例 4: 融合 yolov5s	109
6.3.4	精度验证示例	109
6.3.4.1	示例 1: albert 模型	109
6.3.4.2	示例 2: swinL 模型	110
6.4	ixRT Deploy 量化工具使用指南	110
6.4.1	参数说明	110
6.4.2	使用示例	111
7	ixRT 推理引擎使用调试指南	114
7.1	精度对比工具	114
7.1.1	准备工作	114
7.1.2	使用方法	114

7.1.3	使用示例	115
7.1.3.1	示例 1: 对比 FP16 模型	115
7.1.3.2	示例 2: 对比 INT8 QDQ 模型	115
7.1.3.3	示例 3: ixRT 使用的 ONNX 包含自定义插件	115
7.1.3.4	示例 4: 比较引擎文件和原 ONNX	115
7.1.3.5	示例 5: 保存对比数据	116
7.1.3.6	示例 6: 运行时输入注入	116
7.2	性能调试工具	116
7.3	ixRT Hook 接口	117
7.3.1	C++ API	117
7.3.2	Python API	119
8	附录	122
8.1	ixRT 内置插件列表	122
8.1.1	AffineGrid	123
8.1.2	CustomFCPluginDynamic_IxRT	124
8.1.3	RoiAlign	126
8.1.4	TransformerEncoderFp16	127
8.1.5	BertLnResidual	127
8.1.6	CustomGeluPluginDynamic_IxRT	129
8.1.7	CustomRoPEPluginDynamic_IxRT	129
8.1.8	CMVN	131
8.1.9	EcapaAspAttn	132
8.1.10	CustomSkipLayerNormPluginDynamic_IxRT	132
8.1.11	ViterbiDecode	135
8.1.12	Conformer_Conv_Module	136
8.1.13	EcapaPool	138
8.1.14	Random Normal Like	139
8.1.15	SplitQKV	140
8.1.16	Conformer_Encoder_Beam_Search	141
8.1.17	EcapaScorePool	142
8.1.18	Inverse Sigmoid	143
8.1.19	RebuildPadding_IxRT	143
8.1.20	XSoftmax	146
8.1.21	Conformer_Self_Attention	147
8.1.22	Efficient NMS Plugin	149
8.1.23	L2_Normalization	149
8.1.24	RemovePadding_IxRT	150
8.1.25	YolactDecoder	152
8.1.26	Conv2dSubsampling4	153
8.1.27	CustomEmbLayerNormPluginDynamic_IxRT	155
8.1.28	Reorg	157
8.1.29	YoloV3Decode	158
8.1.30	CTCGreedySearch	160
8.1.31	FacenetNorm	160
8.1.32	multiscaleDeformableAttn	161
8.1.33	RetinaNet Decoder	162

8.1.34 TemporalShift	163
8.1.35 YoloV5Decoder	164
8.1.36 CustomFFNPluginDynamic_IxRT	165
8.1.37 DetectionNMS_IxRT	166
8.1.38 RMS_Norm	168
8.1.39 YoloV7Decoder	169
8.1.40 Custom argmax	170
8.1.41 Focus	171
8.1.42 PointRendCalUncertaint	172
8.1.43 RNN	173
8.1.44 YoloxDecoder	174
8.1.45 fcosNMS	176
8.1.46 PositionWiseFFN	177
8.1.47 CustomQKVToContextPluginDynamic_IxRT	178
8.2 ixRT 内置示例列表	180

1 修订记录

- COREX01-BI430-UG11-00: 2025/5/28
重构 ixRT v1.0.0 用户文档结构和内容, 详情请参考正文
- COREX01-BI420-UG11-01: 2025/3/18
 - 更新“IxRT 推理引擎示例安装指南”:
 - * 更新解压 .run 包和 tar.gz 后的目录结构
 - * 删除 .run 下的运行示例
 - * 更新 tar.gz 下示例的编译方法
- COREX01-BI420-UG11-00: 2025/1/24
 - 更新本次发布的 IxRT 版本为 v0.10.0
 - 新增提示: IxRT 兼容 TensorRT API, 所有 import tensorrt 和 import ixrt 等价
 - “附录 1: IxRT 适配的模型”, 删除 Text2Image 类模型, IxRT 将不再支持维护 Stable Diffusion 模型
 - 更新“附录 4: IxRT 支持的插件列表”, 新增插件 No.26 至 No.51
- COREX01-BI411SP1-UG11-01: 2024/12/26
 - 新增“常见问题”章节
- COREX01-BI411SP1-UG11-00: 2024/12/18
 - 更新“IxRT optimizer 图融合工具使用指南”, 提供 IxRT optimizer 工具的安装说明, 更新工具所在目录
- COREX01-BI411-UG11-01: 2024/11/18
 - 支持的配套 Python 库删除 PyCUDA, 保留 CUDA Python, 涉及的章节有:
 - * IxRT 推理引擎介绍
 - * 配套软件使用指南 -> 配套软件 CUDA Python 使用指南
 - 更新“使用 IxRT 推理引擎进行模型推理最佳实践”章节中的“步骤 5 加载引擎文件并执行推理”, 修改调用推理接口代码
 - 更新“(面向开发者) 使用 IxRT 接口进行开发”章节中的“Python API”, 将 PyCUDA 库修改为 cuda-python 库
 - 删除“教程 1: 对 ResNet18 模型进行 FP16 推理 (Python)”, 内容已不适用
- COREX01-BI411-UG11-00: 2024/8/26
 - 更新“IxRT 推理引擎示例安装指南”, 提示用户可在 [天数智芯官网](#) 下载 IxRT 安装包
 - 更新“使用 IxRT 推理引擎对模型进行 INT8 推理最佳实践”, 提示用户 qat_classification.py 脚本已打包在 IxRT 安装包中, 所在目录为 .../samples/python/quantization/classification/qat_classification.py
- COREX01-BI410-UG11-00: 2024/6/30
 - 更新本次发布的 IxRT 版本为 v0.9.1
 - 新增“IxRT optimizer 图融合工具使用指南”
 - 更新“IxRT-EXEC 自动化模型推理工具使用指南”章节, 新增 --diff_rel_avg 运行参数

- 更新“附录 2: IxRT 支持的算子列表”章节
- 更新“附录 4: IxRT 支持的插件列表”章节，新增以下插件：
 - * CustomEmbLayerNormPluginDynamic_IxRT
 - * CustomQKVToContextPluginDynamic_IxRT
 - * CustomSkipLayerNormPluginDynamic_IxRT
 - * CustomGeluPluginDynamic_IxRT
 - * CustomFCPluginDynamic_IxRT
 - * TemporalShift

• COREX01-BI340-UG11-01: 2024/6/6

- 更新“教程 1: 对 ResNet18 模型进行 FP16 推理 (Python)”，统一库名和配置文件变量名

• COREX01-BI340-UG11-00: 2024/3/25

文档本次发布内容与 V3.3.0-BI150 文档相比 (COREX01-BI330-UG11-00)，有以下更新：

- 更新本次发布的 IxRT 版本为 v0.9.0
- “IxRT 模型推理示例教程”章节中，删除原“教程 1: 对 Bert 模型进行 FP16 推理”小节，删除理由：该内容已失效
- 更新“IxRT 自定义插件算子使用指南”章节，新增“使用插件生成工具”小节，提供插件生成工具的安装和使用方法
- 新增“关于 IxRT 使用的高级话题”章节，提供面向开发者使用 IxRT 的话题
- 更新“附录 2: IxRT 支持的算子列表”章节
- 新增“附录 4: IxRT 支持的插件列表”章节，并提供各插件的算子详情
- “IxRT-EXEC 自动化模型推理工具使用指南”章节中，删除 **--batch_size** 运行参数，删除理由：该参数已失效

2 ixRT 推理引擎版本发布说明

本章节用于记录 ixRT 推理引擎版本发布时的新特性、软件功能改进及优化，以及一些已知问题的修复。

ixRT 推理引擎当前版本是 v1.0.0，适配了 TensorRT v8.4 至 v10.8 的核心功能接口。ixRT 针对功能近似 TensorRT 但不同的接口做了保留，如果存在行为有冲突的接口，保留 TensorRT 中较新版本的接口。

2.1 新特性与功能优化

- 使用 onnx-tensorrt 作为模型解析器。

onnx-tensorrt 是一套开源的解决方案，用于将 ONNX 算子解析后使用推理引擎的模型搭建接口创建模型，可以较好地支持不同地 ONNX OpSet。

- 算子类型完全与 TensorRT 对齐，不混合 ONNX 的算子类型。

- 新增算子类型，包括：

- Cumulative
- Squeeze
- Unsqueeze
- ReverseSequence
- RaggedSoftMax

使用原生支持的算子可以避免使用多个算子组合的方式来满足同样的操作需求。

- Einsum 算子新增支持更丰富的计算模式。

- 废弃 FullyConnected 算子。

- 按照 TensorRT 的新标准，可以使用 MatrixMultiply 和 Elementwise 的组合进行表达。
- 推理引擎内部处理计算图融合。

- Elementwise 算子支持逻辑运算。

- 支持 FP32 推理。

添加了 ConvolutionND, PoolingND, MatrixMultiply 等多个计算密集型算子对 FP32 精度的计算支持。

- 支持条件控制流推理。

对于 NonZero 一类，输出内存用量是由输入数据而不是输入形状决定的算子，支持以下两种输入内存管理方案：

- 一是 OutputAllocator，用户实现自定义版本的 OutputAllocator，并提供给 ExecutionContext，在推理过程中根据需求申请输出空间，并通知输出形状；
- 二是设置了输入的形状后，调用 ExecutionContext 的 getMaxOutputSize 方法，获取指定输出 Tensor 的最大内存用户量，推理结束后调用 getDimension 方法，获取最终的输出结果形状。

- 提升对动态形状的推理能力。

ixRT 提供了能力更强的形状分析机制，可以支持计算图中复杂的形状变换过程。因此在模型层面上合法地输入维度数值调整，不加以任何使用限制。

- 维度数值使用 INT64。

尤其是一些 NLP 模型，单个维度的数值就会超过 INT32。ixRT 跟随 TensorRT，使用 INT64 表达维度数值。

- 使用更轻量化的 C++ 代码集成模式。

C++ 的库文件有：

- libixrt.so (推理)
- libixrt_builder_resource.so (编译)
- libixrt_lean.so (公共组件)
- libixrt_plugin.so (插件)
- libixrtonnxparser.so (解析 ONNX 模型)

其中，

- builder 库用于编译模型阶段，体积较大；
- 在纯推理阶段，最少可以只使用 ixrt 和 lean 库；
- 如果需要用到 ONNX 模型解析，则加上 onnxparser 库；
- 如果需要引用插件，则加上 plugin 库。

- 废弃使用量化表的模型解析形式。

量化模型的支持形式仅限于 ONNX QDQ 形式，以及 ITensor 的 setDynamicRange 接口，原有通过 JSON 导入量化参数表的模式被废弃。

- 废弃内部量化机制。

内部量化机制使用限制较多，而且对用户而言，使用门槛也较高，且从未被引用过。因此当前版本参考 TensorRT 对量化推理的支持形式对齐，通过 QDQ 模型或参数设置接口的方式获取量化参数。

2.2 软件兼容性

- ixRT 可以支持的 Python 版本有：

- Python v3.8
- Python v3.9
- Python v3.10
- Python v3.11

- ixRT 本身不依赖任何开源工具包

- ixRT 提供的辅助工具对 Python 的依赖如下：

- ixrtexec 命令行工具：
 - * cuda_python
 - * onnx
 - * numpy == 1.23.5
 - * onnxruntime, scipy (--verify_acc)
 - * pytorch (--precision int8)
- Model Optimizer 图融合工具：
 - * onnxsim

- * packaging
- * sympy
- Deploy 量化工具：
 - * onnx
 - * numpy == 1.23.5
 - * tabulate
 - * pytorch

2.3 已知问题与使用限制

2.3.1 接口功能限制

为了保障与开源软件 (例如, onnx-tensorrt, python-binding) 的对接能顺利编译通过, 当前版本未直接去除不支持的接口。不支持的接口如下:

- ITensor 类：
 - setBroadcastAcrossBatch
 - getBroadcastAcrossBatch
 - setAllowedFormats
 - getAllowedFormats
- INetworkDefinition 类：
 - markDebug
 - unmarkDebug
 - isDebugTensor
 - addPluginV3
 - hasImplicitBatchDimension
- IInt8Calibrator 类
- IAlgorithmIOInfo 类
- IAlgorithmVariant 类
- IAlgorithmContext 类
- IAlgorithm 类
- IAlgorithmSelector 类
- BuilderFlag 枚举
 - kDEBUG
 - kGPU_FALLBACK
 - kREFIT
 - kTF32
 - kSPARSE_WEIGHTS
 - kSAFETY_SCOPE
 - kDIRECT_IO
 - kREJECT_EMPTY_ALGORITHMS
 - kVERSION_COMPATIBLE
 - kEXCLUDE_LEAN_RUNTIME

- kFP8
- kBF16
- kSTRIP_PLAN
- kREFIT_IDENTICAL
- kWEIGHT_STREAMING
- MemoryPoolType 枚举
- HardwareCompatibilityLevel
- IProgressMonitor 类
- IBuilderConfig 类：
 - setAvgTimingIterations
 - getAvgTimingIterations
 - setEngineCapability
 - getInt8Calibrator
 - setDeviceType
 - getDeviceType
 - isDeviceTypeSet
 - resetDeviceType
 - canRunOnDLA
 - setDLACore
 - getDLACore
 - setDefaultDeviceType
 - getDefaultDeviceType
 - setProfileStream
 - getProfileStream
 - getAlgorithmSelector
 - setCalibrationProfile
 - getCalibrationProfile
 - setQuantizationFlags
 - getQuantizationFlags
 - clearQuantizationFlag
 - setQuantizationFlag
 - getQuantizationFlag
 - setTacticSources
 - getTacticSources
 - createTimingCache
 - setTimingCache
 - setMemoryPoolLimit
 - getMemoryPoolLimit
 - setPreviewFeature
 - getPreviewFeature
 - setHardwareCompatibilityLevel
 - getHardwareCompatibilityLevel
 - setPluginsToSerialize
 - getPluginToSerialize
 - getNbPluginsToSerialize
 - setMaxAuxStreams

- getMaxAuxStreams
- setProgressMonitor
- getProgressMonitor
- IBuilder 类：
 - getMaxDLABatchSize
 - getNbDLACores
 - platformHasTf32
 - isNetworkSupported
- getBuilderSafePluginRegistry 函数
- IConsistencyChecker 类
- IPluginChecker 类
- EngineCapability 枚举
- IPluginResourceContext 类
- IPluginV3 类
- WeightsRole 枚举
- TempfileControlFlag 枚举
- IRuntime 类：
 - setDLACore
 - getDLACore
 - getNbDLACores
 - setTemporaryDirectory
 - getTemporaryDirectory
 - getTempfileControlFlags
 - loadRuntime
 - setEngineHostCodeAllowed
 - getEngineHostCodeAllowed
- IRefitter 类
- IOptimizationProfile 类
 - setExtraMemoryTarget
 - getExtraMemoryTarget
- TacticSource 枚举
- SerializationFlag 枚举
- ISerializationConfig 枚举
- ICudaEngine 类：
 - isRefittable
 - getTensorBytesPerComponent
 - getTensorBytesPerComponent
 - getTensorComponentsPerElement
 - getTensorVectorizedDim
 - getTensorVectorizedDim
 - getEngineCapability
 - hasImplicitBatchDimension
 - getTacticSources
 - getProfilingVerbosity

- createEngineInspector
- getHardwareCompatibilityLevel
- getNbAuxStreams
- createSerializationConfig
- serializeWithConfig
- setWeightStreamingBudget
- getWeightStreamingBudget
- getMinimumWeightStreamingBudget
- getStreamableWeightsSize
- isDebugTensor
- IDebugListener 类
- IExecutionContext 类:
 - setDebugSync
 - getDebugSync
 - getTensorStrides
 - getRunningProfiler
 - setEnqueueEmitsProfile
 - getEnqueueEmitsProfile
 - reportToProfiler
 - setTemporaryStorageAllocator
 - getTemporaryStorageAllocator
 - setPersistentCacheLimit
 - getPersistentCacheLimit
 - setNvtxVerbosity
 - getNvtxVerbosity
 - setTensorDebugState
 - setAllTensorsDebugState
- LayerInformationFormat 枚举
- IEngineInspector 类
- createInferRefitter 函数
- DataType 枚举:
 - kFP8
 - kBF16
 - kFP4
- TensorFormat 枚举:
 - kCHW2
 - kHWC8
 - kCHW4
 - kCHW16
 - kCHW16
 - kDHW8
 - kCDHW32
 - kDLA_LINEAR
 - kDLA_HWC4
 - kHWC16

- kDHWc
- IStreamReader 类
- IPluginResource 类
- IPluginRegistry 类:
 - isParentSearchEnabled
 - setParentSearchEnabled
 - loadLibrary
 - deregisterLibrary
 - acquirePluginResource
 - releasePluginResource
- PluginVersion 枚举: kv2_DYNAMICEXT_PYTHON
- PluginCapabilityType 枚举
- TensorRTPhase 枚举
- IOnnxConfig 类
- IParser 类: 除了 parseFromFile 以外的所有方法
- IParserRefitter 类
- createParserRefitter 函数

2.3.2 算子类型差异

Activation 支持的类型，在 TensorRT 的基础上增加了：

- SILU
- HARD_SWISH
- MISH

2.3.3 算子支持限制

2.3.3.1 不支持的算子

由于暂无使用需求，没有适配的算子类型如下：

- 循环控制流相关算子:
 - TRIP_LIMIT
 - RECURRENCE
 - ITERATOR
 - LOOP_OUTPUT
- PluginV3
- DYNAMIC_QUANTIZE

2.3.3.2 ONNX 算子使用限制

不支持非标准的 ONNX 算子，SILU ONNX 算子列表详见 <https://github.com/onnx/onnx/blob/main/docs/Operators.md>。

2.3.3.3 算子功能限制

暂无使用需求，如下算子的部分属性和功能不可用：

- ConvolutionND:
 - 不支持 Padding Mode:
 - * kEXPLICIT_ROUND_UP
 - * kSAME_UPPER
 - * kSAME_LOWER
 - 不支持非对称 Padding 值
- Deconvolution:
 - 不支持 3D deconvolution
 - 不支持 Padding Mode:
 - * kEXPLICIT_ROUND_UP
 - * kSAME_UPPER
 - * kSAME_LOWER
 - 不支持非对称 Padding 值
- Resize:

不支持的差值模式: InterpolationMode::kCUBIC
- Pooling:

不支持计算模式: kMAX_AVERAGE_BLEND
- Assertion: 不支持变量 Tensor 输入
- GridSample: 不支持 InterpolationMode::kCUBIC 模式
- Unary: 不支持 UnaryOperation::kNOT
- EINSUM: 不支持 INT8
- REDUCE: 仅支持 GPU, 不支持 Shape Tensor 计算
- Quantize: 输入数据类型仅支持 float16 和 float32, 输出数据类型仅支持 INT8
- Dequantize: 输入数据类型仅支持 INT8, 输出数据类型仅支持 float16 和 float32

2.4 修复的问题

- 解决了插件库在编译阶段存在的内存泄露问题。
- 修正了 PluginV2 接口中与 TensorRT 行为不一致的问题。

2.5 生态兼容性

ixRT Model-Zoo 提供了对应版本号的 tag, 确保 tag 下所有模型与其对应的版本在 ixRT 中是经过验证的。

3 ixRT 推理引擎安装指南

本章节旨在介绍 ixRT 推理引擎的安装方法。我们提供以下方式安装 ixRT 推理引擎：

- 在 Docker 镜像中使用 ixRT 推理引擎
- 安装 Python 库
- 通过 C++ 库安装 ixRT 推理引擎 - .run 文件安装方式
- 通过 C++ 库安装 ixRT 推理引擎 - tar.gz 文件安装方式

在安装 ixRT 推理引擎前，请您先参考《软件栈安装指南》安装天数智算软件栈。

3.1 在 Docker 镜像中使用 ixRT 推理引擎

如果您在安装天数智算软件栈时使用的是 Docker 镜像安装包，该镜像中已包含 ixRT C++ 库，无需您再手动安装。

如果您需要调用 Python 库，请参考[安装 Python 库](#)。

3.2 安装 Python 库

1. 通过以下方式获取 ixRT 的 .whl 包并使用 pip3 进行安装：

- 登录 [天数智芯官网](#)，进入 [客户支持](#) > [资源中心](#) 页面进行下载
- 联系您的应用工程师获取

2. 安装完成后，在终端运行如下命令检查安装是否成功：

```
python3 -c "import tensorrt; print(tensorrt.__version__)"
```

如果回显版本号，表示安装成功。

3.3 通过 C++ 库安装 ixRT 推理引擎 - .run 文件安装方式

1. 获取 ixRT 的 .run 包并运行 (可自行登录天数智芯官网下载或联系您的应用工程师获取)：

```
$ sudo bash ixrt-{ixrt_version}+corex.{v.r.m}-linux_{arch}.run
```

通过上述命令即可将 ixRT 安装至系统目录，默认为 /usr/local/corex/，需要管理员权限进行安装。

您也可以使用 --prefix 参数来指定安装目录，比如：

```
bash ixrt-{ixrt_version}+corex.{v.r.m}-linux_{arch}.run -- --prefix=~/.softwares/ixrt
```

2. 安装完成后，设置如下环境变量或将其写到您的 ~/.bashrc 中 (如果您用的是 bash，配置文件是 ~/.bashrc，其它 shell 请参考对应配置文件)，使程序可以链接至 ixRT 库，例如：

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/corex/lib
```

3. 以默认安装目录为例，解压后的目录结构如下：

```
$ tree -L 1 /usr/local/corex
```

```
├── include # 头文件所在路径
└── lib # 库文件所在路径
```

3.4 通过 C++ 库安装 ixRT 推理引擎 - tar.gz 文件安装方式

1. 获取 ixRT 的 tar.gz 包，解压并手动安装 (可自行登录天数智芯官网下载或联系您的应用工程师获取)：

```
$ tar xvfz ixrt-{ixrt_version}+corex.{v.r.m}-linux_{arch}.tar.gz
$ cd IxRT
```

```
# 手动头文件和库到指定目录
```

```
$ cp lib/* {path_to_prefix}/lib
$ cp include/* {path_to_prefix}/include
```

2. 解压后的目录结构如下：

```
IxRT
├── CMakeLists.txt
├── LICENSE
├── README.md
├── cmake # cmake 示例
├── data # samples 依赖数据
├── include # ixRT 头文件所在路径
├── install # 安装脚本
├── lib # ixRT C++ 库文件所在路径
├── requirements # ixRT Python 依赖
├── samples # ixRT 开源代码部分，包含 C++/Python API 使用示例
└── tools # ixRT 提供的工具
```

3. 安装完成后，设置如下环境变量或将其写到您的 ~/.bashrc 中 (如果您用的是 bash，配置文件是 ~/.bashrc，其它 shell 请参考对应配置文件)，使程序可以链接至 ixRT 库，例如：

```
$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/corex/lib
```

3.5 卸载 ixRT 推理引擎

3.5.1 C++ 库

如需卸载通过 .run 或 tar.gz 包安装的 ixRT，请在目标安装目录删除头文件和库文件。

以默认安装目录 /usr/local/corex 为例，通过如下命令删除文件：

```
$ rm -f /usr/local/corex/include/NvInfer*.h  
$ rm -f /usr/local/corex/include/NvOnnx*.h  
$ rm -f /usr/local/corex/lib/libixrt*
```

另外，您可以选择恢复先前安装时设置的 LD_LIBRARY_PATH 变量。

3.5.2 Python 库

通过如下命令卸载 ixRT：

```
$ pip3 uninstall ixrt
```

3.6 升级 ixRT 推理引擎

3.6.1 C++ 库

- 如果使用 .run 安装包，请先确认安装目录与之前版本的安装目录相同，并参考[通过 C++ 库安装 ixRT 推理引擎 - .run 文件安装方式](#) 提供的步骤操作即可覆盖安装。
- 如果使用 tar.gz 安装包，请参考[卸载 ixRT 推理引擎](#) 提供的步骤卸载旧版本的 ixRT，并再次手动安装新版本即可。

3.6.2 Python 库

通过如下命令升级 ixRT：

```
$ pip3 uninstall ixrt -y  
$ pip3 install ixrt-{ixrt_version}  
  --cp{python_version}-cp{python_version}-linux_{arch}.whl
```

4 ixRT 推理引擎开发者文档

本章节内容主要面向 ixRT 推理引擎的开发用户，具有一定技术性，包含：

- ixRT 推理引擎概览
- C++ API 文档
- Python API 文档
- 高级话题
- 量化推理指南
- 动态图形推理指南
- 自定义插件使用指南
- 条件控制流推理指南

4.1 ixRT 推理引擎概览

本节内容主要介绍 ixRT 的基本信息，以及如何使用天数智算软件栈提供的 API 来构建一套最常用的推理解决方案。

4.1.1 使用示例

“附录”中提供的“ixRT 内置示例列表”，展示了使用 ixRT 进行不同推理需求的使用示例。

4.1.2 版本说明

ixRT 的版本号定义为“MAJOR.MINOR.PATCH”，更新规则是：

- MAJOR：在 API 兼容性方面发生变化
- MINOR：重要功能更新
- PATCH：bug 修复

4.1.3 C++ 和 Python API

ixRT 提供了 C++ 和 Python 两套接口，并且保持功能几乎完全对等，详情请参考“C++ API 文档”和“Python API 文档”。

4.1.4 编程模型

ixRT 的工作流主要体现在以下两个阶段：

- 编译
- 推理

4.1.4.1 编译阶段

编译操作的起始点是 Builder 类，负责优化 AI 模型并生成最终的推理解决方案 Engine。构建 Engine 需要进行如下操作：

1. 定义模型
2. 设定配置
3. 编译生成 Engine

NetworkDefinition 类相关接口用于定义模型，常见的方式有以下两种：

- 使用 ONNX 解析器，将 ONNX 模型直接导入为 NetworkDefinition 对象
- 使用 NetworkDefinition 的 API 并手动构建模型

BuilderConfig 类相关的接口用于记录和构建 Engine 时的配置。这些配置项指导了编译模型的过程中可以参考的优化策略，例如，降低运算精度可以提升推理效率。

Builder 使用模型定义和编译配置，用于创建 Engine。在这个过程中会进行模型优化，例如，死代码消除、常量折叠、算子融合等。当一个算子有多个实现时，会通过算法选择流程找出指定参数配置下性能最佳的实现。

Builder 创建的 Engine 存储在一个序列化好的内存对象中，称之为 Plan，是一个 HostMemory 对象。这个文件可以序列化至磁盘，或直接进行反序列化后开始推理流程。

4.1.4.2 推理阶段

推理操作的起始点是 Runtime 类。当使用 runtime 时，关键操作有：

1. 反序列化 plan，创建 Engine 对象
2. 使用 Engine 对象创建 Context 对象

使用 Context 接口进行：

1. 提供输入数据
2. 调用 enqueue 或 execute 接口执行推理过程
3. 获取输出数据

Engine 表达的是一个优化后的模型，提供的访问接口用于获取模型输入和输出 Tensor 的信息，例如，数据类型等。

Context 是从 Engine 创建的，主要用于设定运行时需要的数据和参数，而且可以使用一个 Engine 创建多个 Context，并发的执行所有 Context。

调用推理接口 enqueue 可以在执行时绑定一个 CUDA Stream，使用异步的方式执行推理，即在这个接口调用后将控制权返回主机端，等待执行结束后可以调用 cudaStreamSynchronize 接口。

4.1.5 插件

ixRT 提供了插件接口，允许用户根据自己的需求实现原生算子没有支持的计算操作。在构建模型的过程中，可以在模型中直接添加插件算子并将计算结果输出给其它算子。

ixRT 提供了一个插件库，包含一些 AI 领域相关的插件算子，例如，RoiAlign 等，用户可以直接使用这些插件算子，详情请参考“ixRT 内置插件列表”。

4.1.6 数据类型

ixRT 支持的数据类型包括：

- FP32
- FP16
- INT8
- INT32
- INT64
- BOOL

内置算子支持的数据类型会受到计算模式的影响。FP32 和 FP16 是多数算子都支持的数据类型。INT8 是需要有相应的量化参数支持的情况下，才能使用这个精度进行推理。

- 隐式量化，在待量化的 Tensor 上通过 setDynamicRange 一类接口，设定量化参数。
- 显式量化，在构建模型时，通过插入 Q/DQ 算子。

4.1.7 强类型与弱类型

在创建一个 NetworkDefinition 对象时，需要指定这个模型使用强类型或弱类型，默认为弱类型。

对于强类型网络，ixRT 在编译时会依据输入的类型和算子的类型推导规则获取模型中所有 Tensor 使用的数据类型，并且在计算图优化过程中不会改变这些类型。

对于弱类型网络，ixRT 在编译时会依据输入类型、精度限制、算子性能等因素设置模型中所有 Tensor 的类型，即在计算图优化过程中，Tensor 的数据类型可能会发生变化。

在弱类型网络中，有很多控制精度的接口，例如，使用 BuilderFlag 中的选项决定了模型优化过程中允许使用的最低精度，并不意味着一定会使用这个精度。

更精细的控制，可以使用 Layer 对象的 setPrecision 接口，也可以用于设置 Tensor 的精度，但是作用范围仅限于当前算子。

关于强类型和弱类型的更多信息，请参考“高级话题”。

4.1.8 量化

对模型进行量化可以使用更低的数据精度进行推理，在计算过程中可以显著地降低数据传输压力，节省内存带宽，从而提升推理效率。

ixRT 不提供量化接口，需要用户自己来完成模型的量化工作。用户可以使用的工具有：

- ONNXRuntime
- PPQ
- ixRT Deploy

因此，量化模型支持如下两种形式：

- 带有 Q/DQ 算子的模型
- 构建模型之后，使用 Tensor 对象的 setDynamicRange 接口设定数值范围 (最小值与最大值)

关于量化推理的更多信息，请参考“量化推理指南”。

4.1.9 动态形状

模型输入形状可以在运行时动态调整，因此需要在编译阶段进行如下操作：

1. 构建模型时，标记需要在运行时动态调整的维度；
2. 使用 OptimizationProfile 对象配置动态形状的范围，例如，动态维度的最大值与最小值。

OptimizationProfile 对象保存在 BuilderConfig 对象中，且用户可以创建多个 OptimizationProfile 对象用于配置不同的动态形状范围。模型优化过程将依据这些配置选择算子的最优实现。

关于动态形状推理的更多信息，请参考“动态图形推理指南”。

4.1.10 条件控制流

在构建模型的过程中，用户可以使用 IfCondition 相关接口标记模型中的子图并创建条件控制流，这些控制流支持嵌套。在模型运行时，会根据条件依据的逻辑数值决定待执行的子图。

在标记子图范围时，IfCondition 相关接口类似于 C++ 这类编程语言的括号，逻辑判断条件要求必须是 BOOL 类型。

关于条件控制流推理的更多信息，请参考“条件控制流推理指南”。

4.1.11 ixrtexec 命令行工具

ixRT 提供命令行工具 ixrtexec，该工具允许用户在无需任何开发的场景下快速验证推理流程。流程包括：

1. 解析模型
2. 编译 Engine
3. 评测推理性能

ixrtexec 的一些高级使用选项可以辅助开发者进行调试功能，更多信息，请参考“ixRT 推理引擎辅助工具使用指南”。

4.2 C++ API 文档

本节内容主要介绍 ixRT C++ API 的使用方法。

ixRT 的 C++ API 可以通过头文件 NvInfer.h 访问，并且在 nvinfer1 命名空间中。例如，一个简单的应用程序可能会以以下内容开头：

```
#include "NvInfer.h"
using namespace nvinfer1;
```

4.2.1 日志输出

在下面的示例中，Logger 类继承自 `nvinfer1::ILogger` 并重写了 `log` 函数，用于处理不同级别的日志信息。在 `main` 函数中，logger 对象在创建 `IBuilder` 时被使用，并且在整个函数执行过程中，logger 对象需要保持有效，不能提前销毁，否则可能会导致 TensorRT 相关操作出现问题。

```
#include "NvInfer.h"
#include <iostream>

class Logger : public nvinfer1::ILogger {
public:
    void log(Severity severity, const char* msg) noexcept override {
        // 这里只打印警告及更严重级别的信息
        if (severity <= Severity::kWARNING) {
            std::cout << msg << std::endl;
        }
    }
};

// 假设在某个函数中使用 Logger
void main() {
    Logger logger;
    nvinfer1::IBuilder* builder = nvinfer1::createInferBuilder(logger);
    // 后续使用 builder 创建网络等操作
    // 在函数结束前，builder 等对象需要正确释放内存，且在这期间 logger 对象不能被销毁
    delete builder;
}
```

4.2.2 使用 ixRT API 构建网络

1. 完成 Builder 创建后，搭建模型的第一步是创建 `NetworkDefinition` 对象。

```
// flag 可选 kSTRONGLY_TYPED
// 构建引擎时，根据操作符类型为每个中间和输出张量推断类型，并且在构建过程中严格遵循这些类型
uint32_t flag = 1U <<
    ↪ static_cast<uint32_t>(nvinfer1::NetworkDefinitionCreationFlag::kSTRONGLY_TYPED);
INetworkDefinition* network = builder->createNetworkV2(flag);
```

2. 通过指定输入 Tensor 的名称、数据类型和形状，将输入层添加至网络中。一个网络可以有多个输入：

```
auto data = network->addInput("input", datatype, Dims{1, 1});
```

3. 向网络中添加第一个 convolution layer。

```
std::array<float, 25> filter_data;
std::array<float, 1> bias_data;
nvinfer1::Weights weight{nvinfer1::DataType::kFLOAT, filter_data.data(),
    ↪ filter_data.size()};
nvinfer1::Weights bias{nvinfer1::DataType::kFLOAT, bias_data.data(),
    ↪ bias_data.size()};
```

```
auto conv1 = network->addConvolution( *data->getOutput(0), 1, Dims{5, 5}, weight,
    ↪ bias);
conv1->setStride(Dims{1, 1});
```

4. 添加 ReLU Activation layer。

```
auto relu1 = network->addActivation(*conv1->getOutput(0), ActivationType::kRELU);
```

5. 为 Activation 层的输出添加一个名称，以便在推理时为输出 Tensor 指定内存。

```
relu1->getOutput(0)->setName("output");
```

6. 标记 Activation 的输出作为网络的输出。

```
network->markOutput(*relu1->getOutput(0));
```

7. 获取网络的输入输出数量。

```
// 获取输入个数
auto input_count = network->getNbInputs();
// 获取输出个数
auto output_count = network->getNbOutputs();
```

8. 获取网络中的 layer 数量。

```
auto layer_count = network->getNbLayers();
```

4.2.3 使用 ONNX Parser 导入模型

ONNX Parser API 定义在 `NvOnnxParser.h` 头文件中的 `nvonnxparser` 命名空间中。

```
#include "NvOnnxParser.h"
using namespace nvonnxparser;
```

1. 通过如下方式创建 ONNX Parser 对象。

```
IParser* parser = createParser(*network, logger);
```

2. 解析模型文件并处理相关错误。

```
parser->parseFromFile(modelFile, static_cast<int32_t>(ILogger::Severity::kWARNING));
for (int32_t i = 0; i < parser->getNbErrors(); ++i) {
    std::cout << parser->getError(i)->desc() << std::endl;
}
```

Parser 对象必须在完成 Engine 创建后才可以销毁，network 中包含指向模型权重的指针，Builder 会将这些指针复制至优化后的 Engine 中。由于网络是使用解析器创建的，因此解析器拥有权重所占用的内存。因此，在完成 Engine 之前，请勿删除 Parser 对象。

至此，您已完成了 network 的搭建。有关如何构建 Engine 并使用此网络进行推理的说明，请参考[构建 Engine](#)和[执行推理](#)部分。

4.2.4 构建 Engine

在通过 ixRT 接口构建网络或使用 ONNX Parser 成功导入模型后，即可开始构建 Engine。

在 ixRT 中，`nvinfer1::IBuilderConfig` 接口在构建 TensorRT Engine 时起着关键作用，它允许用户对 Engine 构建过程进行各种配置，以平衡性能、精度和内存的使用。下面将详细介绍如何使用 C++ 接口来配置 `nvinfer1::IBuilderConfig`。

1. 创建构建配置对象。

```
IBuilderConfig* config = builder->createBuilderConfig();
```

2. 配置最大工作空间的大小，该空间大小指定了 TensorRT Engine 在构建和推理过程中可以使用的最大 GPU 内存量。配置精度模式，例如，FP32，FP16，INT8。

```
config->setFlag(BuilderFlag::kFP16);
```

3. 配置动态形状。

```
// 创建优化配置文件
IOptimizationProfile* profile = builder->createOptimizationProfile();
// 假设输入张量为 "input"
profile->setDimensions("input", OptProfileSelector::kMIN, Dims4(1, 3, 224, 224));
profile->setDimensions("input", OptProfileSelector::kOPT, Dims4(8, 3, 224, 224));
profile->setDimensions("input", OptProfileSelector::kMAX, Dims4(16, 3, 224, 224));
// 将优化配置文件添加到构建配置中
config->addOptimizationProfile(profile);
```

4. 完成 BuilderConfig 配置后，即可开始编译 Engine。

```
IHostMemory* engine = builder->buildSerializedNetwork(*network, *config);
```

5. 将 Engine 保存至文件中。

```
std::ofstream engineFile("engine_file", std::ios::binary);
if (!engineFile) {
    std::cerr << "ERROR: could not open engine_file for writing." << std::endl;
    return;
}
engine->serialize(engineFile);
engineFile.close();
```

6. 至此 Engine 已完成创建，释放相关资源。

```
delete parser;
delete network;
delete config;
delete builder;
```

4.2.5 运行时对象

Runtime

负责管理 TensorRT 的运行环境，包括反序列化生成 Engine 和提供硬件资源。生命周期通常与整个应用程序一致，但可以管理多个 Engine。

Engine

优化后的模型表示，包含网络结构和权重，不可变。Engine 由 Runtime 反序列化生成，用于创建多个 Context。

Context

执行推理的实例，管理推理时的中间状态 (如中间张量、CUDA 流)。一个 Engine 可以创建多个 Context，每个 Context 都有独立的工作空间，可以并行调用这些 Context。

依赖关系

上述运行时对象的依赖关系是：Runtime -> Engine -> Context，按照反向顺序销毁，避免悬空指针。

4.2.6 反序列化 Engine

当您有一个已序列化的优化模型，即可开始进行推理，首先需要创建一个运行时接口实例。

```
//创建 runtime
IRuntime* runtime = createInferRuntime(logger);
//反序列化 engine
std::vector<char> modelData = readModelFromFile("model.plan");
ICudaEngine* engine = runtime->deserializeCudaEngine(modelData.data(), modelData.size());
```

4.2.7 执行推理

1. Engine 中包含优化后的模型，创建 ExecutionContext 用于执行推理。

```
IExecutionContext *context = engine->createExecutionContext();
```

2. 分配输入和输出内存。

```
// 分配设备内存
void* d_input, *d_output;
cudaMalloc(&d_input, inputSize);
cudaMalloc(&d_output, outputSize); // outputSize 同理计算
```

3. 拷贝输入数据至 GPU。

```
std::vector<float> h_input(inputSize / sizeof(float));
// 填充 h_input 数据...
cudaMemcpyAsync(d_input, h_input.data(), inputSize, cudaMemcpyHostToDevice, stream);
```

4. 绑定输入输出的内存地址。

```
context->setTensorAddress(inputName, d_input);
context->setTensorAddress(outputName, d_output);
```

5. 执行推理。

```
cudaStream_t stream;
cudaStreamCreate(&stream);
context->enqueueV3(stream);
```

6. 将推理结果拷贝至主机内存并清理资源。

```
std::vector<float> h_output(outputSize / sizeof(float));
cudaMemcpyAsync(h_output.data(), d_output, outputSize, cudaMemcpyDeviceToHost,
    ⇨ stream);
// 等待流中所有操作完成
cudaStreamSynchronize(stream);
// 清理资源
cudaFree(d_input);
cudaFree(d_output);
cudaStreamDestroy(stream);
delete context;
delete engine;
delete runtime;
```

4.2.8 使用示例：ResNet18 模型推理

1. 编译。

```
// 编译 ResNet - 18 模型
// 1. 创建 Logger 对象
Logger logger;
// 2. 创建 Builder 对象
std::unique_ptr<IBuilder> builder(createInferBuilder(logger));

// 3. 创建 BuilderConfig 对象并配置
std::unique_ptr<IBuilderConfig> config(builder->createBuilderConfig());

// 4. 创建 NetworkDefinition 对象
std::unique_ptr<INetworkDefinition> network(builder->createNetworkV2(0));

// 5. 使用 ONNX Parser 解析模型
std::unique_ptr<IParser> parser(createParser(*network, logger));

parser->parseFromFile(onnxFilePath.c_str(),
    ⇨ static_cast<int32_t>(ILogger::Severity::kWARNING));
// 6. 构建引擎
std::unique_ptr<IHostMemory>
    ⇨ serializedModel(builder->buildSerializedNetwork(*network, *config));

// 7. 保存引擎到文件
std::ofstream engineFile("resnet18.engine", std::ios::binary);

engineFile.write(reinterpret_cast<const char*>(serializedModel->data()),
    ⇨ serializedModel->size());
```

```
engineFile.close();
// 8. 释放不再需要的对象
parser.reset();
network.reset();
config.reset();
builder.reset();
```

2. 推理。

```
// 1. 创建 Logger 对象
Logger logger;
// 2. 创建 Runtime 对象
std::unique_ptr<IRuntime> runtime(createInferRuntime(logger));

// 3. 从文件读取引擎数据
std::ifstream engineFile("resnet18.engine", std::ios::binary);

engineFile.seekg(0, std::ios::end);
size_t size = engineFile.tellg();
engineFile.seekg(0, std::ios::beg);
std::vector<char> engineData(size);
engineFile.read(engineData.data(), size);
engineFile.close();
// 4. 反序列化引擎
std::unique_ptr<ICudaEngine>
    engine(runtime->deserializeCudaEngine(engineData.data(), size));

// 5. 创建 ExecutionContext 对象
std::unique_ptr<IExecutionContext> context(engine->createExecutionContext());

// 6. 准备输入输出缓冲区
auto inputDims = engine->getBindingDimensions(0);
auto outputDims = engine->getBindingDimensions(1);
auto inputSize = GetBytes(inputDims, nvinfer1::DataType::kFLOAT);
auto outputSize = GetBytes(outputDims, nvinfer1::DataType::kFLOAT);
std::vector<void*> bindingBuffer(2, nullptr);
cudaMalloc(&bindingBuffer[0], inputSize)
cudaMalloc(&bindingBuffer[1], outputSize)
const std::string dirPath("data/resnet18/");
const std::string imagePath(dirPath + "kitten_224.bmp");
auto cpuFp32Image = LoadImageCPU(imagePath, inputDims);
cudaMemcpy(bindingBuffer[0], cpuFp32Image.get(), inputSize, cudaMemcpyHostToDevice);
// 7. 设置输入输出张量地址
context->setTensorAddress("inputTensorName", inputBuffer);
context->setTensorAddress("outputTensorName", outputBuffer);
// 8. 进行推理
cudaStream_t stream;
cudaStreamCreate(&stream);
context->enqueueV3(stream);
// 9. 同步等待推理完成
cudaStreamSynchronize(stream);
```

```
// 10. 将推理结果拷贝到主机
float cpu_output[outputSize/sizeof(float)];
cudaMemcpy(cpu_output.get(), bindingBuffer[1], outputSize, cudaMemcpyDeviceToHost));
cudaStreamDestroy(stream);
// 11. 释放资源
cudaFree(bindingBuffer[0]);
cudaFree(bindingBuffer[1]);
context.reset();
engine.reset();
runtime.reset();
```

4.3 Python API 文档

本节内容主要介绍 ixRT Python API 的使用方法。

您可以通过 tensorrt 模块访问 Python API。

```
import tensorrt as trt
```

4.3.1 构建阶段

如果要创建生成器 (builder)，必须首先创建日志记录器 (logger)。tensorrt Python 绑定接口提供了一个简单的 logger，用于将日志信息输出至终端 (stdout)。

```
logger = trt.Logger(trt.Logger.WARNING)
```

当然，您也可以实现自己的日志类，用于更加定制化的操作。

```
class MyLogger(trt.ILogger):
    def __init__(self):
        trt.ILogger.__init__(self)

    def log(self, severity, msg):
        pass # 实现您的日志记录方式
```

```
logger = MyLogger()
```

有了 logger 后，用其创建一个构造器 builder。这里 builder 和 logger 的生命周期需要保持一致，否则会导致功能异常。

```
builder = trt.Builder(logger)
```

Engine 构建通常作为离线且单独的一个步骤，用于和运行时的推理分开。构建一个 Engine 可能会花较久的时间，因此，建议您在推理环节直接使用构建好的 Engine 执行推理。

4.3.1.1 创建网络

创建构建器后，编译模型的第一步是通过 builder 创建网络定义，如参考如下代码传入 create_network 函数中的参数表示网络可在推理时动态调整输入的批量大小。**注意，network 和 builder 的生命周期需要保持一致。**

```
flag = 1 << (int)(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
network = builder.create_network(flag)
```

1. 手工搭建网络。

如下为使用 Python API 搭建的网络：

- 函数 add_input 对网络增加输入，依次传入名称、数据类型、形状信息等；
- 函数 add_elementwise 构建 elementwise 算子，依次传入输入，设置计算类型为求和；
- 之后可添加更多的 layer；
- elementwise.get_output(0) 用于获得其第一个输出 Tensor；
- network.mark_output 用于标记函数参数 Tensor 为网络输出。

```
import tensorrt as trt

logger = trt.Logger(trt.Logger.WARNING)
builder = trt.Builder(logger)
flag = 1 << (int)(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
network = builder.create_network(flag)
in_data_1 = network.add_input(name="input_1", dtype=trt.float32, shape=(22, 33, 44,
↪ 55))
in_data_2 = network.add_input(name="input_2", dtype=trt.float32, shape=(22, 33, 44,
↪ 55))
elementwise = network.add_elementwise(input1=in_data_1, input2=in_data_2,
↪ op=trt.ElementWiseOperation.SUM)
# 在这里可以增加更多的算子
network.mark_output(tensor=elementwise.get_output(0))
```

2. 导入模型搭建网络。

通过 trt.OnnxParser 构建 parser 解析器，例如，读取 /tmp/resnet18.onnx 路径下的模型进行网络构建。OnnxParser 在实现中实际调用手工搭建网络中介绍的 API 进行网络构建。

```
import tensorrt as trt
import os

logger = trt.Logger(trt.Logger.VERBOSE)
builder = trt.Builder(logger)
flag = 1 << (int)(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
network = builder.create_network(flag)
parser = trt.OnnxParser(network, logger)
onnx_model = os.path.join("/tmp", "resnet18.onnx")
parser.parse_from_file(onnx_model)
```

4.3.1.2 编译网络与序列化 Engine

函数 `create_builder_config` 返回编译配置实例，参考如下代码，通过其设置编译数据类型为 `float16` (设置 `trt.BuilderFlag.INT8` 则使用量化数据类型)；通过 `build_serialized_network` 函数构建 Engine 后，将文件保存至指定路径下。

```
build_config = builder.create_builder_config()
build_config.set_flag(trt.BuilderFlag.FP16)
plan = builder.build_serialized_network(network, build_config)
engine_file_path = os.path.join("/tmp", "resnet18.engine")
with open(engine_file_path, "wb") as f:
    f.write(plan)
```

4.3.2 运行阶段

运行阶段主要进行反序列化与准备数据后执行。

1. 通过 `tensorrt` 构建 `Runtime`；
2. `Runtime` 调用成员函数 `deserialize_cuda_engine`，从 Engine 文件反序列化后生成 Engine；
3. 通过 Engine 的成员函数 `create_execution_context` 构建执行上下文 Context；
4. 通过 `cudaMalloc` 接口准备设备端的输入输出内存空间，并调用函数 `cudaMemcpy` 对数据进行主机端和设备端的相互传递；
5. 调用 Context 的成员函数 `set_tensor_address`，将其设备端的地址空间向系统传递，之后调用 Context 的成员函数 `execute_async_v3` 进行执行，函数参数为 `stream`，并且需要对流进行 `cudaStreamSynchronize` 同步，执行未结束前，例如，使用 `cudaMemcpyAsync(xxx, stream)` 拷贝输出数据会被阻塞，直至推理结束为止。

同样，这里的 `Logger`, `Runtime`, `Engine` 和 `Context` 的生命周期需要保持一致，否则会导致异常。

```
import tensorrt as trt
import os
import cuda.cudart as cudart
import numpy as np

logger = trt.Logger(trt.Logger.WARNING)
engine_file_path = os.path.join("/tmp", "resnet18.engine")
with open(engine_file_path, "rb") as f:
    runtime = trt.Runtime(logger)
    engine = runtime.deserialize_cuda_engine(f.read())
    assert engine

context = engine.create_execution_context()
in_size = 1 * 3 * 224 * 224
out_size = 1 * 1000
err, in_addr = cudart.cudaMalloc(in_size * 4) #float
assert err == cudart.cudaError_t.cudaSuccess
context.set_tensor_address(engine.get_tensor_name(0), in_addr)
err, out_addr = cudart.cudaMalloc(out_size * 4) #float
```

```
assert err == cudart.cudaError_t.cudaSuccess
context.set_tensor_address(engine.get_tensor_name(1), out_addr)

input = np.random.rand(in_size).astype(np.float32)
output = np.zeros(out_size, dtype=float)

(err,) = cudart.cudaMemcpy(
    in_addr,
    input,
    input.size,
    cudart.cudaMemcpyKind.cudaMemcpyHostToDevice,
)
assert err == cudart.cudaError_t.cudaSuccess

err, stream = cudart.cudaStreamCreate()
assert err == cudart.cudaError_t.cudaSuccess
context.execute_async_v3(stream)
cudart.cudaStreamSynchronize(stream)

(err,) = cudart.cudaMemcpy(
    output,
    out_addr,
    output.size,
    cudart.cudaMemcpyKind.cudaMemcpyDeviceToHost,
)
assert err == cudart.cudaError_t.cudaSuccess

(err,) = cudart.cudaFree(in_addr)
assert err == cudart.cudaError_t.cudaSuccess
(err,) = cudart.cudaFree(out_addr)
assert err == cudart.cudaError_t.cudaSuccess
```

4.4 高级话题

本节内容具有一定技术性，适合 ixRT 的高阶开发者，话题包括：

- 强类型网络
- 弱类型网络
- 网络输入排布
- 空张量支持
- 输入数据监控

4.4.1 版本兼容性说明

不同 ixRT 大版本之间，引擎文件 .engine 通常情况下不兼容，同一个大版本内，有不兼容的可能。因此，为尽可能减少报错，建议您使用同一版本的 ixRT 构建和加载引擎文件。

4.4.2 强类型网络

4.4.2.1 概述

强类型网络 (Strongly Typed Network) 指 ixRT 构建引擎时，根据网络输入类型静态推导后续中间层的输出类型。例如，网络输入是 PF16，后续算子如没有特殊类型限制，输出为 FP16。和强类型网络相对的是弱类型网络，将在下小节内容进行介绍。

强类型网络主要特点包括：

- **显式类型标注**：每个张量和层都有明确的类型标注
- **编译时类型检查**：在网络构建阶段就能发现类型不匹配等问题
- **减少隐式转换**：避免不必要的精度转换，提高推理效率
- **更好的可维护性**：类型信息使网络结构更清晰易懂

强类型网络的优势有：

- **安全性**：提前发现类型错误，避免运行时错误
- **性能优化**：减少不必要的类型转换操作
- **代码清晰**：明确的类型信息使网络结构更易理解
- **调试友好**：类型错误能在编译阶段被发现

Note

注意，如果使用强类型网络，对算子的精度控制 (setPrecision, setLayerOutType) 和对网络的精度控制均不生效。

4.4.2.2 使用强类型网络 API

您可以使用如下接口创建网络，后续编译时会按照强类型规则选择算法。

C++

```
IBuilder* builder = ...;
INetworkDefinition* network = builder->createNetworkV2(1U <<
    static_cast<uint32_t>(NetworkDefinitionCreationFlag::kSTRONGLY_TYPED)))
```

Python

```
builder = trt.Builder(...)
builder.create_network(1 << int(trt.NetworkDefinitionCreationFlag.STRONGLY_TYPED))
```

4.4.2.3 使用示例 - ResNet18

这里以 ResNet18 模型为例，介绍强类型网络在推理中的使用。

1. 首先，使用 PyTorch 导出一个 ResNet18 onnx 文件。

```
import torch
import torchvision.models as models

def export_resnet18_to_onnx(output_path="resnet18.onnx"):
    # 加载预训练的 ResNet18 模型
    model = models.resnet18(pretrained=True)
    model.eval()

    # 创建示例输入张量 (明确指定 batch size=1, 3 通道, 224x224)
    dummy_input = torch.randn(1, 3, 224, 224)

    # 导出模型为 ONNX
    torch.onnx.export(
        model,                    # 要导出的模型
        dummy_input,              # 模型输入
        output_path,              # 输出文件路径
        input_names=["input"],    # 输入节点名称
        output_names=["output"],  # 输出节点名称
        dynamic_axes={
            'input': {0: 'batch_size'}, # 动态 batch size
            'output': {0: 'batch_size'}
        },
        opset_version=11,         # ONNX 算子集版本
        do_constant_folding=True, # 优化常量折叠
        export_params=True,       # 导出训练好的参数
        verbose=True              # 打印导出过程信息
    )
    print(f"Model exported to {output_path}")

# 执行导出
export_resnet18_to_onnx()
```

2. 加载 onnx 文件并构建一个 FP32 的强类型网络。

```
import IxRT as trt

def print_engine_io_info(engine):
    # 如果传入的是文件路径, 先反序列化
    if isinstance(engine, str):
        with open(engine, 'rb') as f:
            runtime = trt.Runtime(trt.Logger(trt.Logger.WARNING))
            engine = runtime.deserialize_cuda_engine(f.read())

    # 数据类型映射
    dtype_map = {
        trt.DataType.FLOAT: 'FP32',
        trt.DataType.HALF: 'FP16',
        trt.DataType.INT8: 'INT8',
        trt.DataType.INT32: 'INT32',
        trt.DataType.BOOL: 'BOOL'
    }
```

```

print("\nEngine IO Information:")
print("{:<20} {:<15} {:<10}".format("Name", "Data Type", "Shape"))
print("-" * 55)

for i in range(engine.num_io_tensors):
    name = engine.get_tensor_name(i)
    dtype = dtype_map.get(engine.get_tensor_dtype(name), 'UNKNOWN')
    shape = tuple(engine.get_tensor_shape(name))
    print("{:<20} {:<15} {}".format(name, dtype, shape))

def build_engine(onnx_file_path, engine_file_path=""):
    # 初始化 logger
    logger = trt.Logger(trt.Logger.VERBOSE)

    # 创建 builder
    builder = trt.Builder(logger)

    # 创建网络定义
    network = builder.create_network(1 <<
    int(trt.NetworkDefinitionCreationFlag.STRONGLY_TYPED))

    # 创建 parser 用于解析 ONNX
    parser = trt.OnnxParser(network, logger)

    # 解析 ONNX 文件
    with open(onnx_file_path, "rb") as model:
        if not parser.parse(model.read()):
            print("Failed to parse the ONNX file.")
            for error in range(parser.num_errors):
                print(parser.get_error(error))
            return None

    # 配置 builder
    config = builder.create_builder_config()

    # 构建 engine
    plan = builder.build_serialized_network(network, config)

    if not plan:
        raise Exception("Failed to build the engine.")

    # 保存 engine 到文件
    if engine_file_path:
        with open(engine_file_path, "wb") as f:
            f.write(plan)
        print(f"Engine saved to {engine_file_path}")

    print("Engine built successfully!")

```

```
def main():
    # 输入 ONNX 文件路径
    onnx_file_path = "resnet18.onnx" # 使用 FP32 的 ONNX 模型

    # 输出 engine 文件路径
    engine_file_path = "resnet18_fp32_input_strongly_typed.engine"

    # 构建 engine
    build_engine(
        onnx_file_path=onnx_file_path,
        engine_file_path=engine_file_path
    )
    print_engine_io_info(engine_file_path)

if __name__ == "__main__":
    main()
```

3. 输出结果。

```
Engine saved to resnet18_fp32_input_strongly_typed.engine
Engine built successfully!
```

Engine IO Information:

Name	Data Type	Shape
input	FP32	(1, 3, 224, 224)
output	FP32	(1, 1000)

4.4.3 弱类型网络

4.4.3.1 概述

弱类型网络 (Weakly Typed Network) 指 ixRT 在编译优化过程中为提升性能，对算子选择合适的计算精度执行。默认情况下，ixRT 使用 FP32 计算，但您可以进行多种精度控制，使得网络以降精度运行，发挥最大性能。

弱类型网络的主要特点包括：

- **动态类型适配**：张量和算子无需固定数据类型，可根据硬件和优化策略自动选择最佳精度 (FP32/FP16/INT8)。例如，输入为 FP32，中间层自动转为 FP16 计算，输出层恢复为 FP32
- **混合精度支持**：允许同一网络中混合使用 FP32、FP16 和 INT8，无需手动插入类型转换算子。相比之下，强类型网络需显式添加 Cast 算子，增加计算开销
- **逐层精度覆盖**：用户可全局设置默认精度 (例如，FP16)，并针对特定层强制指定精度 (例如，保留 FP32)
- **弱化模型预设精度限制**：为达最优性能，ixRT 在编译期间为张量重新分配精度，无需严格遵守原始模型预定的精度限制。例如，常见导出的 ONNX 是 FP32 精度，但内部可以决策成更低的精度，如 FP16

弱类型网络的优势有：

- **更高的推理性能**：自动混合精度 (FP16/INT8) 可提升计算吞吐，显存占用最多可减少 50% 至 75%
- **灵活的精度控制**：关键层 (例如，注意力机制) 可保留 FP32，其余层将为 FP16/INT8，平衡速度与精度。相比之下，强类型网络需预先固化类型，灵活性低

- **简化部署流程**：无需在训练框架中预先量化或插入类型转换，ixRT 会自动优化，常用于 FP32 训练模型直接部署为 FP16/INT8 推理

4.4.3.2 粗粒度精度控制

降精度运行取决于您的硬件架构，在 ixRT 中，您可以使用如下方法查询精度支持情况。

C++

```
if (builder->platformHasFastFp16()) { ... }
```

Python

```
if builder.platform_has_fp16:
```

如果支持，您可以通过配置项指定网络的最低运行精度。

C++

```
builderConfig->setFlag(BuilderFlag::kFP16);
```

Python

```
config.set_flag(trt.BuilderFlag.FP16)
```

4.4.3.3 细粒度精度控制

此外，您还可以更具体地指定某一算子，或是算子的输出为某一精度。通过这样的显式指定，ixRT 在算法选择时会基于您的精度配置，选取最合适的算法。

您可以在搭建网络时设置某一层的运行精度。

C++

```
layer->setPrecision(DataType::kFP16)
```

Python

```
layer.precision = trt.fp16
```

此外，您还可以指定某一算子的输出精度。

C++

```
layer->setOutputType(out_tensor_index, DataType::kFLOAT)
```

Python

```
layer.set_output_type(out_tensor_index, trt.fp32)
```

如何生效用户指定精度？

默认情况下，弱类型网络基于运行速度选择算法。如果您希望指定的精度生效，必须设置下列两个选项中的一个：

- kPREFER_PRECISION_CONSTRAINTS：优先选择用户指定的精度实现，如果无法满足，ixRT 会依据性能优先原则选择其它实现，不会给出编译错误
- kOBEY_PRECISION_CONSTRAINTS：强制 ixRT 使用用户指定的精度实现，如果无法满足，给出编译错误

C++

```
config->setFlag(BuilderFlag::kPREFER_PRECISION_CONSTRAINTS)
// 或
config->setFlag(BuilderFlag::kOBEY_PRECISION_CONSTRAINTS);
```

Python

```
config.set_flag(trt.BuilderFlag.PREFER_PRECISION_CONSTRAINTS)
# 或
config.set_flag(trt.BuilderFlag.OBEY_PRECISION_CONSTRAINTS)
```

4.4.3.4 使用示例

如**使用示例 - ResNet18**所示，如果希望：

模型以 FP16 弱类型网络运行，第一个算子以 FP32 精度计算，内部如果没有该实现则编译退出

那么可以增加如下代码：

```
network.get_layer(0).precision = trt.float16
config.set_flag(trt.BuilderFlag.FP16)
config.set_flag(trt.BuilderFlag.OBEY_PRECISION_CONSTRAINTS)
```

4.4.4 网络输入排布

截止 ixRT v1.0.0 版本，仅支持输入排布为 `TensorFormat::kLINEAR`，其它输入排布暂不支持。

4.4.5 空张量支持

在 ixRT 中，空张量 (Empty Tensor) 是指形状中至少有一个维度为 0 的张量。如果张量形状长度为 0，一般没有特殊处理。空张量通常出现在动态形状推理中，某些维度可能临时为 0。

如果一个引擎的输入或输出是空张量，也需要为其分配内存，类似 C++ 中的 `new float[0]` 返回非空指针。

4.4.6 输入数据监控

ixRT 提供 `IExecutionContext::enqueueV2` 异步执行接口，用于异步发射模型所有 kernel 调用，并监控模型输入的消费情况。基于此，您可以设计一个高效的监控输入数据的推理流程，即在模型运行结束之前加载数据，可以降低数据加载和模型执行做成并行的流水，减少发射 kernel 中的间隔时间。

下面提供一个示例：

```
// ...
// 创建 engine、context

// 创建监控输入是否被消费的 cuda event
cudaEvent_t input_consumed_event;
cudaEventCreate(&input_consumed_event);

cudaStream_t stream = 0;

// 第一次装载 IO bindings
// 1. 在 stream 0 上发射 kernel
// 2. 用 input_consumed_event 指示输入是否被完全消费
void *input_gpu, *output_gpu;
void *bindings[] = {input_gpu, input_gpu, output_gpu};
context->enqueueV2(bindings, stream, &input_consumed_event);
// 如果输入已被完全使用，这里会继续向下运行，否则会在此等待
cudaEventSynchronize(input_consumed_event);

// 第二次装载 IO bindings
// 1. 在 stream 0 上发射 kernel
// 2. 用 input_consumed_event 指示输入是否被完全消费
// 3. 此时第一次装载发射的 kernel 可能还正在执行，这里继续发射后续 kernel，
//    在同一个 stream 上保证连续性，节省时间
void *input_gpu1, *output_gpu1;
void *bindings1[] = {input_gpu1, input_gpu1, output_gpu1};
context->enqueueV2(bindings1, stream, &input_consumed_event);

// 两次执行都将在此结束
CHECK(cudaStreamSynchronize(0) == cudaSuccess);
// 接下来可以访问两次执行的结果
// visit (output_gpu);
// visit (output_gpu1);
```

您还可以使用 `setTensorAddress` + `setInputConsumedEvent` + `enqueueV3` 完成同样的功能，如下所示：

```
// 输入：使用 setTensorAddress 设置输入输出地址
context->setTensorAddress(input_name, input_gpu_address);
// 监测输入是否被消费的 event
context->setInputConsumedEvent(input_consumed_event);
// 发射所有 kernel
context->enqueueV3(stream);
```

```
// 当该行运行结束时，输入已经被全部消费完成
cudaEventSynchronize(input_consumed_event);
// 在此可以继续加载输入，覆写输入地址，或者加载新的输入地址
// 输出需要在 stream 同步完之后，才能获取到结果
```

4.5 量化推理指南

ixRT 支持使用低精度数据类型 (例如, INT8) 表示高精度数值 (例如, float32), 这可以在保证模型准确度的前提下, 大幅减少模型参数和推理时激活值的存储空间, 并利用整数计算加速推理过程。

Note

当前仅支持 INT8 对称量化。

4.5.1 量化原理说明

对称量化假设零点 (zero_point) 固定为 0, 量化区间关于零中心对称。量化区间 (INT8) 为:

$$x_q \in [-128, 127]$$

计算 Scale

per-tensor 量化是整个张量 (tensor) 共享一组量化参数 (scale), per-channel (linear 层为 per-column) 量化是对张量的每一个通道单独计算量化参数 (scale)。

$$scale = \frac{\max(abs(x_{min}^{ch/tensor}), abs(x_{max}^{ch/tensor}))}{127}$$

量化公式 (浮点 -> 整数)

$$x_q = \text{quantize}(x, s) = \text{roundingHalfToEven}\left(\text{clip}\left(\frac{x}{s}, -128, 127\right)\right)$$

反量化公式 (整数 -> 浮点)

$$x = \text{dequantize}(x_q, s) = x_q * s$$

4.5.2 隐式量化与显式量化

网络可以通过两种 (互斥的) 量化方式进行处理, 分别是隐式量化 (Implicit) 和显式量化 (Explicit)。

这两种处理模式的主要区别在于:

您是否需要量化进行显式控制, 或者让 ixRT 构建器自动选择待量化的操作和张量 (隐式)。

- 当网络中具有 QuantizeLayer 和 DequantizeLayer 层时, ixRT 使用显式量化模式;
- 当网络中没有 QuantizeLayer 或 DequantizeLayer 层, 并且在构建器配置中启用了 INT8 时, ixRT 使用隐式量化模式。

在隐式量化模式下，

- 您需要使用 setDynamicRange API 设置每个可被量化的激活张量的数值范围 (最小值与最大值), ixRT 构建期间将计算激活张量的量化系数, 无需为 Weights 设置数值范围, ixRT 构建期间将自动计算 Weights 的量化系数。
- 在处理隐式量化的网络时, ixRT 将模型视为浮点模型, 并在算法选择时测试各个层在各种精度下的运行速度。
- 如果某个层在 INT8 中运行速度更快, 并且该层输入和输出数据已被设置范围, 则会将具有 INT8 精度的计算 kernel 分配给该层; 否则, 将分配高精度浮点 (FP32 或 FP16) 计算 kernel。
- 如果需要高精度浮点来保证精度, 而牺牲性能, 便可以使用 Layer::setPrecision API 来指定该层的运行精度。

在显式量化模式下，

- 量化和反量化操作在图中由 IQuantizeLayer 和 IDequantizeLayer 节点显式表示, 以下将这些节点简称为 Q/DQ 节点。
- 与隐式量化相比, 显式量化明确指定了与量化类型之间转换发生的位置, 在构建期间, ixRT 优化器将模型语义所要求的量化计算区域转换为 INT8 精度计算层。
- 当 ixRT 检测到网络中存在 Q/DQ 节点时, 它会直接使用显式量化处理逻辑构建引擎, 而无需在构建器配置中启用精度控制标志。

4.5.2.1 隐式量化推理流程

1. 创建 Network。

- 使用 onnx-parser 导入高精度浮点 onnx 模型。

```
const auto explicitBatch = 1U << static_cast<uint32_t>(nvinfer1::
    ↪ NetworkDefinitionCreationFlag::kEXPLICIT_BATCH);
INetworkDefinition* network =
    ↪ UPtr<nvinfer1::INetworkDefinition>(builder->createNetworkV2(explicitBatch));
IParser* parser = createParser(*network, logger);
std::string onnxFilePath = "<path-to-onnx-file>";
parser->parseFromFile(onnxFilePath.c_str(),
    ↪ static_cast<int>(logger.getReportableSeverity()));
```

- 使用网络搭建 API, 从零开始创建网络, 以 Convolution+Relu 简单网络为例。

```
const auto explicitBatch = 1U << static_cast<uint32_t>(nvinfer1::
    ↪ NetworkDefinitionCreationFlag::kEXPLICIT_BATCH);
INetworkDefinition* network =
    ↪ UPtr<nvinfer1::INetworkDefinition>(builder->createNetworkV2(explicitBatch));
nvinfer1::Dims input_dims{4, {1, 3, 224, 224}};
auto input_data = network->addInput("input_data", nvinfer1::DataType::kFLOAT,
    ↪ input_dims);
std::vector<float> kernel_64x3x7x7_data(64*3*7*7, 0.1);
std::vector<float> bias_data(64, 0.1);
nvinfer1::Weights weights{nvinfer1::DataType::kFLOAT,
    ↪ kernel_64x3x7x7_data.data(), kernel_64x3x7x7_data.size()};
nvinfer1::Weights bias{nvinfer1::DataType::kFLOAT, bias_data.data(),
    ↪ bias_data.size()};
```

```
nvinfer1::Dims kernel_size_7x7{2, {7, 7}};
auto conv_layer = network->addConvolutionNd(*input_data, 64, kernel_size_7x7,
    ↪ weights, bias);
conv_layer->setPaddingNd({2, {3, 3}});
conv_layer->setStrideNd({2, {2, 2}});
auto act_layer = network->addActivation(*conv_layer->getOutput(0),
    ↪ ActivationType::kRELU);
act_layer->getOutput(0)->setName("relu_out");
network->markOutput(*act_layer->getOutput(0));
```

2. 使用 setDynamicRange API 设置每个可被量化的激活张量的数值范围 (最小值与最大值), 无需为 Weights 设置数值范围。

```
auto input_tensor = conv_layer->getInput(0);
input_tensor->setDynamicRange(/*min*/ -1.2, /*max*/ 1.4);
auto output_tensor = conv_layer->getOutput(0);
output_tensor->setDynamicRange(/*min*/ -2.3, /*max*/ 2.1);
```

3. 构建器配置中启用 INT8。

```
IBuilderConfig* config = builder->createBuilderConfig();
config->setFlag(BuilderFlag::kINT8);
```

4. 开始构建网络。

```
IHostMemory* engine = builder->buildSerializedNetwork(*network, *config);
```

5. 执行推理。

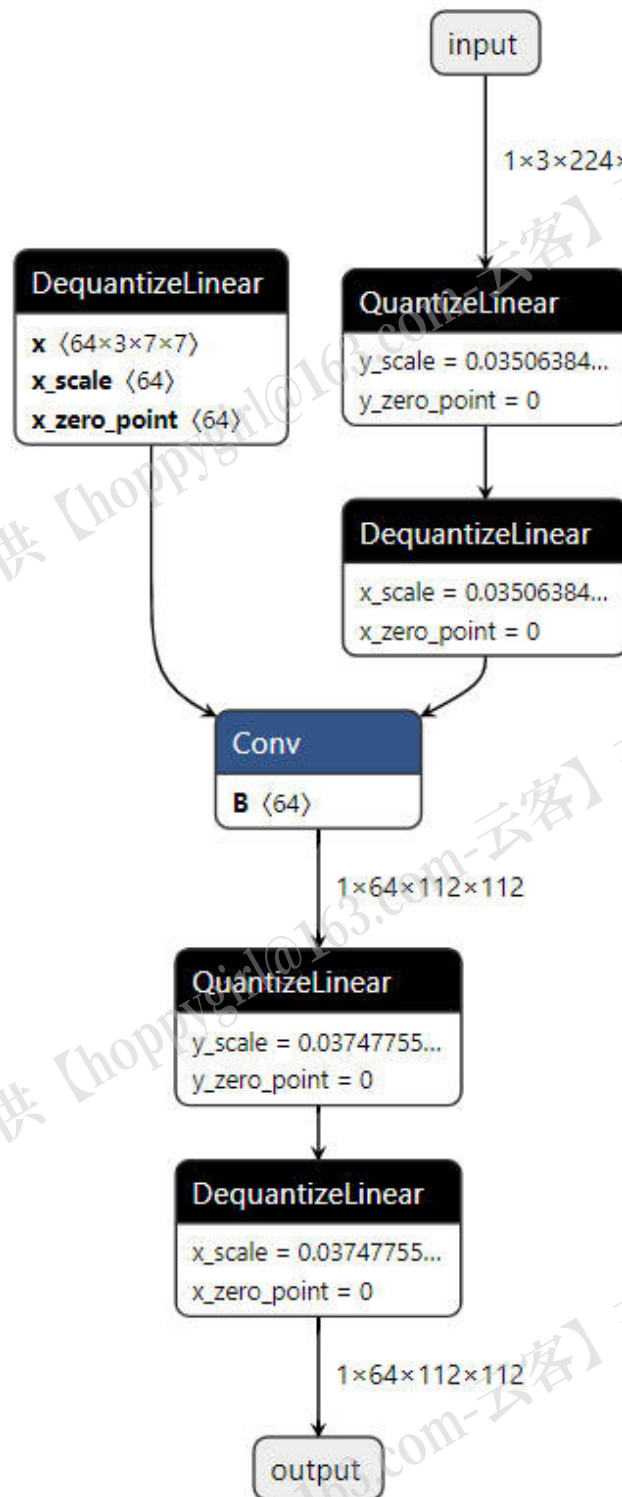
详情请参考“C++ API 文档”。

ixRT 提供一个 ResNet18 隐式量化的推理流程示例, 详情请参考 ixRT 安装路径下 IxRT/samples/sampleResNet 目录 (增加 **tex_i8_implicit** 参数即表示隐式量化推理)。

4.5.2.2 显式量化推理流程

1. 创建 Network。

- 使用 onnx-parser 导入包含 QuantizeLinear/DequantizeLinear 节点的 onnx 模型。



代码示例如下：

```

const auto explicitBatch = 1U << static_cast<uint32_t>(nvinfer1::
    ↪ NetworkDefinitionCreationFlag::kEXPLICIT_BATCH);
INetworkDefinition* network =
    ↪ UPtr<nvinfer1::INetworkDefinition>(builder->createNetworkV2(explicitBatch));
  
```

```
IParser* parser = createParser(*network, logger);
std::string onnxFilePath = "<path-to-qdq-onnx-file>";
parser->parseFromFile(onnxFilePath.c_str(),
    ↪ static_cast<int>(logger.getReportableSeverity()));
```

- 使用网络搭建 API，从零开始创建网络，以 Convolution 简单网络为例。

```
const auto explicitBatch = 1U << static_cast<uint32_t>(nvinfer1::
    ↪ NetworkDefinitionCreationFlag::kEXPLICIT_BATCH);
INetworkDefinition* network =
    ↪ UPtr<nvinfer1::INetworkDefinition>(builder->createNetworkV2(explicitBatch));
nvinfer1::Dims input_dims{4, {1, 3, 224, 224}};
auto input_data = network->addInput("input_data", nvinfer1::DataType::kFLOAT,
    ↪ input_dims);

std::vector<float> scale_0_data{0.03506};
nvinfer1::Weights scale_0_weights{nvinfer1::DataType::kFLOAT,
    ↪ scale_0_data.data(), scale_0_data.size()};
auto scale_0_constant_layer = network->addConstant(Dims{1, {1}},
    ↪ scale_0_weights);
auto quantize_0_layer = network->addQuantize(*input_data,
    ↪ *scale_0_constant_layer->getOutput(0), DataType::kINT8);
auto dequantize_0_layer =
    ↪ network->addDequantize(*quantize_0_layer->getOutput(0),
    ↪ *scale_0_constant_layer->getOutput(0), DataType::kFLOAT);
quantize_0_layer->setAxis(0);
dequantize_0_layer->setAxis(0);

std::vector<int8_t> kernel_64x3x7x7_data(64*3*7*7, 1);
nvinfer1::Weights filter_weights{nvinfer1::DataType::kINT8,
    ↪ kernel_64x3x7x7_data.data(), kernel_64x3x7x7_data.size()};
auto filter_constant_layer = network->addConstant(Dims{4, {64, 3, 7, 7}},
    ↪ filter_weights);
std::vector<float> scale_1_data(64, 0.01);
nvinfer1::Weights scale_1_weights{nvinfer1::DataType::kFLOAT,
    ↪ scale_1_data.data(), scale_1_data.size()};
auto scale_1_constant_layer = network->addConstant(Dims{1, {64}},
    ↪ scale_1_weights);
auto dequantize_1_layer =
    ↪ network->addDequantize(*filter_constant_layer->getOutput(0),
    ↪ *scale_1_constant_layer->getOutput(0), DataType::kHALF);
dequantize_1_layer->setAxis(0);

std::vector<float> bias_data(64, 0.1);
nvinfer1::Weights bias{nvinfer1::DataType::kFLOAT, bias_data.data(),
    ↪ bias_data.size()};
nvinfer1::Dims kernel_size_7x7{2, {7, 7}};
nvinfer1::Weights empty_weights{nvinfer1::DataType::kFLOAT, nullptr, 0};
auto conv_layer = network->addConvolutionNd(*input_data, 64, kernel_size_7x7,
    ↪ empty_weights, bias);
```

```
conv_layer->setInput(1, *dequantize_1_layer->getOutput(0));
conv_layer->setPaddingNd({2, {3, 3}});
conv_layer->setStrideNd({2, {2, 2}});

std::vector<float> scale_2_data{0.03747};
nvinfer1::Weights scale_2_weights{nvinfer1::DataType::kFLOAT,
    ↪ scale_2_data.data(), scale_2_data.size()};
auto scale_2_constant_layer = network->addConstant(Dims{1, {1}},
    ↪ scale_2_weights);
auto quantize_2_layer = network->addQuantize(*conv_layer->getOutput(0),
    ↪ *scale_2_constant_layer->getOutput(0), DataType::kINT8);
auto dequantize_2_layer =
    ↪ network->addDequantize(*quantize_2_layer->getOutput(0),
    ↪ *scale_2_constant_layer->getOutput(0), DataType::kFLOAT);
quantize_2_layer->setAxis(0);
dequantize_2_layer->setAxis(0);
network->markOutput(*dequantize_2_layer->getOutput(0));
```

2. 开始构建网络。

```
IHostMemory* engine = builder->buildSerializedNetwork(*network, *config);
```

3. 执行推理。

详情请参考“C++ API 文档”。

ixRT 提供一个 ResNet18 显式量化的推理流程示例，详情请参考 ixRT 安装路径下 IxRT/samples/sampleResNet 目录 (增加 **tex_i8_explicit** 参数即表示显式量化推理)。

4.5.3 量化工具说明

建议您使用 ixRT Python 包中的 Deploy 量化工具进行量化，从而在天数智芯加速卡上进行高效推理。当然，您也可以使用第三方量化工具进行量化，结合使用 ixRT 在天数智芯加速卡上进行推理。ixRT 使用 QDQ 格式的 ONNX 进行 INT8 推理，请确保您在 INT8 推理情况下使用 QDQ 格式。

以下是使用 ixRT Deploy 量化工具对 ResNet18 进行 PTQ 量化并导出 QDQ ONNX 的示例。

```
# !/usr/bin/env python
# -*- coding: utf-8 -*-
import argparse
from os.path import dirname, join
import torch
from ixrt.deploy.api import static_quantize

class PTQQuantization:
    def __init__(self, model):
        self.dataloader = torch.randn(2, 1, 3, 224, 224).cuda() # 这里使用随机数据，可以使用真
        ↪ 实数据作为校准数据集
        self.model = model
```

```
def run(self, save_quant_onnx_path):
    graph = static_quantize(
        model=self.model,
        calibration_dataloader=self.dataloader,
        save_quant_onnx_path=save_quant_onnx_path,
        quant_format="qdq",
    )

def parse_args():
    file = join(
        dirname(__file__),
        "<float-resnet18-onnx-file-path>",
    )
    save_model_path = join(
        dirname(__file__),
        "<qdq-resnet18-onnx-file-save-path>",
    )
    parser = argparse.ArgumentParser("PTQ quantization")
    parser.add_argument("--model", type=str, default=file)
    parser.add_argument("--save_model_path", type=str, default=save_model_path)

    config = parser.parse_args()
    return config

if __name__ == "__main__":
    config = parse_args()
    quantizer = PTQQuantization(model=config.model)
    quantizer.run(
        save_quant_onnx_path=config.save_model_path,
    )
    print("Done, quantized onnx lies on", config.save_model_path)
```

4.6 动态图形推理指南

动态形状表达会以以下两种形式出现：

- 输入动态形状，即用户主动调整输入 Tensor 的形状
- 输出动态形状，即输出数据的动态形状由输入数据的内容决定，无法根据输入进行形状分析获取

这两种情况并不是互斥的，在一个模型推理过程中，可能会同时出现输入动态形状和输出动态形状。

4.6.1 输入动态形状

输入动态形状特性允许用户在运行时更改输入张量的形状，而无需重新构建模型。这对于处理不同大小的输入数据非常有用。

如果要使用动态形状的能力，需要进行如下操作：

- 编译阶段:

- 对于输入 Tensor, 设定运行时要动态调整的维度, 数值为-1。
- 通过 Builder 创建 OptimizationProfile 对象, 配置动态形状的范围: 最大值、最小值和最优值, 然后将这个对象添加至 BuilderConfig 对象中。

- 运行时:

- 调用 ExecutionContext 的 setInputShape 接口, 设定输入 Tensor 的形状, 这时也可以访问输出 Tensor 的形状。
- 调用 ExecutionContext 的 enqueue / execute 接口, 执行推理。

下面使用一个实际的例子说明如何使用动态形状的能力, 并说明其中的关键组件配置要点。

```
// 用户自定义 log 接口
auto& logger = Logger();
const auto explicitBatch = 1U <<
    ↪ static_cast<uint32_t>(nvinfer1::NetworkDefinitionCreationFlag::kEXPLICIT_BATCH);
auto builder = nvinfer1::createInferBuilder(logger);
auto network = builder->createNetworkV2(explicitBatch);

std::string input_name_1("input_1");
std::string input_name_2("input_2");
std::string output_name("output");
auto input_1 = network->addInput(input_name_1.c_str(), nvinfer1::DataType::kFLOAT,
    ↪ nvinfer1::Dims{3, {-1, 3, 4}});
auto input_2 = network->addInput(input_name_2.c_str(), nvinfer1::DataType::kFLOAT,
    ↪ nvinfer1::Dims{3, {-1, 3, 4}});
auto elementwise_layer = network->addElementWise(*input_1, *input_2,
    ↪ nvinfer1::ElementWiseOperation::kSUM);
elementwise_layer->setName(output_name.c_str());
network->markOutput(*elementwise_layer->getOutput(0));

auto config = builder->createBuilderConfig();
auto profile = builder->createOptimizationProfile();
profile->setDimensions(input_name_1.c_str(), nvinfer1::OptProfileSelector::kMIN,
    ↪ nvinfer1::Dims{3, {1, 3, 4}});
profile->setDimensions(input_name_1.c_str(), nvinfer1::OptProfileSelector::kOPT,
    ↪ nvinfer1::Dims{3, {8, 3, 4}});
profile->setDimensions(input_name_1.c_str(), nvinfer1::OptProfileSelector::kMAX,
    ↪ nvinfer1::Dims{3, {32, 3, 4}});

profile->setDimensions(input_name_2.c_str(), nvinfer1::OptProfileSelector::kMIN,
    ↪ nvinfer1::Dims{3, {1, 3, 4}});
profile->setDimensions(input_name_2.c_str(), nvinfer1::OptProfileSelector::kOPT,
    ↪ nvinfer1::Dims{3, {8, 3, 4}});
profile->setDimensions(input_name_2.c_str(), nvinfer1::OptProfileSelector::kMAX,
    ↪ nvinfer1::Dims{3, {32, 3, 4}});

config->addOptimizationProfile(profile);
auto plan = builder->buildSerializedNetwork(*network, *config);
```

```

auto runtime = nvinfer1::createInferRuntime(logger);
auto engine = runtime->deserializeCudaEngine(plan->data(), plan->size());
auto context = engine->createExecutionContext();
void* input_1_buffer = nullptr;
void* input_2_buffer = nullptr;
void* output_buffer = nullptr;
context->setTensorAddress(input_name_1.c_str(), input_1_buffer);
context->setTensorAddress(input_name_2.c_str(), input_2_buffer);
context->setTensorAddress(output_name.c_str(), output_buffer);
/*
 * 这里可以使用 cudaMalloc 等接口，分配显存，然后使用 cudaMemcpy 等接口，将输入数据拷贝到显存中
 */
context->setInputShape(input_name_1.c_str(), nvinfer1::Dims{3, {4, 3, 4}});
context->setInputShape(input_name_2.c_str(), nvinfer1::Dims{3, {4, 3, 4}});
context->enqueueV3(nullptr);
/*
 * 这里可以使用 cudaMemcpy 等接口，将输出数据从显存中拷贝到主机内存中
 */
delete builder;
delete network;
delete config;
delete plan;
delete runtime;
delete engine;
delete context;
/*
 * 这里可以使用 cudaFree 等接口，释放显存
 */

```

上述示例创建了一个简单的网络，其中包含两个输入张量和一个元素相加的操作。构建模型输入时，使用-1 来标记动态形状的维度，如下代码所示：

```

auto input_1 = network->addInput(input_name_1.c_str(), nvinfer1::DataType::kFLOAT,
    ↪ nvinfer1::Dims{3, {-1, 3, 4}});
auto input_2 = network->addInput(input_name_2.c_str(), nvinfer1::DataType::kFLOAT,
    ↪ nvinfer1::Dims{3, {-1, 3, 4}});

```

接下来需要创建 OptimizationProfile 对象，配置动态形状的范围。

```

auto profile = builder->createOptimizationProfile();
profile->setDimensions(input_name_1.c_str(), nvinfer1::OptProfileSelector::kMIN,
    ↪ nvinfer1::Dims{3, {1, 3, 4}});
profile->setDimensions(input_name_1.c_str(), nvinfer1::OptProfileSelector::kOPT,
    ↪ nvinfer1::Dims{3, {8, 3, 4}});
profile->setDimensions(input_name_1.c_str(), nvinfer1::OptProfileSelector::kMAX,
    ↪ nvinfer1::Dims{3, {32, 3, 4}});

profile->setDimensions(input_name_2.c_str(), nvinfer1::OptProfileSelector::kMIN,
    ↪ nvinfer1::Dims{3, {1, 3, 4}});

```

```
profile->setDimensions(input_name_2.c_str(), nvinfer1::OptProfileSelector::kOPT,
    ↪ nvinfer1::Dims{3, {8, 3, 4}});
profile->setDimensions(input_name_2.c_str(), nvinfer1::OptProfileSelector::kMAX,
    ↪ nvinfer1::Dims{3, {32, 3, 4}});
```

由于两个输入都有动态形状的维度，因此需要创建两个 OptimizationProfile 对象，分别配置两个输入的动态形状范围。

需要注意的是，输入张量只有第一个维度是动态的，其它维度都是固定的。

因此，第一个维度记录的动态范围要求 $\min \leq \text{opt} \leq \max$ ，其它维度的范围，要求 $\min == \text{opt} == \max$ 。其中 opt 是推理过程中对性能要求最高的尺寸，在算法选择阶段会针对这个尺寸选择性能最佳的算子。

将 OptimizationProfile 对象添加至 BuilderConfig 对象中。

```
config->addOptimizationProfile(profile);
```

在运行时使用 ExecutionContext 的 setInputShape 接口，设定输入 Tensor 的形状。

```
context->setInputShape(input_name_1.c_str(), nvinfer1::Dims{3, {4, 3, 4}});
context->setInputShape(input_name_2.c_str(), nvinfer1::Dims{3, {4, 3, 4}});
```

由于两个输入都有动态形状的维度，因此需要分别设定两个输入的形状，且两个形状的设置必须满足算子对形状的合法要求。最基本的要求是，输入的形状必须介于 OptimizationProfile 给定的维度限定范围内，即 $\min \leq \text{input_value} \leq \max$ 。具体到示例中的 ElementWise 算子，要求两个输入的形状必须相同，或者说不同的维度可以广播。

在合法地设置输入形状后，可以直接获取到输出形状。

```
context->getTensorShape(output_name.c_str());
```

上述示例中展示了有两个输入拥有动态，对于较为复杂的模型，可能会有更多的输入，且每个输入都有动态形状的维度。您可以使用如下方法检查输入的动态形状是否都有效地进行了设置：

```
context->allInputDimensionsSpecified();
```

您还可以使用更精细化的方式检查哪些输入的动态形状没有进行有效的设置，例如，调用 inferShapes 接口。

```
int32_t max_names = 2;
std::vector<char const*> infer_shape_names(max_names);
auto nb_names = context->inferShapes(max_names, infer_shape_names.data());
```

Note

- inferShapes 接口的返回值 nb_names 是一个 int32_t 类型的数值，代表了输入的动态形状没有有效设置的个数。
- infer_shape_names 是一个 char const* 类型的数组，代表了输入的动态形状没有有效设置的名称。

4.6.2 输出动态形状

输出动态形状在编译阶段无需特殊的配置，需要关注的主要问题是：如果获取输出的形状并确保为输出分配了足够的存储空间用于接受输出结果。

下面以 NonZero 算子为例，说明如何处理输出动态形状的情况。

```
class OutputAllocator : public nvinfer1::IOutputAllocator {
public:
    OutputAllocator() {}

    void* reallocateOutput(char const* tensorName, void* currentMemory, uint64_t size,
                          uint64_t alignment) noexcept override {
        /*
         * If you have call setTensorAddress to set a non-nullptr for output tensor, and
         * pre-allocate memory meet the
         * memory demand , just return currentMemory
         */
        // Some memory allocators return nullptr when allocating zero bytes, but IXRT
        // requires a non-null ptr
        // even for empty tensors, so allocate a dummy byte.
        size = std::max(size, static_cast<uint64_t>(1));
        if (buffers_.count(tensorName)) {
            if (size > buffers_.at(tensorName).size) {
                ResetBuffer(tensorName);
                cudaMalloc(&buffers_.at(tensorName).buffer, size);
                buffers_.at(tensorName).size = size;
            } else {
                std::cout << "Reuse buffer for " << tensorName << ", request: " << size
                    << ", capacity: " << buffers_.at(tensorName).size << std::endl;
            }
        } else {
            DataCell new_cell;
            cudaMalloc(&new_cell.buffer, size);
            new_cell.size = size;
            buffers_.emplace(tensorName, new_cell);
        }

        return buffers_.at(tensorName).buffer;
    }

    void notifyShape(char const* tensorName, nvinfer1::Dims const& dims) noexcept override
    {
        if (buffers_.count(tensorName)) {
            auto& shape = buffers_.at(tensorName).shape;
            shape.clear();
            for (auto i = 0; i < dims.nbDims; ++i) {
                shape.emplace_back(dims.d[i]);
            }
        } else {

```

```

        std::cout << "Call reallocate output for " << tensorName << std::endl;
    }
}

std::vector<int64_t> getShape(const char* tensor_name) const {
    if (buffers_.count(tensor_name)) {
        return buffers_.at(tensor_name).shape;
    } else {
        return {};
    }
}

void* getBuffer(const char* tensor_name) {
    if (buffers_.count(tensor_name)) {
        return buffers_.at(tensor_name).buffer;
    } else {
        return nullptr;
    }
}

~OutputAllocator() override {
    for (const auto& item : buffers_) {
        const auto& tensor_name = item.first;
        ResetBuffer(tensor_name.c_str());
    }
}

int64_t getSize(char const* tensorName) const {
    if (buffers_.count(tensorName)) {
        return buffers_.at(tensorName).size;
    } else {
        return 0;
    }
}

uint64_t getNbAllocator() const { return buffers_.size(); }

std::vector<std::string> getNameList() const {
    if (buffers_.empty()) {
        return {};
    } else {
        std::vector<std::string> ret;
        for (const auto& item : buffers_) {
            const auto& name = item.first;
            ret.emplace_back(name);
        }
        return ret;
    }
}

```

```
private:
    void ResetBuffer(const char* tensor_name) {
        if (not buffers_.count(tensor_name)) {
            return;
        }

        if (buffers_.at(tensor_name).buffer) {
            cudaFree(buffers_.at(tensor_name).buffer);
        }
    }

    struct DataCell {
        DataCell() : buffer(nullptr), size(0) {}
        void* buffer;
        uint64_t size;
        std::vector<int64_t> shape{};
    };

    std::unordered_map<std::string, DataCell> buffers_;
};

Logger logger(nvinfer1::ILogger::Severity::kVERBOSE);
const auto explicitBatch = 1U <<
    ↪ static_cast<uint32_t>(nvinfer1::NetworkDefinitionCreationFlag::kEXPLICIT_BATCH);
auto builder = nvinfer1::createInferBuilder(logger);
auto network = builder->createNetworkV2(explicitBatch);
nvinfer1::Dims input_dims{2, {2, 5}};
nvinfer1::DataType data_type = nvinfer1::DataType::kFLOAT;
std::string input_name = "input_data";
auto input_data = network->addInput(input_name.c_str(), data_type, input_dims);
auto nonzero_layer = network->addNonZero(*input_data);
std::string output_name = "non_zero_out";
auto nonzero_output = nonzero_layer->getOutput(0);
nonzero_output->setName(output_name.c_str());
network->markOutput(*nonzero_output);

auto config = builder->createBuilderConfig();
auto plan = builder->buildSerializedNetwork(*network, *config);
auto runtime = nvinfer1::createInferRuntime(logger);
auto engine = runtime->deserializeCudaEngine(plan->data(), plan->size());
std::array<float, 10> input_fp_data{0.f, 0.f, 3.f, 4.f, 0.f, 6.f, 7.f, 8.f, 9.f, 10.f};
auto input_bytes = GetBytes(input_dims, data_type);
void* input_gpu_buffer = nullptr;
void* output_gpu_buffer = nullptr;
auto context = engine->createExecutionContext();
/*
 * For now
context->getMaxOutputSize(input_name.c_str()) == 1
context->getMaxOutputSize(output_name.c_str()) == 256
*/
cudaMalloc(&input_gpu_buffer, input_bytes);
cudaMemcpy(input_gpu_buffer, input_fp_data.data(), input_bytes, cudaMemcpyHostToDevice);
```

```

context->setTensorAddress(input_name.c_str(), input_gpu_buffer);
cudaMalloc(&output_gpu_buffer, KGPUStartAddrAlignValue);
context->setTensorAddress(output_name.c_str(), output_gpu_buffer);

OutputAllocator> output_allocator;
context->setOutputAllocator(output_name.c_str(), &output_allocator);
/*
context->getTensorShape(output_name.c_str()) == Dims{2, {2, -1}};
*/
context->enqueueV3(nullptr);
cudaStreamSynchronize(nullptr);
/*
CHECK(context->getTensorShape(output_name.c_str()) == Dims{2, {2, 7}});
*/
auto output_shape = output_allocator->getShape(output_name.c_str());
// Get output data
auto nb_elements = std::accumulate(output_shape.begin(), output_shape.end(), 1,
    ↪ std::multiplies<int64_t>());
std::vector<int32_t> output_data(nb_elements);
auto valid_content_bytes = nb_elements * sizeof(int32_t);
cudaMemcpy(output_data.data(), output_allocator->getBuffer(output_name.c_str()),
    ↪ valid_content_bytes,
    cudaMemcpyDeviceToHost);
/*
* output result should be like this:
output_data.at(0) == 0;
output_data.at(1) == 0;
output_data.at(2) == 1;
output_data.at(3) == 1;
output_data.at(4) == 1;
output_data.at(5) == 1;
output_data.at(6) == 1;
output_data.at(7) == 2;
output_data.at(8) == 3;
output_data.at(9) == 0;
output_data.at(10) == 1;
output_data.at(11) == 2;
output_data.at(12) == 3;
output_data.at(13) == 4;
*/
cudaFree(input_gpu_buffer);
cudaFree(output_gpu_buffer);
delete builder;
delete network;
delete config;
delete plan;
delete runtime;
delete engine;
delete context;

```

NonZero 算子的特点是：输出形状由输入数据内容决定。因此，ixRT 提供以下两套机制用于处理输出形状动态变化的情况。

- **方法 1**：即上述示例程序展示的使用 IOutputAllocator 接口，用户自行实现 IOutputAllocator 接口，其中核心的接口是 reallocateOutput 和 notifyShape。reallocateOutput 在每次推理流程开始时被调用，分配了相关输出 Tensor 的在当前输入形状下的最大内存需求。notifyShape 在每次推理流程中，输出形状可在求解时被调用，不一定是整个推理流程结束之后，例如，NonZero 后面还有 Unary 算子，将 Unary 算子的输出作为模型输出，notifyShape 方法在 NonZero 算子执行结束后就会被调用。
- **方法 2**：即上述示例程序中注释部分展示的使用 ExecutionContext 的 getMaxOutputSize 接口，获取输出 Tensor 的最大内存需求 (以 byte 为单位)。每次推理流程结束后调用 ExecutionContext 的 getTensorShape 接口，获取输出 Tensor 的形状。

Note

需要注意的是：

- 返回的内存需求是一个对齐的值，大于等于形状分析的结果；使用了 256 byte 对齐。
- OutputAllocator 与 ExecutionContext 的 setTensorAddress 接口并不冲突，可以同时使用。

对于输入形状是静态的情况，动态输出形状的最大内存需求也是固定的。如果是动态输入形状的情况，在更新输入形状后，动态输出形状的最大内存需求可能也会发生变化。从实践的角度来看，对于输入存在动态形状的情况，推荐使用 **方法 1**；输入形状为静态的情况，推荐使用 **方法 2**。

4.6.3 Shape Tensor 与 Execution Tensor

在模型设计阶段，不可避免会涉及到大量的形状变化，例如，Reshape、Transpose、Slice 等。这些操作在模型推理过程中会被转换为对应的算子，例如，Reshape 算子、Transpose 算子、Slice 算子等。模型中根据传递数据的性质不同，区分为两种类型的 Tensor，分别是 Shape Tensor 和 Execution Tensor。

Shape Tensor 泛指在推理过程中服务于形状分析和逻辑判断的 Tensor，存储和运算都在 CPU 上。目前 Shape Tensor 支持的数据类型有 Float、INT32、INT64、BOOL。

Execution Tensor 是模型推理过程中真正参与 GPU 计算的 Tensor，存储在 GPU 显存中。

在模型构建阶段，有些 Tensor 会被分类为 Shape Tensor，但是这个 Tensor 的消费者可能是 Execution Tensor；或者是相反的情况，这种情形下，ixRT 会在有设备类型冲突的位置插入内存搬运算子，将数据转换为同一种类型。

ixRT 中的算子，有较大一部分既可以支持 Shape Tensor，也可以支持 Execution Tensor，例如，Elementwise、Shuffle 等。

多数算子输入和输出的设备类型是有关联的，现在使用 E 表示 Execution Tensor，S 表示 Shape Tensor，t 表示两者皆可。关联规则可以表示为：

```
IAssertionLayer: S ->
IConcatenationLayer: t x t x ... -> t
ICumulativeLayer: t x t -> t
IIfConditionalInputLayer: t -> t
IIfConditionalOutputLayer: t -> t
IConstantLayer: -> t
```

```

IActivationLayer: t -> t
IElementWiseLayer: t x t -> t
IFillLayer: S -> t
IFillLayer: S x t x t -> t
IGatherLayer: t x t -> t
IIdentityLayer: t -> t
IReduceLayer: t -> t
IResizeLayer (one input): E -> E
IResizeLayer (two inputs): E x S -> E
ISelectLayer: t x t x t -> t
IShapeLayer: t1 -> t2
IShuffleLayer (one input): t -> t
IShuffleLayer (two inputs): t x S -> t
ISliceLayer (one input): t -> t
ISliceLayer (two inputs): t x S -> t
ISliceLayer (three inputs): t x S x S -> t
ISliceLayer (four inputs): t x S x S x S -> t
ITopK: E x S -> E
IUnaryLayer: t -> t
all other layers: E x ... -> E x ...

```

例如，IMatrixMultiply 算子的规则是：

```
E x E -> E
```

表达式左边是两个输入，右边是输出 IUnary 算子，规则是：

```
t -> t
```

表示输入和输出的设备类型可变，类型要求是一致的。IShapeLayer 算子的规则是：

```
t2 -> t2
```

输入输出类型可变，类型不要求一致。

通过 ITensor 的接口 ITensor::isShapeTensor() 和 ITensor::isExecutionTensor() 可以访问到 Tensor 的设备类型。需要注意的是，模型搭建过程中，初始状态总是被设置为 E，但是这个属性是会变化的。一些情况下，一个 Tensor 的消费者被调整为 Shape，相关的一个子图的 Tensor 都会被更新为 Shape Tensor。

本章的第一小节**输入动态形状**，展示的是 Execution Tensor 作为输入的动态形状操作。

4.6.4 算子的形状输入

Shape 算子可以用于在运行时获取形状信息，输入任意类型的 Tensor，输出包含输入维度信息的 1D Tensor。如果输入是一个标量，维度的值为空。

一些算子允许有可选的 Shape Tensor 输入，可以在运行时接收形状信息，支持动态形状推理特性。对于 Shuffle 算子，可以设定第二个输入来指定输出的形状，例如，一个模型中有两个输入 X 和 Y，要求 Y 总是保持与 X 相同的形状，可以这样表达：

```
auto* reshape = networkDefinition.addShuffle(Y);
reshape.setInput(1, networkDefintion.addShape(X)->getOutput(0));
```

对于 Resize 和 Slice 算子，可以通过接收可选的 Shape Tensor 输入，在运行时调整输出形状。对于 Concatenation、ElementWise、Gather、Identity、Reduce 算子，可以直接对 Shape Tensor 进行计算。

4.6.5 Shape Tensor I/O

Shape Tensor 也可以作为模型的输入或输出，当模型的输入是 Shape Tensor 时，输入内容变化，展现了动态形状的能力，示例如下：

```
Logger logger(nvinfer1::ILogger::Severity::kVERBOSE);
auto builder = nvinfer1::createInferBuilder(logger);
const auto explicitBatch = 1U <<
    ↪ static_cast<uint32_t>(nvinfer1::NetworkDefinitionCreationFlag::kEXPLICIT_BATCH);
auto network = builder->createNetworkV2(explicitBatch);
// Build network
nvinfer1::Dims input_dims{1, {25}};
std::string data_input_name("data_input");
auto input = network->addInput(data_input_name.c_str(), nvinfer1::DataType::kFLOAT,
    ↪ input_dims);
std::string shape_input_name("shape_input");
auto shape_input = network->addInput(shape_input_name.c_str(),
    ↪ nvinfer1::DataType::kINT32, nvinfer1::Dims{1, {1}});
nvinfer1::Dims start{1, {0}};
nvinfer1::Dims output_size{1, {5}};
nvinfer1::Dims stride{1, {2}};
auto slice_layer = network->addSlice(*input, start, output_size, stride);
slice_layer->setInput(1, *shape_input);
auto slice_output = slice_layer->getOutput(0);
std::string slice_output_name("output");
slice_output->setName(slice_output_name.c_str());
network->markOutput(*slice_output);

auto config = builder->createBuilderConfig();
auto profile = builder->createOptimizationProfile();
int32_t start_min_value = 0;
profile->setShapeValues(shape_input_name.c_str(), nvinfer1::OptProfileSelector::kMIN,
    ↪ &start_min_value, 1);
int32_t start_opt_value = 5;
profile->setShapeValues(shape_input_name.c_str(), nvinfer1::OptProfileSelector::kOPT,
    ↪ &start_opt_value, 1);
int32_t start_max_value = 10;
profile->setShapeValues(shape_input_name.c_str(), nvinfer1::OptProfileSelector::kMAX,
    ↪ &start_max_value, 1);
config->addOptimizationProfile(profile);
auto plan = builder->buildSerializedNetwork(*network, *config);
auto runtime = nvinfer1::createInferRuntime(logger);
```

```

auto engine = runtime->deserializeCudaEngine(plan->data(), plan->size());
cout << "Number of engine IO is " << engine->getNbIOTensors() << endl;
cout << data_input_name << " is shape IO : " <<
    ↪ engine->isShapeInferenceIO(data_input_name.c_str()) << endl;
cout << data_input_name << " is shape IO : " <<
    ↪ engine->isShapeInferenceIO(shape_input_name.c_str()) << endl;
void* input_data;
void* output_data;
auto input_bytes = input_dims.d[0] * 4;
auto output_bytes = output_size.d[0] * 4;
cudaMalloc(&input_data, input_bytes);
cudaMalloc(&output_data, output_bytes);
constexpr std::array<float, 25> slice_data{/* Fill data here*/};
cudaMemcpy(input_data, slice_data.data(), input_bytes, cudaMemcpyHostToDevice);

auto context = engine->createExecutionContext();
context->setTensorAddress(data_input_name.c_str(), input_data);
context->setTensorAddress(slice_output_name.c_str(), output_data);
int32_t start_value = 3;
context->setTensorAddress(shape_input_name.c_str(), &start_value);
cudaStream_t stream;
cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
auto exec_status = context->enqueueV3(stream);
cudaStreamSynchronize(stream);

std::vector<float> output(output_size.d[0], 0.f);
cudaMemcpy(output.data(), output_data, output_bytes, cudaMemcpyDeviceToHost);
// Show output data
cudaFree(input_data);
cudaFree(output_data);
cudaStreamDestroy(stream);
delete builder;
delete network;
delete config;
delete plan;
delete runtime;
delete engine;
delete context;

```

对于 Slice 算子，最多可以接受 5 个输入，除了第一个输入外，其它都是可选输入。在上述示例中，将模型输入设定为 Slice 的第二个输入，这个输入的需求是 Shape Tensor。因此，需要为这个输入标记动态形状的范围，例如，这里控制的是 start 的范围：

```

auto config = builder->createBuilderConfig();
auto profile = builder->createOptimizationProfile();
int32_t start_min_value = 0;
profile->setShapeValues(shape_input_name.c_str(), nvinfer1::OptProfileSelector::kMIN,
    ↪ &start_min_value, 1);
int32_t start_opt_value = 5;
profile->setShapeValues(shape_input_name.c_str(), nvinfer1::OptProfileSelector::kOPT,
    ↪ &start_opt_value, 1);

```

```
int32_t start_max_value = 10;
profile->setShapeValues(shape_input_name.c_str(), nvinfer1::OptProfileSelector::kMAX,
    ↪ &start_max_value, 1);
config->addOptimizationProfile(profile);
```

编译阶段，Shape Tensor 设定动态范围需要使用 setShapeValues 接口。

```
int32_t start_value = 3;
context->setTensorAddress(shape_input_name.c_str(), &start_value);
```

运行时，要为 Shape Tensor 设定主机端的地址作为输入。

4.6.6 形状表达总结

根据上述对 Shape Tensor 和 Execution Tensor 的介绍，下表列出了 Dims 表达不同 Tensor 时的描述方式。

表达形式	Dims 表达示例	是 Shape Tensor 或 Execution Tensor	备注
标量	Dims{0, {}}	Shape Tensor Execution Tensor	
1D Tensor	Dims{1, {n}}	Shape Tensor Execution Tensor	
nD Tensor	Dims{n, {d1,...}}	Execution Tensor	n > 1
空 Tensor	Dims{m, {..., 0, ...}}	Shape Tensor Execution Tensor	m > 0

从上表中可以看到关于 Tensor 设备类型的限制，对于 Shape Tensor，只能是 1D Tensor 或标量，Execution Tensor 可以是任意形式。

空 Tensor 的表达，只要其中一个维度为 0，就可以表达空 Tensor。

4.7 自定义插件使用指南

ixRT 支持类型丰富的算子，但是内置算子可以提供的计算支持始终无法及时跟上 AI 算法的演进。因此，ixRT 提供了插件扩展接口，用于满足自定义计算的需求。

4.7.1 使用 C++ 接口添加自定义插件

4.7.1.1 插件接口

实现一个自定义插件主要关注以下两个接口类：

- 定义插件功能的类 IPluginV2
- 创建插件的类 IPluginCreator

4.7.1.1.1 IPluginV2

插件的实现类需要继承 IPluginV2IOExt 或者 IPluginV2DynamicExt，两者的差异在于插件实现是否支持动态形状。

其中，IPluginV2IOExt 基类不支持动态形状，IPluginV2DynamicExt 支持动态形状。

4.7.1.1.2 IPluginCreator

IPluginCreator 是自定义插件的创建基类，用户可以使用它获取插件的名称、版本和参数。该基类还提供了在网络构建阶段创建插件对象和在推理期间反序列化插件的功能。

```
class TRT_DEPRECATED IPluginCreator : public IPluginCreatorInterface {
public:
    // 返回插件名称
    virtual AsciiChar const* getPluginName() const noexcept = 0;

    // 返回插件版本
    virtual AsciiChar const* getPluginVersion() const noexcept = 0;

    // 创建插件时，需要知道插件的所有属性字段参数。
    // `getFieldNames` 方法返回 `PluginFieldCollection`，其中包含了插件字段数量 `nbFields` 和每个
    // 插件字段的数据指针 `PluginField const* fields`
    virtual PluginFieldCollection const* getFieldNames() noexcept = 0;

    // 返回一个插件对象，创建失败则返回 nullptr
    virtual IPluginV2* createPlugin(AsciiChar const* name, PluginFieldCollection const*
    ↪ fc) noexcept = 0;

    // 在解析插件层时调用，返回一个插件对象
    virtual IPluginV2* deserializePlugin(AsciiChar const* name, void const* serialData,
    ↪ size_t serialLength) noexcept = 0;

    // 设置 plugin creator 的命名空间，可以在注册 plugin creator 的时候设置。
    virtual void setPluginNamespace(AsciiChar const* pluginNamespace) noexcept = 0;

    // 返回插件创建对象
    virtual AsciiChar const* getPluginNamespace() const noexcept = 0;
```

```

IPluginCreator() = default;
~IPluginCreator() override = default;

protected:
IPluginCreator(IPluginCreator const&) = default;
IPluginCreator(IPluginCreator&&) = default;
IPluginCreator& operator=(IPluginCreator const&) &= default;
IPluginCreator& operator=(IPluginCreator&&) &= default;
public:
InterfaceInfo getInterfaceInfo() const noexcept override { return
↳ InterfaceInfo{"PLUGIN_CREATOR_V1", 1, 0}; }
};

```

插件本身通常会包含一些属性，IPluginCreator 使用 PluginFieldCollection 结构体来创建和传递属性。

```

struct PluginFieldCollection {
    // 属性字段数量
    int32_t nbFields;
    // 指向每个字段属性的指针
    PluginField const* fields;
};

```

4.7.1.2 示例：实现一个自定义插件

以实现一个自定义 Concat 算子为例，首先定义该插件类的 Version 和 PluginName：

```

char const* kConcatPluginVersion{"1"};
char const* kConcatPluginName{"ConcatPlugin_IxRT"};

```

4.7.1.2.1 插件实现类

创建自定义 Plugin 类

```

#include "NvInfer.h"
#include "NvInferPlugin.h"

class ConcatPluginDynamic : public nvinfer1::IPluginV2DynamicExt
{
public:
    ConcatPluginDynamic(std::string const name, int axis);
    ConcatPluginDynamic(std::string const name, const void* data, size_t length);
    ConcatPluginDynamic() noexcept = delete;
    ~ConcatPluginDynamic() override = default;

    // IPluginV2 methods

```

```

const char* getPluginType() const noexcept override;
const char* getPluginVersion() const noexcept override;
int32_t initialize() noexcept override;
int getNbOutputs() const noexcept override;
void terminate() noexcept override;
size_t getSerializationSize() const noexcept override;
void serialize(void* buffer) const noexcept override;
void destroy() noexcept override;
void setPluginNamespace(const char* pluginNamespace) noexcept override;
const char* getPluginNamespace() const noexcept override;

// IPluginV2Ext methods
nvinfer1::DataType getOutputDataType(int index, const nvinfer1::DataType* inputTypes,
                                     int nbInputs) const noexcept override;

// IPluginV2DynamicExt methods
IPluginV2DynamicExt* clone() const noexcept override;
nvinfer1::DimsExprs getOutputDimensions(int outputIndex, const nvinfer1::DimsExprs*
    inputs,
                                     int nbInputs, nvinfer1::IExprBuilder& exprBuilder)
    noexcept override;
bool supportsFormatCombination(int pos, const nvinfer1::PluginTensorDesc* inOut,
                                int nbInputs, int nbOutputs) noexcept override;
void configurePlugin(const nvinfer1::DynamicPluginTensorDesc* in, int nbInputs,
                    const nvinfer1::DynamicPluginTensorDesc* out, int nbOutputs)
    noexcept override;
size_t getWorkspaceSize(const nvinfer1::PluginTensorDesc* inputs, int nbInputs,
                        const nvinfer1::PluginTensorDesc* outputs, int nbOutputs) const
    noexcept override;
int enqueue(const nvinfer1::PluginTensorDesc* inputDesc, const
    nvinfer1::PluginTensorDesc* outputDesc,
    const void* const* inputs, void* const* outputs, void* workspace,
    cudaStream_t stream) noexcept override;

private:
int mAxis;
int mNumInputs{1};
std::string mNameSpace{};
std::string mLayerName{};
};

```

说明:

继承 IPluginV2DynamicExt 类，并重载其中的成员函数。

实现关键方法

构造函数

```

ConcatPluginDynamic::ConcatPluginDynamic(std::string const name, int axis)
:mLayerName(name), mAxis(axis) {

```

```

}

ConcatPluginDynamic::ConcatPluginDynamic(std::string const name, const void* data, size_t
↪ length)
:mLayerName(name) {
    const char* d = static_cast<const char*>(data);
    memcpy(&mAxis, data, sizeof(int));
}

```

说明:

- ConcatPluginDynamic(std::string const name, int axis): 编译阶段使用的构造函数, 需要指定层的名称以及插件层的各个属性。
- ConcatPluginDynamic(std::string const name, const void* data, size_t length): 反序列化过程中使用的构造函数, 从指定的地址中反序列化相关参数, 对 data 数据解析需要与下面定义的 serialize 的参数序列化一致, length 与下面定义的 getSerializationSize 返回值一致。

序列化和反序列化

```

size_t ConcatPluginDynamic::getSerializationSize() const noexcept
{
    return sizeof(mAxis);
}

void ConcatPluginDynamic::serialize(void* buffer) const noexcept
{
    memcpy(buffer, &mAxis, sizeof(int));
}

```

说明:

当前插件存在内部参数时,

- serialize: 用于序列化相关插件计算时所需参数至指定地址中, 当前示例中仅有 mAxis 需要记录。
- getSerializationSize: 返回插件参数序列化占用的空间大小, 与 serialize 实际使用到的参数占用空间一致。

管理插件命名和版本

```

char const* kConcatPluginVersion{"1"};
char const* kConcatPluginName{"ConcatPlugin_IxRT"};
void ConcatPluginDynamic::setPluginNamespaces(char const* libNamespace) noexcept {
    try {
        assert(libNamespace != nullptr);
        mNameSpace = libNamespace;
    } catch (std::exception const& e) {
        std::cerr << e.what() << std::endl;
    }
}

char const* ConcatPluginDynamic::getPluginNamespace() const noexcept { return
↪ mNameSpace.c_str(); }

```

```
char const* ConcatPluginDynamic::getPluginType() const noexcept { return
↳ kConcatPluginName; }

char const* ConcatPluginDynamic::getPluginVersion() const noexcept { return
↳ kConcatPluginVersion; }
```

说明：

- 用户后续可以在插件命名空间 (默认为 "") 下，通过插件名称 ConcatPlugin_IxRT 和版本 1 获取到当前实现的插件。
- version、name、namespace 共同构成插件的唯一标识符，防止重名。

获取插件输出 Tensor 数量

```
int32_t ConcatPluginDynamic::getNbOutputs() const noexcept { return 1; }
```

说明：

当前插件共有 1 个输出，返回 1。

获取动态输出维度

```
nvinfer1::DimsExprs ConcatPluginDynamic::getOutputDimensions(
    int outputIndex, const nvinfer1::DimsExprs* inputs, int nbInputs,
    nvinfer1::IExprBuilder& exprBuilder) noexcept
{
    assert(outputIndex == 0);
    assert(nbInputs > 0);

    nvinfer1::DimsExprs output(inputs[0]);

    // 沿指定轴连接所有输入
    for (int i = 1; i < nbInputs; ++i)
    {
        output.d[mAxis] = exprBuilder.operation(
            nvinfer1::DimensionOperation::kSUM,
            *output.d[mAxis],
            *inputs[i].d[mAxis]);
    }

    return output;
}
```

说明：

- getOutputDimensions：在编译阶段由 ixRT 内部调用，通过输入数据的表达式获取到输出数据的表达式。
- ixRT 中存在动态形状的情况，因此编译阶段不会进行数值计算，而是构建计算表达式。这些表达式将在编译结束后存储在 Engine 中，运行时使用这些表达式进行计算，可以获取每次运行时的真实形状。

- `getOutputDimensions` 返回第 `outputIndex` 个输出的形状，类型为 `DimsExprs`，其中存储着每个维度的形状表达式，相关的数据结构包括：

```
class DimsExprs {
public:
    int32_t nbDims; // 表达式维度
    IDimensionExpr const* d[Dims::MAX_DIMS]; // 每一维的表达式
};

class IDimensionExpr : public INoCopy {
public:
    // 返回表达式在编译时是否为常量
    bool isConstant() const noexcept { return mImpl->isConstant(); }

    // 获取常量值
    int64_t getConstantValue() const noexcept { return mImpl->getConstantValue(); }

protected:
    apiv::VDimensionExpr* mImpl;
    virtual ~IDimensionExpr() noexcept = default;
};

class IExprBuilder : public INoCopy {
public:
    // 返回一个固定值表达式
    IDimensionExpr const* constant(int64_t value) noexcept { return
    ↪ mImpl->constant(value); }

    // 返回一个由两个表达式计算得到的表达式
    IDimensionExpr const* operation(DimensionOperation op, IDimensionExpr const&
    ↪ first,
                                IDimensionExpr const& second) noexcept {
        return mImpl->operation(op, first, second);
    }

protected:
    apiv::VExprBuilder* mImpl;
    virtual ~IExprBuilder() noexcept = default;
};
```

- 对于一个新建的 `DimsExprs output`，设置其输出维度为 3，即 `output.nbFields = 3`，常见的构建每个维度表达式 `IDimensionExpr` 的方法有：
 - 从 `inputs` 某一维度进行拷贝：`output.d[0] = inputs[0].d[0]`;
 - 固定值，例如为 6 的情况：`output.d[1] = exprBuilder.constant(6)`;
 - 使用两个 `IDimensionExpr` 进行计算得到：`output.d[2] = exprBuilder.operation(nvinfer1::Dims`
可用的 `operation` 包括：

```
enum class DimensionOperation : int32_t {
    kSUM = 0, // first + second
    kPROD = 1, // first * second
```

```
kMAX = 2,          // max(first, second)
kMIN = 3,          // min(first, second)
kSUB = 4,          // first - second
kEQUAL = 5,        // first == second 则返回 1 否则返回 0
kLESS = 6,         // first < second 则返回 1 否则返回 0
kFLOOR_DIV = 7,    // first / second 向下取整
kCEIL_DIV = 8      // first / second 向上取整
};
```

- 在当前 ConcatPluginDynamic 的示例中输出第 mAxis 个维度是所有输入的第 mAxis 个维度之和，其余维度不变。因此先直接复制 inputs[0] 的所有维度，再更新第 mAxis 个维度。

插件输出类型

```
DataType ConcatPluginDynamic::getOutputDataType(int32_t index, DataType const*
↪ inputTypes, int32_t nbInputs) const noexcept {
    assert(inputTypes != nullptr);
    assert(index == 0);
    return inputTypes[0];
}
```

说明：

获取对应 index 的输出对象数据类型，由用户的插件算子实现方法决定，不同类型见 nvinfer1::DataType 的定义。

支持格式组合

ixRT 内部在编译过程中判断每个插件层该使用哪种排布 format/类型 type 组合时，会调用 supportsFormatCombination 来审查每个位置上指定的 format/type 组合是否符合插件要求，满足条件的返回 true。编译时通过遍历不同的 format/type 组合，找到符合当前插件的组合。使用 PluginTensorDesc 结构传递数据类型和数据排布。

```
struct PluginTensorDesc {
    Dims dims;          // 形状
    DataType type;       // 数据类型
    TensorFormat format; // 数据排布
    float scale;         // int8 类型的量化系数
};

bool ConcatPluginDynamic::supportsFormatCombination(
    int pos, const nvinfer1::PluginTensorDesc* inOut, int nbInputs, int nbOutputs) noexcept
{
    assert(nbInputs > 0 && pos < nbInputs + nbOutputs);

    const auto& desc = inOut[pos];

    if (desc.format != nvinfer1::TensorFormat::kLINEAR)
        return false;

    if (pos == 0)
        return true;
}
```

```
return inOut[0].type == desc.type;
}
```

说明：

- pos 的取值从 0 开始，表示 1 个输入，后续按照输入数量和输出数量累加。以存在 2 个输入和 1 个输出的 concat 为例，输入的 pos 分别为 0 和 1，输出的 pos 为 2。
- 编译过程中格式审查会从 pos 为 0 开始，因此可以假定较小的 pos 的 format/type 已经确定。

同时需要注意的是：不能在函数中判断大于当前 pos 的 format/type 组合。

设置 Workspace Size

Workspace 由 ixRT 管理，在编译阶段由整体的内存规划决定。插件需要返回所需的 Workspace 空间大小：

```
size_t ConcatPluginDynamic::getWorkspaceSize(PluginTensorDesc const* inputs, int32_t
    nbInputs, PluginTensorDesc const* outputs,
    int32_t nbOutputs) const noexcept {
    return 0;
}
```

配置 Plugin

```
void ConcatPluginDynamic::configurePlugin(
    const nvinfer1::DynamicPluginTensorDesc* in, int nbInputs,
    const nvinfer1::DynamicPluginTensorDesc* out, int nbOutputs) noexcept
{
    // 验证配置
    for (int i = 0; i < nbInputs; ++i)
    {
        assert(in[i].desc.type == in[0].desc.type);
        assert(in[i].desc.format == nvinfer1::TensorFormat::kLINEAR);
    }
    mNumInputs = nbInputs;
}
```

说明：

configurePlugin 在编译和运行阶段都会被调用：

- 编译阶段，其作用是用于算法选择。
- 执行阶段，在每次输入形状变更后，调用此方法用于执行推理前的准备工作，例如，获取插件相关执行参数。

初始化/终止/删除 Plugin

```
int32_t ConcatPluginDynamic::initialize() noexcept {
    return 0;
}
```

```
void ConcatPluginDynamic::terminate() noexcept {}

void ConcatPluginDynamic::destroy() noexcept {
    delete this;
}
```

说明:

- initialize: 可以在运行插件前调用, 做插件的初始化工作, 例如, 使用一些计算库的初始化。不同于 configurePlugin, initialize 只能调用一次。
- terminate: 在 context 销毁时调用, 当前插件所拥有的资源需要被释放。
- destroy: 每次 builder、network、engine 或者 context 被销毁时都会被调用。

拷贝 Plugin

```
IPluginV2DynamicExt* ConcatPluginDynamic::clone() const noexcept {
    try {
        auto plugin = new ConcatPluginDynamic(*this);
        plugin->setPluginNamespace(mNameSpace.c_str());
        return plugin;
    } catch (std::exception const& e) {
        std::cerr << e.what() << std::endl;
    }
    return nullptr;
}
```

说明:

该函数在创建一个新的包含此插件层的 builder、network 或者 engine 时都会被调用, 会返回一个新的插件对象。

执行函数实现

```
inline uint32_t getElementSize(nvinfer1::DataType t) noexcept {
    switch (t) {
        case nvinfer1::DataType::kINT32:
        case nvinfer1::DataType::kFLOAT:
            return 4;
        case nvinfer1::DataType::kHALF:
            return 2;
        case nvinfer1::DataType::kBOOL:
        case nvinfer1::DataType::kINT8:
            return 1;
    }
    return 0;
}

int ConcatPluginDynamic::enqueue(
    const nvinfer1::PluginTensorDesc* inputDesc, const nvinfer1::PluginTensorDesc*
    ⇨ outputDesc,
    const void* const* inputs, void* const* outputs, void* workspace, cudaStream_t stream)
    ⇨ noexcept
```

```

{
    int numConcat = 1;
    for (int i = 0; i < mAxis; ++i)
    {
        numConcat *= outputDesc[0].dims.d[i];
    }

    int concatSize = 1;
    for (int i = mAxis + 1; i < outputDesc[0].dims.nbDims; ++i)
    {
        concatSize *= outputDesc[0].dims.d[i];
    }

    char* output = static_cast<char*>(outputs[0]);
    size_t offset = 0;
    const size_t elemSize = getElementSize(outputDesc[0].type);

    for (int i = 0; i < mNumInputs; ++i)
    {
        const char* input = static_cast<const char*>(inputs[i]);
        const size_t copySize = inputDesc[i].dims.d[mAxis] * concatSize * elemSize;

        for (int n = 0; n < numConcat; ++n)
        {
            cudaMemcpyAsync(
                output + offset + n * outputDesc[0].dims.d[mAxis] * concatSize * elemSize,
                input + n * inputDesc[i].dims.d[mAxis] * concatSize * elemSize,
                copySize,
                cudaMemcpyDeviceToDevice,
                stream);
        }
        offset += copySize;
    }

    return 0;
}

```

说明：

enqueue 封装了插件的实现逻辑。

插件创建类

创建 Plugin Creator

```

class ConcatPluginDynamicCreator : public nvinfer1::IPluginCreator
{
public:
    ConcatPluginDynamicCreator() noexcept;
    const char* getPluginName() const noexcept override;
    const char* getPluginVersion() const noexcept override;

```

```

const nvinfer1::PluginFieldCollection* getFieldNames() noexcept override;
nvinfer1::IPluginV2* createPlugin(const char* name, const
↪ nvinfer1::PluginFieldCollection* fc) noexcept override;
nvinfer1::IPluginV2* deserializePlugin(const char* name, const void* serialData,
↪ size_t serialLength) noexcept override;
void setPluginNamespace(const char* pluginNamespace) noexcept override;
const char* getPluginNamespace() const noexcept override;

private:
    static nvinfer1::PluginFieldCollection mFC;
    static std::vector<nvinfer1::PluginField> mPluginAttributes;
    std::string mNameSpace;
};

```

说明：

继承 IPluginCreator 类重载其中的函数。

构造函数

```

PluginFieldCollection ConcatPluginDynamicCreator::mFC{};
std::vector<PluginField> ConcatPluginDynamicCreator::mPluginAttributes;

ConcatPluginDynamicCreator::ConcatPluginDynamicCreator() noexcept
{
    mPluginAttributes.emplace_back(nvinfer1::PluginField("axis", nullptr,
        nvinfer1::PluginFieldType::kINT32, 1));

    mFC.nbFields = mPluginAttributes.size();
    mFC.fields = mPluginAttributes.data();
}

```

说明：

描述插件所需的参数名，并封装至对应的 PluginFieldCollection 中。

管理 Plugin 命名空间/属性

```

char const* ConcatPluginDynamicCreator::getPluginName() const noexcept { return
↪ kConcatPluginName; }

char const* ConcatPluginDynamicCreator::getPluginVersion() const noexcept { return
↪ kConcatPluginVersion; }

PluginFieldCollection const* ConcatPluginDynamicCreator::getFieldNames() noexcept {
↪ return &mFC; }

void ConcatPluginDynamicCreator::setPluginNamespace(const char* pluginNamespace) noexcept
{
    mNameSpace = pluginNamespace;
}

```

```
const char* ConcatPluginDynamicCreator::getPluginNamespace() const noexcept
{
    return mNameSpace.c_str();
}
```

说明：

- 与插件实现类中的名称和版本相同。
- getFieldNames 方法返回插件的属性数据，一般用于在创建插件之前，获取 PluginFieldCollection 的基本信息，例如，有多少项参数。

创建 Plugin 方法

```
nvinfer1::IPluginV2* ConcatPluginDynamicCreator::createPlugin(const char* name, const
↳ nvinfer1::PluginFieldCollection* fc) noexcept
{
    try {
        int axis = 0; // 默认轴

        // 解析 PluginFieldCollection 中的参数
        const nvinfer1::PluginField* fields = fc->fields;
        for (int i = 0; i < fc->nbFields; ++i)
        {
            if (strcmp(fields[i].name, "axis") == 0 && fields[i].type ==
↳ nvinfer1::PluginFieldType::kINT32)
            {
                axis = *(static_cast<const int*>(fields[i].data));
            }
        }
        IPluginV2DynamicExt* plugin = new ConcatPluginDynamic(name, axis);
        plugin->setPluginNamespace(mNameSpace.c_str());
        return plugin;
    }
    catch (std::exception const& e) {
        std::cerr << "Create plugin failed: " << e.what() << std::endl;
        return nullptr;
    }
}
```

说明：

解析 PluginFieldCollection 中的参数，并用于创建新的插件。

反序列化 Plugin 方法

```
nvinfer1::IPluginV2* ConcatPluginDynamicCreator::deserializePlugin(const char* name,
↳ const void* serialData, size_t serialLength) noexcept
{
    try {
```

```

    assert(serialData != nullptr);
    return new ConcatPluginDynamic(name, serialData, serialLength);
}
catch (std::exception const& e) {
    std::cerr << "Deserialize plugin failed: " << e.what() << std::endl;
    return nullptr;
}
}
}

```

说明:

该方法由 ixRT 根据插件名称和版本在内部调用, 返回一个插件对象。

注册 Plugin

```
REGISTER_TENSORRT_PLUGIN(ConcatPluginDynamicCreator);
```

说明:

通过 REGISTER_TENSORRT_PLUGIN 宏注册插件至 ixRT 中。

4.7.2 调用自定义插件

Note

假设上述自定义插件代码编译后得到的动态库为 libconcat_plugin.so。

4.7.2.1 在 C++ 中使用用户自定义插件

4.7.2.1.1 加载插件库

```

#include "NvInfer.h"
#include "NvInferPlugin.h"
#include <dlfcn.h>

void loadConcatPluginLibrary() {
    void* handle = dlopen("./libconcat_plugin.so", RTLD_LAZY);
    if (!handle) {
        std::cerr << "Cannot open library: " << dlerror() << std::endl;
        return;
    }

    // 验证插件已注册
    auto pluginRegistry = getPluginRegistry();
    auto creator = pluginRegistry->getPluginCreator("ConcatPlugin_IxRT", "1");
    if (!creator) {
        std::cerr << "Plugin not found in registry" << std::endl;
    }
}

```

```

    } else {
        std::cout << "Successfully loaded ConcatPlugin_IxRT" << std::endl;
    }
}

```

说明:

使用 dlopen 加载对应的插件动态库, 通过 getPluginRegistry 返回的 IPluginRegistry, 可以注册插件及获取已注册插件。其中常用的函数包括:

- bool registerCreator(IPluginCreator& creator, AsciiChar const* const pluginNamespace): 注册插件的函数。
- IPluginCreator* getPluginCreator(AsciiChar const* const pluginName, AsciiChar const* const pluginVersion, AsciiChar const* const pluginNamespace = ""): 传入插件名称、版本、命名空间获取对应插件的创建方法。
- IPluginCreator* const* getPluginCreatorList(int32_t* const numCreators): 返回已注册的插件数量 numCreators 和所有注册插件的 creator 方法。

4.7.2.1.2 构建网络时使用插件

```

nvinfer1::ITensor* createConcatPlugin(nvinfer1::INetworkDefinition* network,
    int axis,
    nvinfer1::ITensor** inputs,
    int numInputs) {
    auto creator = getPluginRegistry()->getPluginCreator("ConcatPlugin_IxRT", "1");

    if (!creator) {
        std::cerr << "Failed to get plugin creator" << std::endl;
        return nullptr;
    }

    // 准备插件字段
    nvinfer1::PluginField axisField{"axis", &axis, nvinfer1::PluginFieldType::kINT32, 1};
    nvinfer1::PluginFieldCollection pluginFC{1, &axisField};

    // 创建插件
    auto concatPlugin = creator->createPlugin("dynamic_concat", &pluginFC);
    if (!concatPlugin) {
        std::cerr << "Failed to create plugin" << std::endl;
        return nullptr;
    }

    // 添加插件层
    auto concatLayer = network->addPluginV2(inputs, numInputs, *concatPlugin);

    // 返回插件层的输出 Tensor
    return concatLayer->getOutput(0);
}

```

说明：

创建插件时，createPlugin 需要传入插件所需的 PluginFieldCollection。

4.7.2.1.3 运行带插件的网络

```
using Severity = nvinfer1::ILogger::Severity;
class Logger : public nvinfer1::ILogger {
public:
    explicit Logger(Severity severity = Severity::kWARNING) :
        ↪ mReportableSeverity(severity) {}
    nvinfer1::ILogger& getIxRTLogger() noexcept { return *this; }
    void log(Severity severity, const char* msg) noexcept override {
        if (severity <= mReportableSeverity) {
            std::cout << "[IXRT] " << msg << std::endl;
        }
    }
    void setReportableSeverity(Severity severity) noexcept { mReportableSeverity =
        ↪ severity; }
    Severity getReportableSeverity() const { return mReportableSeverity; }

private:
    Severity mReportableSeverity;
}; // class Logger

struct ObjectDeleter {
    template <typename T>
    void operator()(T* obj) const {
        delete obj;
    }
};

int main(int argc, char* argv[]) {
    loadConcatPluginLibrary();
    Logger logger(nvinfer1::ILogger::Severity::kVERBOSE);
    auto builder =
        ↪ std::unique_ptr<nvinfer1::IBuilder>(nvinfer1::createInferBuilder(logger));
    const auto explicitBatch = 1U <<
        ↪ static_cast<uint32_t>(nvinfer1::NetworkDefinitionCreationFlag::kEXPLICIT_BATCH);
    auto network = std::unique_ptr<nvinfer1::INetworkDefinition>(builder->
        ↪ createNetworkV2(explicitBatch));
    nvinfer1::Dims input0_dims{3, {1, -1, 3}};
    auto input0_data = network->addInput("input0_data", nvinfer1::DataType::kFLOAT,
        ↪ input0_dims);
    nvinfer1::Dims input1_dims{3, {1, -1, 3}};
    auto input1_data = network->addInput("input1_data", nvinfer1::DataType::kFLOAT,
        ↪ input1_dims);
    std::vector<nvinfer1::ITensor*> inputs_vec;
    inputs_vec.emplace_back(input0_data);
```

```

inputs_vec.emplace_back(input1_data);
auto concat_output = createConcatPlugin(network.get(), 1, inputs_vec.data(), 2);
concat_output->setName("output_data");
network->markOutput(*concat_output);

auto config =
    ↪ std::unique_ptr<nvinfer1::IBuilderConfig>(builder->createBuilderConfig());
config->setFlag(nvinfer1::BuilderFlag::kFP16);

auto profile = builder->createOptimizationProfile();
profile->setDimensions("input0_data", nvinfer1::OptProfileSelector::kMIN,
    ↪ nvinfer1::Dims{3, 1, 1, 3});
profile->setDimensions("input0_data", nvinfer1::OptProfileSelector::kOPT,
    ↪ nvinfer1::Dims{3, 1, 2, 3});
profile->setDimensions("input0_data", nvinfer1::OptProfileSelector::kMAX,
    ↪ nvinfer1::Dims{3, 1, 3, 3});
profile->setDimensions("input1_data", nvinfer1::OptProfileSelector::kMIN,
    ↪ nvinfer1::Dims{3, 1, 1, 3});
profile->setDimensions("input1_data", nvinfer1::OptProfileSelector::kOPT,
    ↪ nvinfer1::Dims{3, 1, 2, 3});
profile->setDimensions("input1_data", nvinfer1::OptProfileSelector::kMAX,
    ↪ nvinfer1::Dims{3, 1, 3, 3});
config->addOptimizationProfile(profile);

std::unique_ptr<nvinfer1::IHostMemory> plan{builder->buildSerializedNetwork(*network,
    ↪ *config)};
std::unique_ptr<nvinfer1::IRuntime> runtime{nvinfer1::createInferRuntime(logger)};

auto engine = std::shared_ptr<nvinfer1::ICudaEngine>(runtime->deserializeCudaEngine(
    ↪ plan->data(), plan->size(),
    ↪ ObjectDeleter());
auto context =
    ↪ std::unique_ptr<nvinfer1::IExecutionContext>(engine->createExecutionContext());
nvinfer1::Dims input_dims1{3, {1, 1, 3}};
nvinfer1::Dims input_dims2{3, {1, 2, 3}};
context->setInputShape("input0_data", input_dims1);
context->setInputShape("input1_data", input_dims2);

float input_data_0[3] = {1, 2, 3};
float input_data_1[6] = {1, 2, 3, 4, 5, 6};
std::vector<void*> binding_buffer(3, nullptr);
cudaMalloc(&binding_buffer[0], 3 * sizeof(float));
cudaMalloc(&binding_buffer[1], 6 * sizeof(float));
cudaMalloc(&binding_buffer[2], 9 * sizeof(float));
cudaMemcpy(binding_buffer[0], input_data_0, 3 * sizeof(float),
    ↪ cudaMemcpyHostToDevice);
cudaMemcpy(binding_buffer[1], input_data_1, 6 * sizeof(float),
    ↪ cudaMemcpyHostToDevice);

float output_data_h[9];

```

```
float output_data[9] = {1.0, 2.0, 3.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0};
auto status = context->executeV2(binding_buffer.data());
cudaMemcpy(output_data_h, binding_buffer[2], 9 * sizeof(float),
↳ cudaMemcpyDeviceToHost);
for (auto i = 0; i < 9; ++i) {
    std::cout<<"output:"<<i<<" result:"<<output_data_h[i]<<"
↳ ground:"<<output_data[i]<<std::endl;
}
return 0;
}
```

4.7.2.2 在 Python 中使用用户自定义插件

4.7.2.2.1 加载插件

```
import ixrt
from ixrt import Dims, DynamicPluginTensorDesc, PluginTensorDesc
from ctypes import CDLL
import numpy as np
import cuda.cuda as cuda
import cuda.cudart as cudart

# 加载对应的动态运行库
def load_concat_plugin():
    plugin_path = "./libconcat_plugin.so"
    IXRT_LOGGER = ixrt.Logger(ixrt.Logger.VERBOSE)
    CDLL(plugin_path)
    plugin_creators = ixrt.get_plugin_registry().plugin_creator_list
    for plugin_creator in plugin_creators:
        print("plugin name:", plugin_creator.name)
```

4.7.2.2.2 构建网络时使用插件

```
def create_network_from_plugin(network):
    plugin_registry = ixrt.get_plugin_registry()
    assert plugin_registry
    concat_plugin_creator = plugin_registry.get_plugin_creator("ConcatPlugin_IxRT", "1")
    assert concat_plugin_creator

    axis_field = ixrt.PluginField(
        "axis",
        np.array([1], dtype=np.int32),
        ixrt.PluginFieldType.INT32,
    )
    field_collection = ixrt.PluginFieldCollection([axis_field])
```

```
concat_plugin = concat_plugin_creator.create_plugin(
    "py_concat_plugin_ixrt", field_collection
)

concat_plugin.initialize()

input0 = network.add_input("input0_data", dtype=ixrt.float32, shape=(1, -1, 3))
input1 = network.add_input("input1_data", dtype=ixrt.float32, shape=(1, -1, 3))
concat_layer = network.add_plugin_v2([input0, input1], concat_plugin)

network.mark_output(concat_layer.get_output(0))
```

说明：

通过 get_plugin_creator 接口获取插件创建方法。

4.7.2.2.3 运行带插件的网络

```
def run_network_from_plugin():
    IXRT_LOGGER = ixrt.Logger(ixrt.Logger.ERROR)
    builder = ixrt.Builder(IXRT_LOGGER)
    EXPLICIT_BATCH = 1 << (int)(
        ixrt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH
    )
    network = builder.create_network(EXPLICIT_BATCH)

    create_network_from_plugin(network)

    build_config = builder.create_builder_config()
    precision = ixrt.BuilderFlag.FP16
    build_config.set_flag(precision)
    profile = builder.create_optimization_profile()
    profile.set_shape(
        "input0_data", Dims([1, 1, 3]), Dims([1, 2, 3]), Dims([1, 3, 3])
    )
    profile.set_shape(
        "input1_data", Dims([1, 1, 3]), Dims([1, 2, 3]), Dims([1, 3, 3])
    )
    build_config.add_optimization_profile(profile)

    plan = builder.build_serialized_network(network, build_config)
    runtime = ixrt.Runtime(IXRT_LOGGER)
    engine = runtime.deserialize_cuda_engine(plan)
    context = engine.create_execution_context()
    input_shape = [1, 1, 3]
    input_idx = engine.get_binding_index("input0_data")
    context.set_binding_shape(input_idx, Dims(input_shape))
    input_shape = [1, 2, 3]
```

```

input_idx = engine.get_binding_index("input1_data")
context.set_binding_shape(input_idx, Dims(input_shape))

# Setup I/O bindings
inputs = []
outputs = []
allocations = []
for i in range(engine.num_bindings):
    is_input = False
    if engine.binding_is_input(i):
        is_input = True
    name = engine.get_binding_name(i)
    dtype = engine.get_binding_dtype(i)
    shape = context.get_binding_shape(i)
    if is_input:
        batch_size = shape[0]
        size = np.dtype(ixrt.nptype(dtype)).itemsize
        for s in shape:
            size *= s
        err, allocation = cudart.cudaMalloc(size)
        assert err == cudart.cudaError_t.cudaSuccess
        binding = {
            "index": i,
            "name": name,
            "dtype": np.dtype(ixrt.nptype(dtype)),
            "shape": list(shape),
            "allocation": allocation,
            "nbytes": size,
        }
        allocations.append(allocation)
        if engine.binding_is_input(i):
            inputs.append(binding)
        else:
            outputs.append(binding)
input0_data = np.array([[1,2,3]], np.float32)
input1_data = np.array([[1,2,3],[4,5,6]], np.float32)
input_datas = [input0_data, input1_data]
output_data = np.zeros((1,3,3), np.float32)
for idx, input_data in enumerate(input_datas):
    assert inputs[idx]["nbytes"] == input_data.nbytes
    (err,) = cudart.cudaMemcpy(
        inputs[idx]["allocation"],
        input_data,
        inputs[idx]["nbytes"],
        cudart.cudaMemcpyKind.cudaMemcpyHostToDevice,
    )
    assert err == cudart.cudaError_t.cudaSuccess
# 执行网络
context.execute_v2(allocations)

```

```
(err,) = cudart.cudaMemcpy(
    output_data,
    outputs[0]["allocation"],
    output_data.nbytes,
    cudart.cudaMemcpyKind.cudaMemcpyDeviceToHost,
)
assert err == cudart.cudaError_t.cudaSuccess
print("output_data:", output_data)

for binding_i in inputs + outputs:
    (err,) = cudart.cudaFree(binding_i["allocation"])
    assert err == cudart.cudaError_t.cudaSuccess

if __name__ == "__main__":
    load_concat_plugin()
    run_network_from_plugin()
```

4.7.2.3 使用 ixRT 内置插件

ixRT 提供部分内置插件。

4.7.2.3.1 在 C++ 中使用内置 Plugin

```
#include "NvInfer.h"
#include "NvInferPlugin.h"
// 继承 nvinfer1::ILogger 来实现自定义 Logger
using Severity = nvinfer1::ILogger::Severity;
class Logger : public nvinfer1::ILogger {
public:
    explicit Logger(Severity severity = Severity::kWARNING) :
        ↪ mReportableSeverity(severity) {}
    nvinfer1::ILogger& getIxRTLogger() noexcept { return *this; }
    void log(Severity severity, const char* msg) noexcept override {
        if (severity <= mReportableSeverity) {
            std::cout << "[IXRT] " << msg << std::endl;
        }
    }
    void setReportableSeverity(Severity severity) noexcept { mReportableSeverity =
        ↪ severity; }
    Severity getReportableSeverity() const { return mReportableSeverity; }

private:
    Severity mReportableSeverity;
}; // class Logger

Logger logger(nvinfer1::ILogger::Severity::kVERBOSE);
// 初始化加载所有 plugin
```

```
initLibNvInferPlugins(&logger, "");
// 创建 customfc plugin
std::string pluginName = "CustomFCPluginDynamic_IxRT";
std::string pluginVersion = "1";
int out_dims = 4;
int type_id = 1;
std::vector<float> W;
W.resize(4 * 4);
fill(W.begin(), W.end(), 1.0);
auto creator = getPluginRegistry()->getPluginCreator(pluginName.c_str(),
    pluginVersion.c_str());
std::vector<PluginField> mFC;
mFC.emplace_back(PluginField("out_dims", &out_dims, PluginFieldType::kINT32, 1));
mFC.emplace_back(PluginField("type_id", &type_id, PluginFieldType::kINT32, 1));
mFC.emplace_back(PluginField("W", W.data(), PluginFieldType::kFLOAT32, W.size()));

PluginFieldCollection pfc = PluginFieldCollection(mFC.size(), mFC.data());
auto fc_plugin = creator->createPlugin("fustom_fc", &pfc);
```

4.7.2.3.2 在 Python 中使用内置 Plugin

```
import ixrt
from ctypes import CDLL
import numpy as np

IXRT_LOGGER = ixrt.Logger(ixrt.Logger.VERBOSE)
# 加载所有 plugin
ixrt.init_libnvinfer_plugins(IXRT_LOGGER, '')
plugin_registry = ixrt.get_plugin_registry()
assert plugin_registry
yolov3_decoder_plugin_creator = plugin_registry.get_plugin_creator(
    "YoloV3Decoder", "1"
)

registry_list = plugin_registry.plugin_creator_list
print("registry_list: ", [registry.name for registry in registry_list])

num_class_field = ixrt.PluginField(
    "num_class", np.array([80], dtype=np.int32), ixrt.PluginFieldType.INT32
)
stride_field = ixrt.PluginField(
    "stride", np.array([32], dtype=np.int32), ixrt.PluginFieldType.INT32
)
anchor_field = ixrt.PluginField(
    "anchor",
    np.array([10, 13, 16, 30, 33, 23], dtype=np.float32),
    ixrt.PluginFieldType.FLOAT32,
)
```

```
faster_impl_field = ixrt.PluginField(
    "faster_impl", np.array([1], dtype=np.int32), ixrt.PluginFieldType.INT32
)
field_collection = ixrt.PluginFieldCollection(
    [num_class_field, stride_field, anchor_field, faster_impl_field]
)

yolov3_decoder_plugin = yolov3_decoder_plugin_creator.create_plugin(
    "py_yolov3_decoder_ixrt", field_collection
)
```

4.8 条件控制流推理指南

ixRT v1.0.0 支持条件 if-then-else 流控制，用于实现网络子图的条件执行。

4.8.1 概述

if-then-else 控制流由条件边界层定义，主要包括以下几种层：

- IConditionLayer：用于表示一个谓词 (predicate)，决定是否执行真分支 (true-branch) 或假分支 (false-branch)。它接受一个布尔标量 (0D 标量) 作为输入且没有输出，并根据该条件值选择执行哪个分支。
- IIfConditionalInputLayer：指定条件分支的输入，可以为真分支或假分支提供输入，也可以同时为两个分支提供输入；每个分支可以独立包含 0 个或多个输入。
- IIfConditionalOutputLayer：指定条件分支的输出，依据条件判断的结果，从真分支或假分支的输出中选取一个作为整个条件结构的最终输出，用于在运行时选择真分支或假分支的输出。
- IIfConditional：用于构建条件执行逻辑的一个高级抽象接口，允许在网络中创建基于条件判断的分支结构。
- IIfConditionalBoundaryLayer：作为 IConditionLayer、IIfConditionalInputLayer 和 IIfConditionalOutputLayer 的父类，提供统一的边界控制接口，通过 getConditional() 方法获得相关联的 IIfConditional。

4.8.2 条件控制流的结构

条件控制流由一个布尔标量谓词条件和两个分支子图定义。当条件评估为 true 时，执行 trueSubgraph 子图；当条件评估为 false 时，执行 falseSubgraph 子图。即：

- trueSubgraph：属于 IIfConditional 的两个分支之一，当条件评估为 true 时执行的子图。
- falseSubgraph：属于 IIfConditional 的两个分支之一，当条件评估为 false 时执行的子图。

```
If condition is true then:
    output = trueSubgraph(trueInputs);
Else
    output = falseSubgraph(falseInputs);
Emit output
```

真分支和假分支类似于许多编程语言中的条件判断定义。

4.8.3 构建条件控制流

4.8.3.1 定义条件控制流

要定义一个 if-then-else 条件控制流，需要进行以下操作：

1. 使用 `INetworkDefinition::addIfConditional` 创建一个 `IIfConditional` 实例，然后添加边界层和分支层。

```
IIfConditional* simpleIf = network->addIfConditional();
```

2. 设置条件谓词。条件张量是一个在设备端的 0D 布尔张量 (标量)，可以由网络中的早期层动态计算得到。

```
auto cond = network->addInput("cond", DataType::kBOOL, Dims{0});
IConditionLayer* condition = simpleIf->setCondition(*cond);
```

说明：

- `IIfConditional::setCondition` 方法接受一个参数，即条件张量，用于决定执行哪个分支。
- `IConditionLayer` 有一个输入 (条件) 且没有输出。

3. 添加输入层用于定义分支的输入。

```
IIfConditionalInputLayer* inputX = simpleIf->addInput(*x);
IIfConditionalInputLayer* inputY = simpleIf->addInput(*y);
```

说明：

`IIfConditionalInputLayer` 将一个 Tensor 标记为条件控制子图的输入，其输出可以被任意的真分支或假分支引用。真分支和假分支的输入不要求是相同的类型和形状，每个分支可以独立包含 0 个或多个输入。

4. 定义真分支和假分支的子图。

```
auto* trueSubgraph = n.addElementWise(*inputX, *inputY,
    ↪ ElementWiseOperation::kSUM)->getOutput(0);
auto* falseSubgraph = n.addElementWise(*inputX, *inputY,
    ↪ ElementWiseOperation::kSUB)->getOutput(0);
```

5. 添加输出层用于定义条件的输出。

```
IIfConditionalOutputLayer* outputLayer = simpleIf->addOutput(*trueSubgraph,
    ↪ *falseSubgraph);
```

说明：

- `IIfConditionalOutputLayer` 抽象了 if 条件的单个输出。它有两个输入，分别是：来自 true 子图的输出 (输入索引 0) 和来自 false 子图的输出 (输入索引 1)。
- `IIfConditionalOutputLayer` 的输出可以被认为是最终输出的占位符，最终输出将在运行时确定。
- `IIfConditionalOutputLayer` 的作用类似于传统 SSA 控制流图中的 $\phi(\Phi)$ 函数节点，其语义是：选择 true 子图或 false 子图的输出。
- `IIfConditional` 的所有输出都必须源自 `IIfConditionalOutputLayer` 实例。

4.8.3.2 抽象模型结构图

下图提供了 if 条件抽象模型的图形表示，绿色矩阵表示条件的内部。

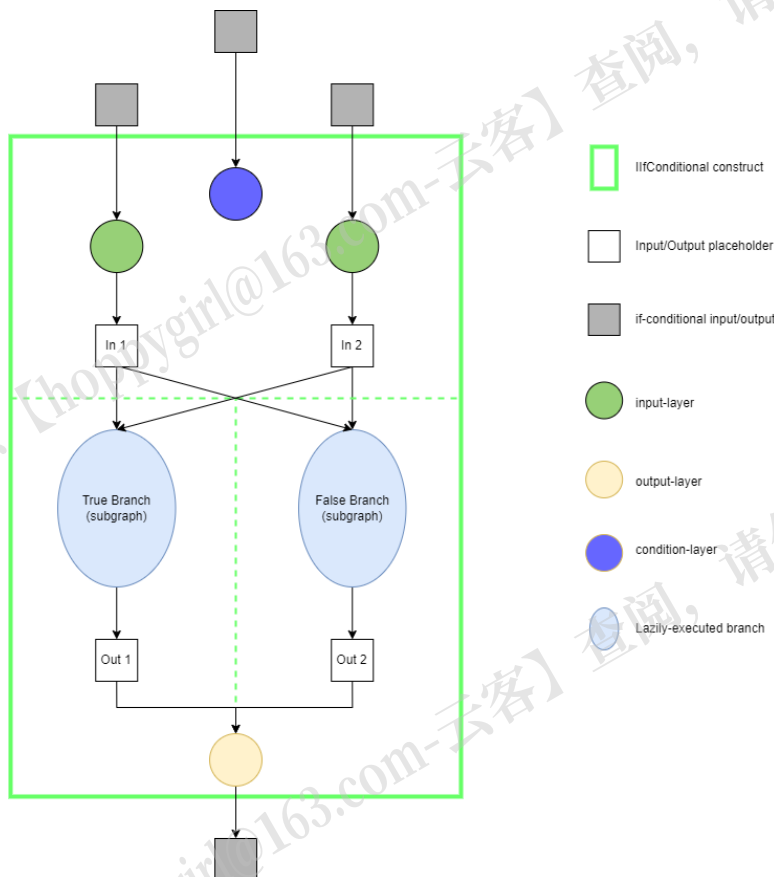


图 1: if 条件抽象模型示意图

4.8.3.3 条件控制流使用示例

以下是一个简单的 if-then-else 条件控制流示例。如果条件为 true，则执行加法操作，否则执行减法操作。

```
ITensor* addCondition(INetworkDefinition& n, bool predicate)
{
    static const Dims scalarDims = Dims{0, {}};
    static float constexpr zero{0};
    static float constexpr one{1};

    float* const val = predicate ? &one : &zero;

    ITensor* cond = n.addConstant(scalarDims, DataType::kINT32, val, 1)->getOutput(0);

    auto* cast = n.addIdentity(cond);
    cast->setOutputType(0, DataType::kB00L);
    cast->getOutput(0)->setType(DataType::kB00L);
```

```

return cast->getOutput(0);
}

IBuilder* builder = createInferBuilder(glogger);
INetworkDefinition& n = *builder->createNetworkV2(0U);
auto x = n.addInput("x", DataType::kFLOAT, Dims{1, {5}});
auto y = n.addInput("y", DataType::kFLOAT, Dims{1, {5}});
ITensor* cond = addCondition(n, true);

auto* simpleIf = n.addIfConditional();
simpleIf->setCondition(*cond);

auto inputX = simpleIf->addInput(*x)->getOutput(0);
auto inputY = simpleIf->addInput(*y)->getOutput(0);

auto* trueSubgraph = n.addElementWise(*inputX, *inputY,
    ElementWiseOperation::kSUM)->getOutput(0);
auto* falseSubgraph = n.addElementWise(*inputX, *inputY,
    ElementWiseOperation::kSUB)->getOutput(0);

auto* output = simpleIf->addOutput(*trueSubgraph, *falseSubgraph)->getOutput(0);
n.markOutput(*output);

```

4.8.3.4 使用限制与注意事项

- 组件实例数量：
 - 必须恰好有一个 IConditionLayer 实例，用于表示条件判断的谓词并指定执行的分支。
 - 可以有 0 个或多个 IIfConditionalInputLayer 实例，其作用是标记条件分支的输入，且是可选的，用于控制哪些层属于分支。
 - 至少有一个 IIfConditionalOutputLayer 实例，用于标记条件的单个输出，并决定真分支和假分支子图的边界。
- 分支输出的定义：
 - 每个分支 (子图) 必须至少有一个输出，如果没有输出，会被认为是不合法的。
 - 且分支的输出可以标记为网络的输出，也可以作为其它算子的输入。
- 输出一致性：真分支或假分支的输出必须在形状和数据类型上完全一致。
- 循环依赖：不能创建循环依赖的条件结构。
- 嵌套条件：可以在条件结构中嵌套其它条件结构，以实现更复杂的控制流。

4.8.4 执行策略

条件控制流的执行支持以下两种策略：

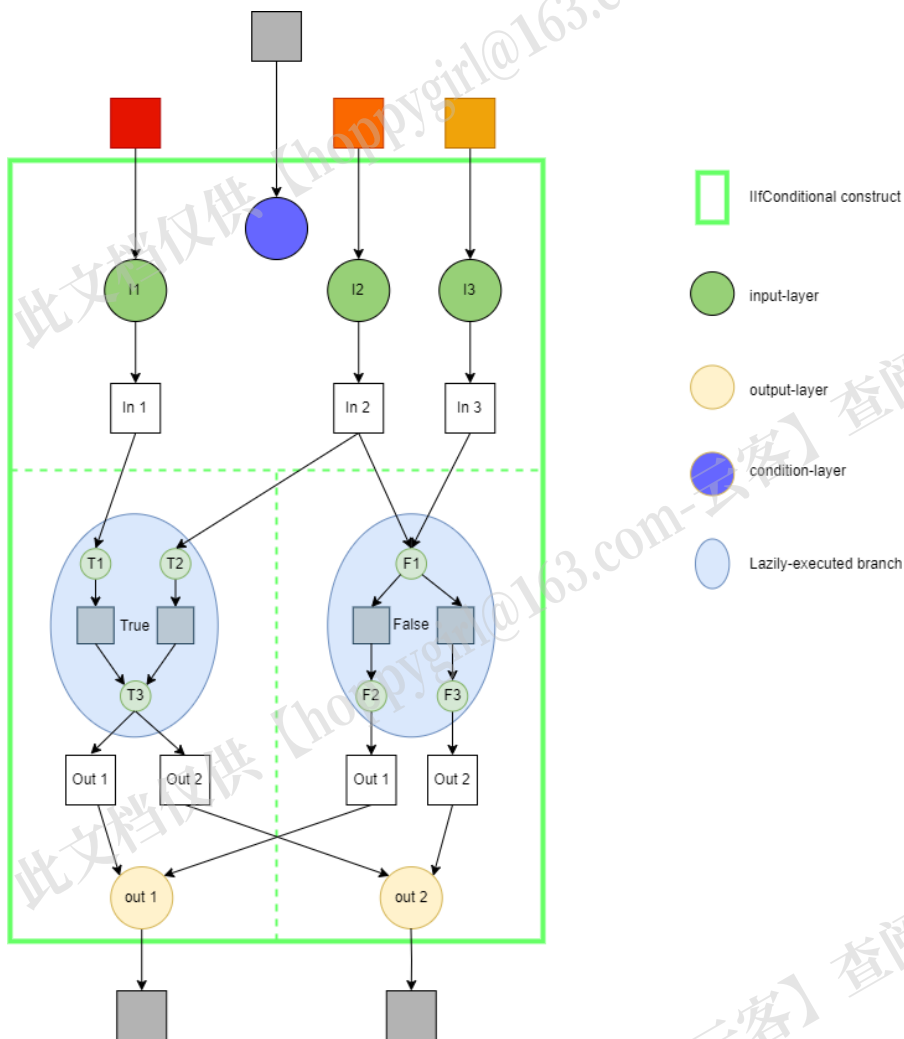
- Lazy Execution：根据条件选中的分支执行推理流程，并将结果输出至后续算子或模型输出。即在这种执行策略中，要么执行真分支，要么执行假分支，但不会同时执行两者，被选中的分支的输出会被传递至后续层。

- Eager Execution：在这种执行策略中，所有不在分支子图中的算子都会在条件结构之前被执行。

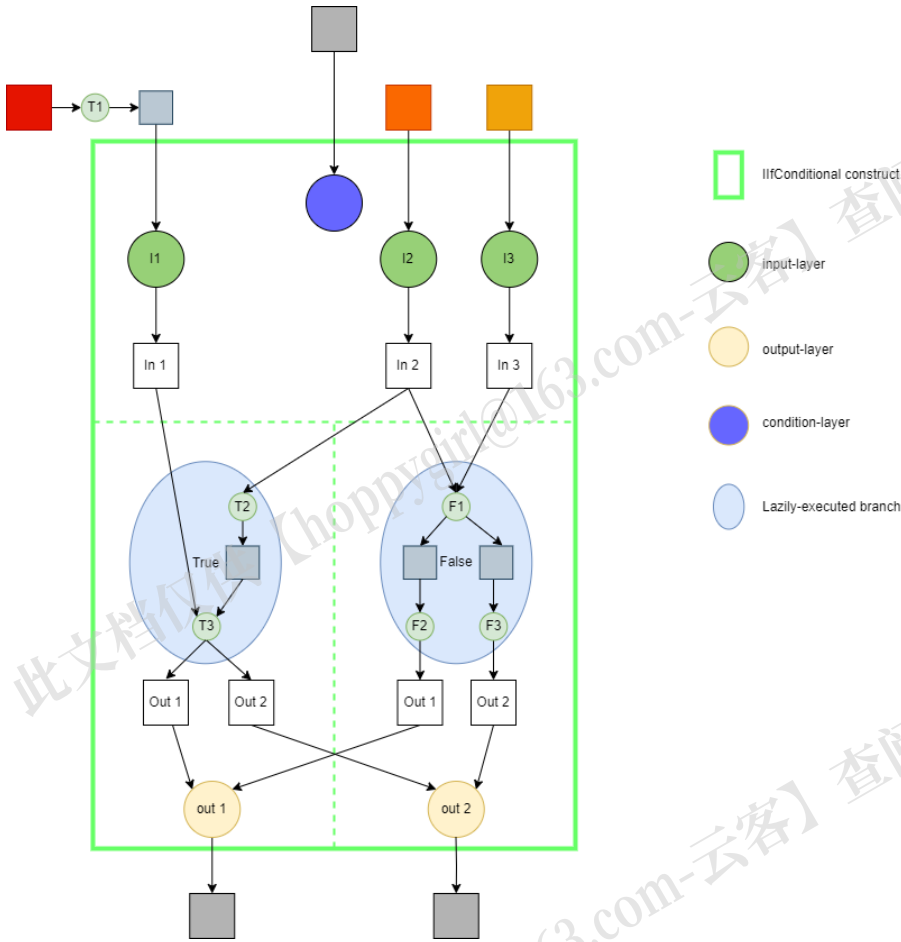
控制执行策略

IIfConditionalInputLayer 是一个特殊的层，用于标记条件子图的输入。通过其，可以追踪哪些层依赖于条件输入，从而决定这些层是否需要条件执行。例如，如果某个层依赖于 IIfConditionalInputLayer 的输出，则该层及其依赖的所有后续层都会被视为条件子图的一部分，因此会被 Lazy Execution；如果某个层不依赖于任何 IIfConditionalInputLayer，则该层会被 Eager Execution。在极端情况下，没有添加任何的 IIfConditionalInputLayer，则所有层都会被 Eager Execution。

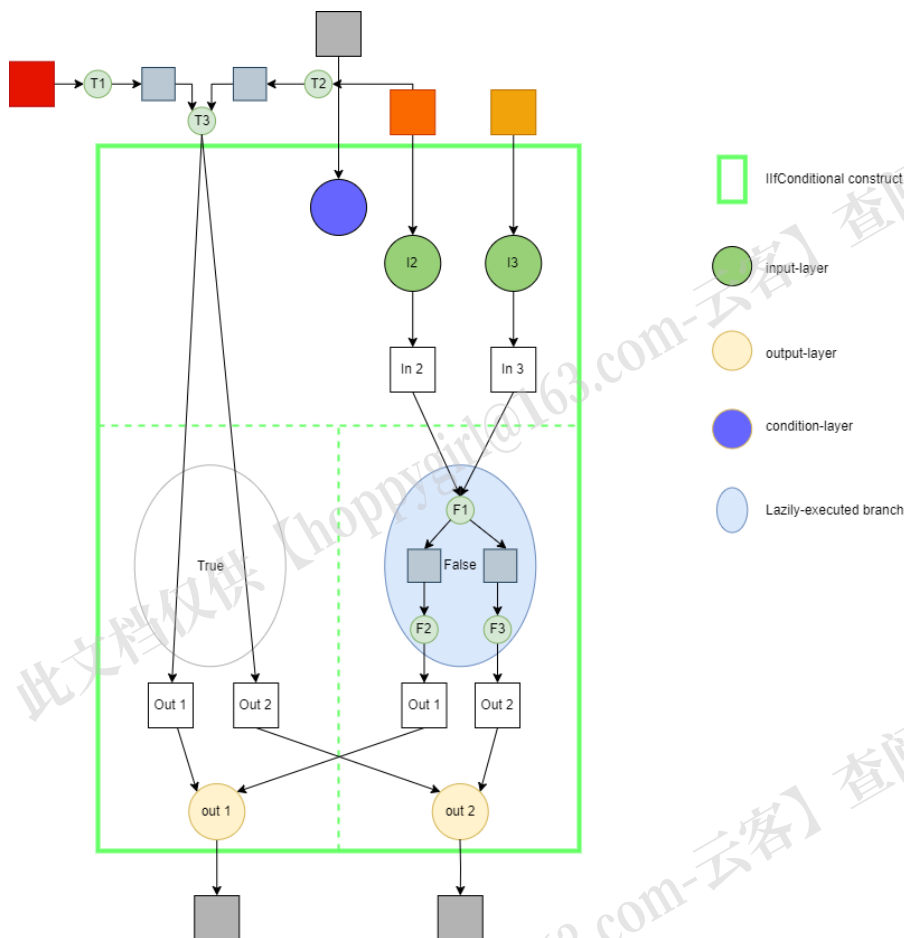
以下三张示意图描述了 IIfConditionalInputLayer 放置的位置控制执行调度。



在上图中，真分支由 T1、T2 和 T3 组成，当条件评估为 true 时，这些层都是 Lazy Execution。



在上图中，输入层 I1 放置在 T1 层之后，将 T1 从真分支中移除。这种情况下，在评估条件结构之前，T1 层会先被执行。



在上图中，输入层 I1 被移除，并将 T1、T2 和 T3 层从真分支中移除。这种情况下，当条件结构评估为 true 时，真分支不计算任何内容，因为输出已经提前计算了。

4.8.5 嵌套条件控制流

嵌套条件控制流是指在一个条件分支中嵌套另一个条件控制流，用于实现更复杂的控制流。即通过在真分支或假分支中再创建一个新的 `addIfConditional` 来实现。

以下代码是嵌套条件控制流的使用示例。

```
auto cond = network->addInput("cond", DataType::kBOOL, Dims{0});
auto x = network->addInput("x", DataType::kFLOAT, Dims{1, {9}});
auto y = network->addInput("y", DataType::kFLOAT, Dims{1, {9}});

auto condition_marker = network->addIfConditional();
condition_marker->setName("condition_marker");
condition_marker->setCondition(*cond);

// Add input layers to demarcate entry into true/false branches.
IfConditionalInputLayer *condition_in_x = condition_marker->addInput(*x);
auto in_x = condition_in_x->getOutput(0);
IfConditionalInputLayer *condition_in_y = condition_marker->addInput(*y);
```

```

auto in_y = condition_in_y->getOutput(0);

// true subgraph
auto *true_subgraph = network->addElementWise(*in_x, *in_y,
↳ ElementWiseOperation::kSUM)->getOutput(0);

// false subgraph
auto *sub_out_tensor = network->addElementWise(*in_x, *in_y,
↳ ElementWiseOperation::kSUB)->getOutput(0);

// nest false subgraph
auto nest_cond = network->addInput("nest_cond", DataType::kBOOL, Dims{0});
auto z = network->addInput("z", DataType::kFLOAT, Dims{1, {9}});

auto nest_condition_marker = network->addIfConditional();
nest_condition_marker->setName("nest_condition_marker");
nest_condition_marker->setCondition(*nest_cond);

IIfConditionalInputLayer *nest_condition_in_x =
↳ nest_condition_marker->addInput(*sub_out_tensor);
auto nest_in_x = nest_condition_in_x->getOutput(0);
IIfConditionalInputLayer *nest_condition_in_z = nest_condition_marker->addInput(*z);
auto nest_in_z = nest_condition_in_z->getOutput(0);

auto *true_nest_subgraph =
network->addElementWise(*nest_in_x, *nest_in_z,
↳ ElementWiseOperation::kSUM)->getOutput(0);
auto *false_nest_subgraph =
network->addElementWise(*nest_in_x, *nest_in_z,
↳ ElementWiseOperation::kSUB)->getOutput(0);
IIfConditionalOutputLayer *nest_condition_output =
nest_condition_marker->addOutput(*true_nest_subgraph, *false_nest_subgraph);
auto *false_subgraph = nest_condition_output->getOutput(0);

IIfConditionalOutputLayer *condition_output = condition_marker->addOutput(*true_subgraph,
↳ *false_subgraph);
auto *output = condition_output->getOutput(0);
network->markOutput(*output);

```

5 配套软件 CUDA Python 使用指南

CUDA Python 可以让您以 Pythonic 的方式轻松访问天数智算软件栈支持的 CUDA API。CUDA Python 为 CUDA Driver 和 Runtime API 提供了 Cython 和 Python 封装，简化了基于 GPU 的并行计算。在使用 ixRT Python API 进行部署模型时，您可以使用 CUDA Python 来实现数据 H2D、D2H 和同步等操作。

本次发布适配来自 <https://github.com/NVIDIA/cuda-python>。

5.1 安装 CUDA Python

通过以下方式获取 .whl 包并使用 **pip3** 进行安装：

- 登录 [天数智芯官网](#)，进入 **客户支持 > 资源中心** 页面进行下载
- 联系您的应用工程师获取

```
$ pip3 install cuda_python-{v.r.m}
  +corex.{v.r.m}-cp{python_version}-cp{python_version}-linux_{arch}.whl
```

5.2 使用示例

使用 Runtime API 示例如下：

```
import cuda.cudart as cudart
import cuda.cuda as cuda
import numpy as np
import ctypes
import base64

def cudaGetErrorEnum(error):
    if isinstance(error, cuda.CUresult):
        err, name = cuda.cuGetErrorName(error)
        return name if err == cuda.CUresult.CUDA_SUCCESS else "<unknown>"
    elif isinstance(error, cudart.cudaError_t):
        return cudart.cudaGetErrorName(error)[1]
    else:
        raise RuntimeError('Unknown error type: {}'.format(error))

def checkCudaErrors(result):
    if result[0].value:
        raise RuntimeError("CUDA error code={}({})".format(result[0].value,
            cudaGetErrorEnum(result[0])))
    if len(result) == 1:
        return None
    elif len(result) == 2:
        return result[1]
    else:
```

```

        return result[1:]

def assertSuccess(err):
    assert(err == cudart.cudaError_t.cudaSuccess)

def test_cudart_memcpy():
    # Set device
    devID = 0
    checkCudaErrors(cudart.cudaSetDevice(devID))

    # Allocate dev memory
    size = 1024 * np.uint8().itemsize
    err, dptr = cudart.cudaMalloc(size)
    assertSuccess(err)

    # Set h1 and h2 memory to be different
    h1 = np.full(size, 1).astype(np.uint8)
    h2 = np.full(size, 2).astype(np.uint8)
    assert(np.array_equal(h1, h2) is False)

    # mem set
    cudart.cudaMemset(dptr, 0, size)
    assertSuccess(err)

    # h1 to D
    err, = cudart.cudaMemcpy(dptr, h1, size, cudart.cudaMemcpyKind.cudaMemcpyHostToDevice)
    assertSuccess(err)

    # D to h2
    err, = cudart.cudaMemcpy(h2, dptr, size, cudart.cudaMemcpyKind.cudaMemcpyDeviceToHost)
    assertSuccess(err)

    # Validate h1 == h2
    assert(np.array_equal(h1, h2))

    # Cleanup
    err, = cudart.cudaFree(dptr)
    assertSuccess(err)

def test_cudart_stream():
    stream_legacy = cudart.cudaStream_t(cudart.cudaStreamLegacy)
    stream_per_thread = cudart.cudaStream_t(cudart.cudaStreamPerThread)

    err_rt, stream_with_flags =
    ↪ cudart.cudaStreamCreateWithFlags(cudart.cudaStreamNonBlocking)
    assertSuccess(err_rt)
    err_rt, = cudart.cudaStreamSynchronize(stream_with_flags)
    assertSuccess(err_rt)
    err_rt, = cudart.cudaStreamDestroy(stream_with_flags)
    assert(err_rt == cudart.cudaError_t.cudaSuccess)

```

```

error = cudart.cudaGetLastError()[0]
assert error in [
    cudart.cudaError_t.cudaSuccess,
    cudart.cudaError_t.cudaErrorPeerAccessAlreadyEnabled,
    cudart.cudaError_t.cudaErrorPeerAccessNotEnabled
]

def test_cuda_device_access_peer():
    # Number of GPUs
    print("Checking for multiple GPUs...")
    gpu_n = checkCudaErrors(cudart.cudaGetDeviceCount())
    print("CUDA-capable device count: {}".format(gpu_n))

    if gpu_n < 2:
        print("Two or more GPUs with Peer-to-Peer access capability are required")
        return

    prop = [checkCudaErrors(cudart.cudaGetDeviceProperties(i)) for i in range(gpu_n)]
    # Check possibility for peer access
    print("\nChecking GPU(s) for support of peer to peer memory access...")

    p2pCapableGPUs = [-1, -1]
    for i in range(gpu_n):
        p2pCapableGPUs[0] = i
        for j in range(gpu_n):
            if i == j:
                continue
            i_access_j = checkCudaErrors(cudart.cudaDeviceCanAccessPeer(i, j))
            j_access_i = checkCudaErrors(cudart.cudaDeviceCanAccessPeer(j, i))
            print("> Peer access from {} (GPU{}) -> {} (GPU{}) : {}\n".format(
                prop[i].name, i, prop[j].name, j, "Yes" if i_access_j else "No"))
            print("> Peer access from {} (GPU{}) -> {} (GPU{}) : {}\n".format(
                prop[j].name, j, prop[i].name, i, "Yes" if j_access_i else "No"))
            if i_access_j and j_access_i:
                p2pCapableGPUs[1] = j
                break
        if p2pCapableGPUs[1] != -1:
            break

    if p2pCapableGPUs[0] == -1 or p2pCapableGPUs[1] == -1:
        print("Two or more GPUs with Peer-to-Peer access capability are required.")
        print("Peer to Peer access is not available amongst GPUs in the system, waiving
        ↪ test.")
        return

    # Use first pair of p2p capable GPUs detected
    gpuid = [p2pCapableGPUs[0], p2pCapableGPUs[1]]

    # Enable peer access

```

```

print("Enabling peer access between GPU{} and GPU{}..."
      .format(gpuid[0], gpuid[1]))
checkCudaErrors(cudaSetDevice(gpuid[0]))
checkCudaErrors(cudaDeviceEnablePeerAccess(gpuid[1], 0))
checkCudaErrors(cudaSetDevice(gpuid[1]))
checkCudaErrors(cudaDeviceEnablePeerAccess(gpuid[0], 0))

# Allocate buffers
buf_size = 1024 * 1024 * 16 * np.dtype(np.float32).itemsize
print("Allocating buffers ({}MB on GPU{}, GPU{} and CPU Host)..."
      .format(int(buf_size /
        ↪ 1024 / 1024), gpuid[0], gpuid[1]))
checkCudaErrors(cudaSetDevice(gpuid[0]))
g0 = checkCudaErrors(cudaMalloc(buf_size))
checkCudaErrors(cudaSetDevice(gpuid[1]))
g1 = checkCudaErrors(cudaMalloc(buf_size))
h0 = checkCudaErrors(cudaMallocHost(buf_size)) # Automatically portable with UVA

# Create CUDA event handles
print("Creating event handles...")
eventflags = cuda.EventBlockingSync
start_event = checkCudaErrors(cuda.EventCreateWithFlags(eventflags))
stop_event = checkCudaErrors(cuda.EventCreateWithFlags(eventflags))

# P2P memcpy() benchmark
checkCudaErrors(cuda.EventRecord(start_event, cuda.Stream_t(0)))

for i in range(100):
    # With UVA we don't need to specify source and target devices, the
    # runtime figures this out by itself from the pointers
    # Ping-pong copy between GPUs
    if i % 2 == 0:
        checkCudaErrors(cudaMemcpy(g1, g0, buf_size,
        ↪ cudaMemcpyKind.cudaMemcpyDefault))
    else:
        checkCudaErrors(cudaMemcpy(g0, g1, buf_size,
        ↪ cudaMemcpyKind.cudaMemcpyDefault))

    checkCudaErrors(cuda.EventRecord(stop_event, cuda.Stream_t(0)))
    checkCudaErrors(cuda.EventSynchronize(stop_event))
    time_memcpy = checkCudaErrors(cuda.EventElapsedTime(start_event, stop_event))
    print("cudaMemcpyPeer / cudaMemcpy between GPU{} and GPU{}: {:.2f}GB/
    ↪ s".format(gpuid[0], gpuid[1],
        (1.0 / (time_memcpy / 1000.0)) * ((100.0 * buf_size) / 1024.0 / 1024.0 /
        ↪ 1024.0))

    # Prepare host buffer and copy to GPU 0
    print("Preparing host buffer and memcpy to GPU{}..."
          .format(gpuid[0]))

    h0_local = (ctypes.c_float * int(buf_size /
    ↪ np.dtype(np.float32).itemsize)).from_address(h0)
    for i in range(int(buf_size / np.dtype(np.float32).itemsize)):

```

```

h0_local[i] = i % 4096

checkCudaErrors(cudaSetDevice(gpuid[0]))
checkCudaErrors(cudaMemcpy(g0, h0, buf_size,
↪ cudaMemcpyKind.cudaMemcpyDefault))

# Disable peer access (also unregisters memory for non-UVA cases)
print("Disabling peer access...")
checkCudaErrors(cudaSetDevice(gpuid[0]))
checkCudaErrors(cudaDeviceDisablePeerAccess(gpuid[1]))
checkCudaErrors(cudaSetDevice(gpuid[1]))
checkCudaErrors(cudaDeviceDisablePeerAccess(gpuid[0]))

# Cleanup and shutdown
print("Shutting down...")
checkCudaErrors(cudaEventDestroy(start_event))
checkCudaErrors(cudaEventDestroy(stop_event))
checkCudaErrors(cudaSetDevice(gpuid[0]))
checkCudaErrors(cudaFree(g0))
checkCudaErrors(cudaSetDevice(gpuid[1]))
checkCudaErrors(cudaFree(g1))
checkCudaErrors(cudaFreeHost(h0))

for i in range(gpu_n):
    checkCudaErrors(cudaSetDevice(i))

def test_interop_graphExec():
    err_dr, = cuda.cuInit(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, device = cuda.cuDeviceGet(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, ctx = cuda.cuCtxCreate(0, device)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, graph = cuda.cuGraphCreate(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, node = cuda.cuGraphAddEmptyNode(graph, [], 0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

    # DRV to RT
    err_dr, graphExec, errorNode = cuda.cuGraphInstantiate(graph, b'', 0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_rt, = cudart.cudaGraphExecDestroy(graphExec)
    assert(err_rt == cudart.cudaError_t.cudaSuccess)

    # RT to DRV
    err_rt, graphExec, errorNode = cudart.cudaGraphInstantiate(graph, b'', 0)
    assert(err_rt == cudart.cudaError_t.cudaSuccess)
    err_dr, = cuda.cuGraphExecDestroy(graphExec)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

```

```

err_rt, = cudart.cudaGraphDestroy(graph)
assert(err_rt == cudart.cudaError_t.cudaSuccess)
err_dr, = cuda.cuCtxDestroy(ctx)
assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

def test_cudart_cudaStreamGetCaptureInfo():
    # create stream
    err, stream = cudart.cudaStreamCreate()
    assertSuccess(err)

    # validate that stream is not capturing
    err, status, pid = cudart.cudaStreamGetCaptureInfo(stream)
    assertSuccess(err)
    assert(status == cudart.cudaStreamCaptureStatus.cudaStreamCaptureStatusNone)

    # start capture
    err, = cudart.cudaStreamBeginCapture(
        stream, cudart.cudaStreamCaptureMode.cudaStreamCaptureModeGlobal
    )
    assertSuccess(err)

    # validate that stream is capturing now
    err, status, pid = cudart.cudaStreamGetCaptureInfo(stream)
    assertSuccess(err)
    assert(status == cudart.cudaStreamCaptureStatus.cudaStreamCaptureStatusActive)

    # clean up
    err, pgraph = cudart.cudaStreamEndCapture(stream)
    assertSuccess(err)

def test_ipc():
    BUFFER_SIZE = 1024 * np.uint8().itemsize

    err, = cudart.cudaSetDevice(0)
    assertSuccess(err)

    host_buffer = np.full(BUFFER_SIZE, 1).astype(np.uint8)

    err, device_buffer_ptr = cudart.cudaMalloc(BUFFER_SIZE)
    assertSuccess(err)

    err, ipc_mem_handle = cudart.cudaIpcGetMemHandle(device_buffer_ptr)
    assertSuccess(err)
    memory_handle_str = base64.b64encode(ipc_mem_handle.reserved).decode('utf-8')

    err, = cudart.cudaMemcpy(device_buffer_ptr, host_buffer, BUFFER_SIZE,
        ↪ cudart.cudaMemcpyKind.cudaMemcpyHostToDevice)
    assertSuccess(err)

```

```

host_buffer = np.full(BUFFER_SIZE, 0).astype(np.uint8)
err, = cudart.cudaMemcpy(host_buffer, device_buffer_ptr, BUFFER_SIZE,
↪ cudart.cudaMemcpyKind.cudaMemcpyDeviceToHost)
assertSuccess(err)

err, = cudart.cudaFree(device_buffer_ptr)
assertSuccess(err)

err, = cudart.cudaDeviceReset()
assertSuccess(err)

err_dr, = cuda.cuInit(0)
assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

err_dr, device = cuda.cuDeviceGet(0)
assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

err_dr, ctx = cuda.cuCtxCreate(0, device)
assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

new_mem_hdl = cudart.cudaIpcMemHandle_t()
new_mem_hdl.reserved = base64.b64decode(memory_handle_str)
err, devPtr =
↪ cudart.cudaIpcOpenMemHandle(new_mem_hdl, cudart.cudaIpcMemLazyEnablePeerAccess)
assertSuccess(err)

host_buffer = np.full(BUFFER_SIZE, 0).astype(np.uint8)
err, = cudart.cudaMemcpy(host_buffer, devPtr, BUFFER_SIZE,
↪ cudart.cudaMemcpyKind.cudaMemcpyDeviceToHost)
assertSuccess(err)

host_buffer = np.full(BUFFER_SIZE, 8).astype(np.uint8)
err, = cudart.cudaMemcpy(devPtr, host_buffer, BUFFER_SIZE,
↪ cudart.cudaMemcpyKind.cudaMemcpyHostToDevice)
assertSuccess(err)

err, = cudart.cudaIpcCloseMemHandle(devPtr)
assertSuccess(err)

err, = cudart.cudaDeviceReset()
assertSuccess(err)

err_dr, = cuda.cuCtxDestroy(ctx)
assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

```

使用 Driver API 示例如下：

```
import cuda.cuda as cuda
import cuda.cudart as cudart
import numpy as np

def ASSERT_DRV(err):
    if isinstance(err, cuda.CUresult):
        if err != cuda.CUresult.CUDA_SUCCESS:
            raise RuntimeError('Cuda Error: {}'.format(err))
    elif isinstance(err, cudart.cudaError_t):
        if err != cudart.cudaError_t.cudaSuccess:
            raise RuntimeError('Cudart Error: {}'.format(err))
    else:
        raise RuntimeError('Unknown error type: {}'.format(err))

def test_cuda_memcpy():
    # Init CUDA
    err, = cuda.cuInit(0)
    assert(err == cuda.CUresult.CUDA_SUCCESS)

    # Get device
    err, device = cuda.cuDeviceGet(0)
    assert(err == cuda.CUresult.CUDA_SUCCESS)

    # Construct context
    err, ctx = cuda.cuCtxCreate(0, device)
    assert(err == cuda.CUresult.CUDA_SUCCESS)

    # Allocate dev memory
    size = int(1024 * np.uint8().itemsize)
    err, dptr = cuda.cuMemAlloc(size)
    assert(err == cuda.CUresult.CUDA_SUCCESS)

    # Set h1 and h2 memory to be different
    h1 = np.full(size, 1).astype(np.uint8)
    h2 = np.full(size, 2).astype(np.uint8)
    assert(np.array_equal(h1, h2) is False)

    # h1 to D
    err, = cuda.cuMemcpyHtoD(dptr, h1, size)
    assert(err == cuda.CUresult.CUDA_SUCCESS)

    # D to h2
    err, = cuda.cuMemcpyDtoH(h2, dptr, size)
    assert(err == cuda.CUresult.CUDA_SUCCESS)

    # Validate h1 == h2
    assert(np.array_equal(h1, h2))

    # Cleanup
    err, = cuda.cuMemFree(dptr)
```

```

assert(err == cuda.CUresult.CUDA_SUCCESS)
err, = cuda.cuCtxDestroy(ctx)
assert(err == cuda.CUresult.CUDA_SUCCESS)

def test_interop_stream():
    err_dr, = cuda.cuInit(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, device = cuda.cuDeviceGet(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, ctx = cuda.cuCtxCreate(0, device)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

    # DRV to RT
    err_dr, stream = cuda.cuStreamCreate(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

    # Event
    err_dr, event0 = cuda.cuEventCreate(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, event1 = cuda.cuEventCreate(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    cuda.cuEventRecord(event0, stream)
    cuda.cuEventRecord(event1, stream)
    _, ms = cuda.cuEventElapsedTime(event0, event1)

    err_rt, = cudart.cudaStreamDestroy(stream)
    assert(err_rt == cudart.cudaError_t.cudaSuccess)

    # RT to DRV
    err_rt, stream = cudart.cudaStreamCreate()
    assert(err_rt == cudart.cudaError_t.cudaSuccess)
    err_dr, = cuda.cuStreamDestroy(stream)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

    err_dr, = cuda.cuCtxDestroy(ctx)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

def test_interop_event():
    err_dr, = cuda.cuInit(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, device = cuda.cuDeviceGet(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, ctx = cuda.cuCtxCreate(0, device)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

    # DRV to RT
    err_dr, event = cuda.cuEventCreate(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_rt, = cudart.cudaEventDestroy(event)

```

```

assert(err_rt == cudart.cudaError_t.cudaSuccess)

# RT to DRV
err_rt, event = cudart.cudaEventCreate()
assert(err_rt == cudart.cudaError_t.cudaSuccess)
err_dr, = cuda.cuEventDestroy(event)
assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

err_dr, = cuda.cuCtxDestroy(ctx)
assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

def test_device_Get_attribute():
    err_dr, = cuda.cuInit(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, device = cuda.cuDeviceGet(0)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)
    err_dr, ctx = cuda.cuCtxCreate(0, device)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

    err, major =
    ↪ cuda.cuDeviceGetAttribute(cuda.CUdevice_attribute.CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_MAJOR,
    ↪ device)
    ASSERT_DRV(err)
    err, minor =
    ↪ cuda.cuDeviceGetAttribute(cuda.CUdevice_attribute.CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_MINOR,
    ↪ device)
    ASSERT_DRV(err)

    err_dr, = cuda.cuCtxDestroy(ctx)
    assert(err_dr == cuda.CUresult.CUDA_SUCCESS)

```

5.3 API 参考

当前支持以下 CUDA Python Runtime API:

```

cudart.cudaSetDevice
cudart.cudaMalloc
cudart.cudaMallocHost
cudart.cudaMallocAsync
cudart.cudaMemset
cudart.cudaMemcpy
cudart.cudaMemcpyAsync
cudart.cudaMemcpyKind
cudart.cudaFree
cudart.cudaFreeHost
cudart.cudaFreeAsync
cudart.cudaStream_t
cudart.cudaStreamLegacy

```

```
cudaStreamPerThread
cudaStreamCreateWithFlags
cudaStreamNonBlocking
cudaStreamSynchronize
cudaStreamCreate
cudaStreamDestroy
cudaStreamGetCaptureInfo
cudaStreamCaptureStatus
cudaStreamCaptureMode
cudaStreamGetCaptureInfo
cudaStreamBeginCapture
cudaStreamEndCapture
cudaGraphInstantiate
cudaGraphExecDestroy
cudaGraphDestroy
cudaGetLastError
cudaError_t
cudaGetDeviceCount
cudaGetDeviceProperties
cudaDeviceCanAccessPeer
cudaDeviceEnablePeerAccess
cudaDeviceReset
cudaEventBlockingSync
cudaEventCreateWithFlags
cudaEventRecord
cudaEventSynchronize
cudaEventElapsedTime
cudaEventDestroy
cudaIpcGetMemHandle
cudaIpcMemHandle_t
cudaIpcMemLazyEnablePeerAccess
cudaIpcOpenMemHandle
cudaIpcCloseMemHandle
```

当前支持以下 CUDA Python Driver API:

```
cuda.cuInit
cuda.CUresult
cuda.cuDeviceGet
cuda.cuCtxCreate
cuda.cuCtxDestroy
cuda.cuDeviceGetAttribute
cuda.CUdevice_attribute
cuda.cuMemAlloc
cuda.cuMemcpyHtoD
cuda.cuMemcpyHtoDAsync
cuda.cuMemcpyDtoH
cuda.cuMemcpyDtoHAsync
cuda.cuMemFree
cuda.cuStreamCreate
```

```
cuda.cuStreamDestroy  
cuda.cuStreamSynchronize  
cuda.cuEventCreate  
cuda.cuEventRecord  
cuda.cuEventElapsedTime
```

6 ixRT 推理引擎辅助工具使用指南

ixRT 推理引擎提供一些辅助性工具供您使用，包括：

- 插件生成工具使用指南
- ixrtexec 命令行工具使用指南
- ixRT Model Optimizer 图融合工具使用指南
- ixRT Deploy 量化工具使用指南

本章节将详细介绍上述工具的使用方法，并提供使用示例供您参考。

6.1 插件生成工具使用指南

在开发自定义插件算子时，您可以通过 TPG (TensorRT Plugin Generator) 工具，并基于 ONNX 或 YAML 文件轻松生成 ixRT 插件算子，避免绝大多数模板代码的编写，从而只需实现关键部分。

本次发布适配来自 https://github.com/NVIDIA-AI-IOT/tensorrt_plugin_generator。

6.1.1 安装插件生成工具

通过以下方式获取 .whl 包并使用 pip3 进行安装：

- 登录 [天数智芯官网](#)，进入 **客户支持 > 资源中心** 页面进行下载
- 联系您的应用工程师获取

```
$ pip3 install trtpg-{v.r.m}
↪ +corex.{v.r.m}-cp{python_version}-cp{python_version}-linux_{arch}.whl
```

6.1.2 使用插件生成工具

使用 ixRT 提供的插件生成工具的操作步骤如下：

1. 假设您的 ONNX 中包含 ixRT 不支持的 custom_add 和 custom_conv 两个算子，请使用如下命令转为 YAML 配置文件：

```
$ trtpg extract --onnx model.onnx --custom_operators custom_add,custom_conv --yaml
↪ plugin.yml
```

2. 检查生成 YAML 配置文件中是否包含 **need_user_to_specify** 项。如果包含，说明 TPG 无法从 ONNX 中推断出该项信息，此时需要您手动更改为对应的值。参考 [官方教程](#) 了解 YAML 文件的修改规则。
3. 生成插件代码：

```
$ trtpg generate --yaml plugin.yml --output ./plugin_codes
```

在生成插件代码的过程中，您可能需要指定 ixRT 头文件的安装路径、库文件安装路径与 CUDA 安装路径。参考如下命令了解对应传参方式：

```
$ trtpg generate -h
```

4. 完成插件推理代码。

您需要实现以下方式：

方法名	是否必须要实现	解释
getOutputDimensions()	必须	形状推导，计算输出的形状
enqueue()	必须	最重要的部分，该算子的计算逻辑
initialize()	可选	为运行时额外分配的内存
terminate()	可选	释放所有 initialize() 中分配的内存
configurePlugin()	可选	在动态形状下，可以调用此方法，效果是不同形状对应不同的最优实现
getWorkspaceSize()	可选	initialize() 需要分配的内存大小

更多使用细节，请参考 [官方文档](#)。

6.2 ixrtexec 命令行工具使用指南

ixrtexec 是 ixRT 提供的一个命令行工具，用于在不开发代码的情况下快速运行和验证模型。

6.2.1 参数说明

ixrtexec 可使用的参数如下表所列。

参数	说明
--onnx	必选，指定待运行的 onnx 模型，与 --load_engine 至少指定一个
--load_engine	可选，指定加载 Engine 的位置，默认为 None，表示不加载 Engine。与 --onnx 至少指定一个
--shapes	可选，指定动态形状模型推理时的输入数据形状
--min_shape, --opt_shape, --max_shape	可选，指定动态形状输入的范围，如果 onnx 为动态形状，需要将三个形状均设置好，格式为 name:shape,name1:shape1
--input_types	可选，指定模型输入数据类型，示例为 input0:float16,input1:int32，如果未指定，则使用 onnx 中默认输入的数据类型

参数	说明
--output_types	可选，指定模型输出数据类型，示例为 output0:float32, output1:int32，如果未指定，则使用 onnx 中默认输出的数据类型
--iterations	可选，指定模型运行次数，默认为 10
--warmUp	可选，指定模型 warmup 次数，默认为 10
--precision	可选，指定模型运行时的精度，可指定为 int8, fp16, fp32，默认为 fp16
--save_engine	可选，指定保存 Engine 文件的位置，默认为 None，表示不保存 Engine 文件
--dump_graph	可选，以 onnx 形式保存计算图的文件位置，默认为 None
--run_profiler	可选，指定进行性能分析，结果会显示在窗口中，默认不进行性能分析
--export_profiler	可选，保存性能分析结果为指定的 csv 文件，示例为 result.csv
--log_level	可选，指定 ixRT 运行时显示日志的等级，可指定为： verbose info warning error internal_error 默认为 warning
--load_inputs	可选，从自定义的输入文件加载输入数据，格式为 input_name1:file1,input_name2:file2
--verify_acc	可选，调用精度对比工具，逐层比较 ixRT 和 ONNXRuntime 的计算结果，并调试精度
--cosine_sim	可选，在精度对比中使用，余弦相似度阈值，默认为 0.999。余弦相似度阈值越接近 1，表明 ixRT 计算结果和比较结果越接近，例如 --cosine_sim 0.99999
--diff_max	可选，在精度对比中使用，Tensor 中的最大插值的绝对值，默认不参与对比。此阈值越大，表明 ixRT 计算结果和比较结果的差异越大，例如 --diff_max 5.0
--diff_sum	可选，在精度对比中使用，Tensor 差值和的绝对值，默认不参与对比。此域值越大，表明 ixRT 计算结果和比较结果的差异越大，例如 --diff_sum 5.0
--diff_rel_avg	可选，在精度对比中使用，Tensor 相对误差的平均值，默认参与对比，默认值为 0.03。此阈值越大，表明 ixRT 计算结果和比较结果的差异越大，例如 --diff_rel_avg 0.1

参数	说明
<code>--ort_onnx</code>	可选，在精度对比中使用，指定 ONNXRuntime 加载原版的 ONNX，例如 <code>--ort_onnx origin_model.onnx</code>
<code>--ort_cpu</code>	可选，在精度对比中使用，指定 ONNXRuntime 以 CPU 运行，不指定默认以 GPU 运行
<code>--plugins</code>	可选，指定提前加载的插件库列表，例如 <code>--plugins liba.so libb.so</code>
<code>--hooks</code>	可选，传入在执行时运行的 Hooks，可选项有 <code>print_info</code>
<code>--inject_tensors</code>	可选，在精度对比中使用，指定某条输入边从外界加载至 ixRT，例如，使用 <code>--inject_tensors 170</code> 指定 170 从 ONNXRuntime 的运行结果输入， <code>--inject_tensors 170:170.npy</code> 指定从本地文件加载；多条边使用英文逗号隔开
<code>--save_verify_data</code>	可选，指定一个文件夹以保存精度验证时每一层的输出结果
<code>--only_verify_outputs</code>	可选，如果在精度验证时指定，仅验证网络的输出
<code>--timingCacheFile</code>	可选，指定保存算法选择时间统计结果的文件路径
<code>- -builderOptimizationLevel</code>	可选，设置构建 Engine 时的优化等级，默认为 3

6.2.2 使用示例

6.2.2.1 示例 1：INT8 模式运行 ResNet50 模型

```
$ ixrtexec --onnx ./resnet50.onnx --precision int8
```

6.2.2.2 示例 2：FP16 模式运行 ResNet50 模型

```
$ ixrtexec --onnx ./resnet50.onnx
# 或
$ ixrtexec --onnx ./resnet50.onnx --precision fp16
```

6.2.2.3 示例 3：运行精度对比

```
$ ixrtexec --onnx ./resnet50.onnx --precision fp16 --verify_acc
```

显示每一层的输出比对结果和整体网络的比对结果。

6.2.2.4 示例 4：进行性能分析并保存结果文件

```
$ ixrtexec --onnx ./resnet18.onnx --precision fp16 --run_profiler --export_profiler
  ↪ result.csv
```

运行完上述命令后，将在界面显示每层的性能分析数据，示例如下图所示：

=== Profile Layer (10 iterations) ===				
Layer Name	Time(ms)	Avg. Time(ms)	Median Time(ms)	Time(%)
n::type_images_Conv_0	0.213	0.021	0.021	0.692
n::reformat_e::type_images_Conv_0_Conv_0	0.170	0.017	0.017	0.553
Conv_0	0.749	0.075	0.076	2.432
MaxPool_2	0.426	0.043	0.043	1.383
Conv_3	0.529	0.053	0.053	1.717
Conv_5	0.570	0.057	0.056	1.851
Conv_8	0.514	0.051	0.052	1.667
Conv_10	0.554	0.055	0.056	1.798
Conv_16	0.210	0.021	0.021	0.683
Conv_13	0.411	0.041	0.041	1.335
Conv_15	0.803	0.080	0.080	2.607
Conv_19	0.621	0.062	0.061	2.016
Conv_21	0.783	0.078	0.078	2.544
Conv_27	6.555	0.656	0.662	21.283
Conv_24	0.919	0.092	0.092	2.982
Conv_26	1.624	0.162	0.162	5.273
Conv_30	1.416	0.142	0.142	4.597
Conv_32	1.623	0.162	0.162	5.269
Conv_35	1.397	0.140	0.140	4.534
Conv_38	0.334	0.033	0.033	1.083
Conv_37	3.682	0.368	0.368	11.953
Conv_41	2.394	0.239	0.239	7.771
Conv_43	3.673	0.367	0.367	11.925
GlobalAveragePool_46	0.291	0.029	0.029	0.944
n::reformat_onnx::Flatten_189_Flatten_47	0.000	0.000	0.000	0.000
Flatten_47	0.000	0.000	0.000	0.000
Gemm_48	0.231	0.023	0.023	0.749
n::type_output_output	0.111	0.011	0.011	0.359

图 2: 每层性能分析数据

显示每层 Layer 和 Kernel 的关系以及性能分析数据如下图所示：

=== Profile Kernel (10 iterations) ===

Layer Name	Kernel Name	Time(ms)	Avg. Time(ms)	Median Time(ms)	Time(%)
n::type_images_Conv_0	Fp32ToFp16KernelI304	0.213	0.021	0.021	0.708
n::reformat_e::type_images_Conv_0	BatchTransposeKernelHalf_Small_H<16u, 512u, 16u, 64u>	0.170	0.017	0.017	0.566
Conv_0	filterPadAlign32<__half>	0.140	0.014	0.014	0.465
Conv_0	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 64u, 32u, 32u,	0.555	0.055	0.055	1.844
MaxPool_2	MaxPoolingForwardKernelNhwcfp16x2<float>	0.426	0.043	0.043	1.416
Conv_3	inputZeroPadding<__half>	0.170	0.017	0.017	0.565
Conv_3	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 64u, 32u, 32u,	0.305	0.030	0.031	1.012
Conv_5	inputZeroPadding<__half>	0.178	0.018	0.017	0.593
Conv_5	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 64u, 32u, 32u,	0.333	0.033	0.033	1.108
Conv_8	inputZeroPadding<__half>	0.156	0.016	0.015	0.519
Conv_8	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 64u, 32u, 32u,	0.303	0.030	0.031	1.008
Conv_10	inputZeroPadding<__half>	0.170	0.017	0.017	0.566
Conv_10	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 64u, 32u, 32u,	0.329	0.033	0.033	1.094
Conv_16	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 128u, 32u, 64u,	0.210	0.021	0.021	0.699
Conv_13	implConvolution2DTcuKernelHalfCommon<128, 128, 64, 32, 32, true, false	0.411	0.041	0.041	1.367
Conv_15	inputZeroPadding<__half>	0.176	0.018	0.017	0.585
Conv_15	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 128u, 32u, 64u,	0.573	0.057	0.057	1.904
Conv_19	implConvolution2DTcuKernelHalfCommon<128, 128, 64, 32, 32, true, false	0.621	0.062	0.061	2.064
Conv_21	inputZeroPadding<__half>	0.154	0.015	0.015	0.512
Conv_21	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 128u, 32u, 64u,	0.573	0.057	0.057	1.904
Conv_27	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 256u, 32u, 64u,	6.555	0.656	0.662	21.788
Conv_24	ImgToCol_FP16<128u>	0.190	0.019	0.019	0.633
Conv_24	MatrixMulTensorCore<256u, 256u, 32u, 32u, 32u, true, false, false, 1,	0.674	0.067	0.067	2.239
Conv_26	inputZeroPadding<__half>	0.150	0.015	0.015	0.500
Conv_26	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 256u, 32u, 64u,	1.419	0.142	0.142	4.715
Conv_30	ImgToCol_FP16<256u>	0.192	0.019	0.019	0.640
Conv_30	MatrixMulTensorCore<256u, 256u, 32u, 32u, 32u, true, false, false, 1,	1.169	0.117	0.118	3.886
Conv_32	inputZeroPadding<__half>	0.150	0.015	0.015	0.500
Conv_32	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 256u, 32u, 64u,	1.418	0.142	0.142	4.713
Conv_35	ImgToCol_FP16<256u>	0.171	0.017	0.017	0.568
Conv_35	MatrixMulTensorCore<256u, 256u, 32u, 32u, 32u, true, false, false, 1,	1.171	0.117	0.118	3.893
Conv_38	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 256u, 32u, 64u,	0.334	0.033	0.033	1.109
Conv_37	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 256u, 32u, 64u,	3.682	0.368	0.368	12.237
Conv_41	ImgToCol_FP16<512u>	0.175	0.017	0.017	0.580
Conv_41	MatrixMulTensorCore<256u, 256u, 32u, 32u, 32u, true, false, false, 1,	2.164	0.216	0.216	7.192
Conv_43	implConvolution2DTcuKernelSmallBatchFP16Template<256u, 256u, 32u, 64u,	3.673	0.367	0.367	12.208
GlobalAveragePool_46	AveragePoolingForwardKernelNhwcfp16x2<float>	0.291	0.029	0.029	0.966
Gemm_48	mr_fast_gemm_f_f_h_h_tcu<32u, 64u, 32u, 16u, 32u, true, false,	0.231	0.023	0.023	0.767
n::type_output_output	CudaFp16ToFp32Kernel	0.111	0.011	0.011	0.368

Profile file saved : result.csv

图 3: 每层 Layer 和 Kernel 的关系以及性能分析数据

同时，程序会生成一个 csv 文件，例如，result.csv。

6.2.2.5 示例 5: 运行含插件的模型

加载 ixRT 内置的插件库，只需要指定 ixrt_plugin 即可。自定义插件需要指定 so 文件的路径，例如：

```
$ ixrtexec --onnx ./yolov3_with_plugin.onnx --precision fp16 --plugins ixrt_plugin
```

6.3 ixRT Model Optimizer 图融合工具使用指南

ixRT Model Optimizer 是 ixRT 提供的一个图融合工具，用于将 onnx 图中的算子融合成对应的 ixRT 插件，一般与 ixRT 配合使用。

该工具在 ixRT 安装包的 tools/optimizer 目录中，您可以通过下载并解压 ixRT 的 tar.gz 安装包获得此工具，详情请参考“ixRT 推理引擎安装指南”。

6.3.1 功能说明

ixRT Model Optimizer 提供以下功能：

- 动态图支持：支持融合动态图和静态图
- 模型支持：目前测试通过的模型系列有：

- bert
- swint
- roformer
- t5
- yolo
- gpt2
- vit
- conformer
- PVT
- omdet
- rnn

6.3.2 参数说明

ixRT Model Optimizer 可使用的参数如下表所列。

参数	说明
--onnx	必选，指定待运行的 onnx 模型的路径
--num_heads	可选，指定模型对应 Attention 模块注意力头的个数
--hidden_size	可选，模型隐藏层的大小
--input_shapes	可选，固定动态模型的输入形状，以从静态形状推理，示例为 --input_shapes "input_name1:3x224x224, input_name2:3x224x224" 类型
--dump_onnx	可选，用于图融合过程中 dump 出中间的 onnx 图，生成 _sim 结尾的 onnx 模型
--model_type	可选，可以指定待融合模型类型，默认为"bert"，可选的有：["bert", "swint", "roformer", "t5", "yolo", "gpt2", "vit", "conformer", "PVT", "omdet", "rnn"]
--log_level	可选，指定 ixRT 运行时显示日志的等级，可指定为 debug, info, error，默认为 info
--not_sim	可选，是否 simplify 模型

6.3.3 运行使用示例

6.3.3.1 示例 1：融合 albert/videoBERT/roberta/deberta

```
$ cd tools/optimizer
$ python3 optimizer.py --onnx ${MODEL_PATH}
```

6.3.3.2 示例 2: 融合 swinL

```
$ cd tools/optimizer
$ python3 optimizer.py --onnx ${MODEL_PATH} --input_shapes pixel_values.1:${BS}x3x384x384
↪ --model_type swint
```

6.3.3.3 示例 3: 融合 roformer

```
$ cd tools/optimizer
$ python3 optimizer.py --onnx ${MODEL_PATH} --model_type roformer
```

6.3.3.4 示例 4: 融合 yolov5s

```
$ cd tools/optimizer
$ python3 optimizer.py --onnx ${MODEL_PATH} --model_type yolo
```

6.3.4 精度验证示例

6.3.4.1 示例 1: albert 模型

模型变量示例:

```
MODEL_PATH="data/albert/albert-base-squad.onnx"
MODEL_END_PATH="data/albert/albert-base-squad_end.onnx"
MODEL_ENGINE_PATH="data/albert/albert-base-squad_end.engine"
```

运行命令:

```
$ cd tools/optimizer
$ python3 optimizer.py --onnx ${MODEL_PATH} --dump_onnx
$ ixrtexec --onnx ${MODEL_END_PATH} --min_shape input_ids.1:${BS}x384,attention_mask.
↪ 1:${BS}x384,token_type_ids.1:${BS}x384 \
    --opt_shape input_ids.1:${BS}x384,attention_mask.
↪ 1:${BS}x384,token_type_ids.1:${BS}x384 \
    --max_shape input_ids.1:${BS}x384,attention_mask.
↪ 1:${BS}x384,token_type_ids.1:${BS}x384 \
    --save_engine ${MODEL_ENGINE_PATH} --log_level verbose
↪ --plugins ixrt_plugin
$ ixrtexec --load_engine ${MODEL_ENGINE_PATH} --ort_onnx ${MODEL_PATH} --plugins
↪ ixrt_plugin --verify_acc
```

6.3.4.2 示例 2: swinL 模型

模型变量示例：

```
BS=1
MODEL_PATH="data/swint/swin-transformer-large.onnx"
MODEL_END_PATH = "data/swint/swin-transformer-large_end.onnx"
MODEL_ENGINE_PATH = "data/swint/swin-transformer-large_end.engine"
MODEL_SIM_STATIC_SIM_PATH = "data/swint/swin-transformer-large_sim_static_sim.onnx"
```

运行命令：

```
$ cd tools/optimizer

# 固定输入形状为 ${BS}x3x384x384
$ python3 optimizer.py --onnx ${MODEL_PATH} --input_shapes pixel_values.1:${BS}x3x384x384
  ↳ --model_type swint --dump_onnx

# Build engine
$ ixrtexec --onnx ${MODEL_END_PATH} --save_engine ${MODEL_ENGINE_PATH} --log_level
  ↳ verbose --plugins ixrt_plugin

# 测试性能
$ ixrtexec --load_engine ${MODEL_ENGINE_PATH} --plugins ixrt_plugin

# 测试精度
$ ixrtexec --load_engine ${MODEL_ENGINE_PATH} --ort_onnx ${MODEL_SIM_STATIC_SIM_PATH}
  ↳ --plugins ixrt_plugin --verify_acc
```

6.4 ixRT Deploy 量化工具使用指南

static_quantize 用于静态量化 ONNX 模型或 PyTorch 模型，如果是 PyTorch 模型，会首先被转为 ONNX 模型，然后再开始进行量化。

6.4.1 参数说明

参数	说明
model	ONNX 文件路径或 PyTorch nn.Module
calibration_data_loader	校准数据集 DataLoader

参数	说明
observer	定点算法，可选项有： minmax hist_percentile percentile entropy ema
disable_bias_correction	禁用偏置修正，如果为 True，则不会修改偏置，否则修改部分偏置
analyze	启用量化分析
save_quant_onnx_path	保存量化后的 onnx 至本地路径
save_quant_params_path	保存量化参数至本地路径
data_preprocess	数据预处理方法
quant_format	量化输出模型格式，支持“qdq”和“ppq”
device	运行设备号
disable_quant_names	禁止量化的算子名称
disable_quant_types	禁止量化的算子类型
operator_config_by_type	通过算子类型设置对算子的量化方法
operator_config_by_name	通过算子名称设置对算子的量化方法
passes	量化前对 Graph 进行处理的 Pass
verification	量化模型精度校验类对象，用于校验量化后模型精度，必须实现 init, get_data, calculate, summary 四个成员函数

6.4.2 使用示例

量化一个 ResNet18 模型，并保存至 quantized_resnet18.onnx：

```
import torch
import torchvision.datasets
from torch.utils.data import DataLoader
from torchvision import models
from torchvision import transforms as T
from ixrt.deploy import static_quantize

class CalibrationImageNet(torchvision.datasets.ImageFolder):
    def __init__(self, *args, **kwargs):
        super(CalibrationImageNet, self).__init__(*args, **kwargs)
```

```

img2label_path = os.path.join(self.root, "val_map.txt")
if not os.path.exists(img2label_path):
    raise FileNotFoundError(f"Not found label file `{img2label_path}`.")

self.img2label_map = self.make_img2label_map(img2label_path)

def make_img2label_map(self, path):
    with open(path) as f:
        lines = f.readlines()

    img2label_map = dict()
    for line in lines:
        line = line.lstrip().rstrip().split("\t")
        if len(line) != 2:
            continue
        img_name, label = line
        img_name = img_name.strip()
        if img_name in [None, ""]:
            continue
        label = int(label.strip())
        img2label_map[img_name] = label
    return img2label_map

def __getitem__(self, index):
    path, target = self.samples[index]
    sample = self.loader(path)
    if self.transform is not None:
        sample = self.transform(sample)
    img_name = os.path.basename(path)
    target = self.img2label_map[img_name]

    return sample, target

data_path = "path/to/imagenet"
dataset = CalibrationImageNet(
    data_path,
    transform=T.Compose(
        [
            T.Resize(256),
            T.CenterCrop(224),
            T.ToTensor(),
            T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
        ]
    ),
)

calibration_dataloader = DataLoader(
    dataset,
    shuffle=False,
    batch_size=32,

```

```
drop_last=False,  
num_workers=0,  
)  
  
static_quantize("resnet18.onnx",  
                calibration_dataloader=calibration_dataloader, # torch.utils.data.DataLoader  
                save_quant_onnx_path="quantized_resnet18.onnx",  
                observer="minmax",  
                data_preprocess=lambda x: x[0].to("cuda"),  
                quant_format="qdq")
```

7 ixRT 推理引擎使用调试指南

为了方便地使用 ixRT 推理引擎，天数智芯提供以下 ixRT 调试工具：

- 精度对比工具
- 性能调试工具
- ixRT Hook 接口

7.1 精度对比工具

在极少数情况下，您使用 ixRT 的推理结果可能与预期不符，导致不符预期的原因有多种，例如，计算图逻辑错误、插件实现错误，或 ixRT 内部算子计算错误等。当这种情况出现时，建议您使用 ixRT 提供的精度对比工具快速找出首先计算错误的节点。

精度对比工具的大致原理是：分别计算 ixRT 和第三方框架的逐层输出结果，并对输出结果进行数值对比。

ixRT 使用 ONNXRuntime 作为基础比较框架。

7.1.1 准备工作

ixRT 精度对比工具集成在 ixrtexec 工具中，在使用前，您需要先安装以下依赖包：

```
$ pip3 install onnx onnxruntime scipy
```

7.1.2 使用方法

您只需要在使用 ixrtexec 时增加 **--verify_acc** 即可调用精度对比工具，命令示例如下：

```
$ ixrtexec --onnx model.onnx --verify_acc
```

精度对比工具将尽可能对所有的 Tensor 进行分析，对比和标准 ONNXRuntime 计算结果的差别。下面是一个输出示例：

```
+-----+-----+-----+
| RIGHT | Gemm_48_elementwise | ---- Relative Diff Max ---- |
|        |                       | 0.001920149295387315         |
|        |                       | ---- Absolute Diff Max ----  |
|        |                       | 0.003383636474609375         |
|        | output               | ---- Absolute Diff Sum ----  |
|        | (1, 1000)            | 0.6609795987606049           |
|        |                       | ---- Cosine Similarity ----  |
|        |                       | 0.9999998662775001           |
+-----+-----+-----+
```

其中, output 张量由 Gemm_48_elementwise 算子产生, 经过和 ONNXRuntime 对比后, 精度基本一致。因此结果为 RIGHT, 反之为 WRONG。第三列可以了解到 ixRT 结果和 ONNXRuntime 结果的比较统计值。

每一项对比指标都有默认值, ixrtexec 还提供命令行接口, 供您调整其中某一项的默认值。详情如下:

- `--cosine_sim val`: 基于 [余弦相似度](#), 您可以通过 `--cosine_sim` 指定余弦相似度阈值, 默认为 0.999, 余弦相似度阈值越接近 1, 表明 ixRT 计算结果和比较结果越接近
- `--diff_max val`: Tensor 中的最大差值的绝对值, 默认不参与对比, 您可以通过 `--diff_max` 开启, 此域值越大, 表明 ixRT 计算结果和比较结果的差异越大
- `--diff_sum val`: Tensor 差值和的绝对值, 默认不参与对比, 您可以通过 `--diff_sum` 开启, 此域值越大, 表明 ixRT 计算结果和比较结果的差异越大

7.1.3 使用示例

7.1.3.1 示例 1: 对比 FP16 模型

ixRT 使用 FP16 计算, ONNXRuntime 使用 FP32 计算, 对比精度命令如下:

```
$ ixrtexec --onnx model.onnx --verify_acc --precision float16
```

7.1.3.2 示例 2: 对比 INT8 QDQ 模型

ixRT 使用 INT8 计算, ONNXRuntime 使用 FP32 计算, 对比精度命令如下:

```
$ ixrtexec --onnx model.onnx --verify_acc --precision int8
```

7.1.3.3 示例 3: ixRT 使用的 ONNX 包含自定义插件

如果使用 ixRT 推理的 ONNX 包含自定义插件, 那么 ONNXRuntime 不能直接运行。您可以通过 `--ort_onnx` 将该 ONNX 的初始版本传给 ONNXRuntime, 通过 `--onnx` 将带有插件的 ONNX 传给 ixRT。参考命令如下:

```
$ ixrtexec --onnx model_customized.onnx --ort_onnx model.onnx --verify_acc
```

说明: 上述命令中, 假设 model.onnx 是 model_customized.onnx 的初始版本, 能被 ONNXRuntime 直接运行。

7.1.3.4 示例 4: 比较引擎文件和原 ONNX

如果您预先编译好了引擎文件, 可以使用 `--load_engine` 直接加载, 从而节省对比时间; 通过 `--ort_onnx` 将该模型的初始版本传给 ONNXRuntime。参考命令如下:

```
$ ixrtexec --load_engine model.engine --ort_onnx model.onnx --verify_acc
```

7.1.3.5 示例 5：保存对比数据

分别保存 ixRT 和 ONNXRuntime 产生的中间结果，用于进一步分析。参考命令如下：

```
$ ixrtexec --onnx model_customized.onnx --ort_onnx model.onnx --verify_acc
  ↳ --save_debug_data /path/to/save
```

说明：中间张量保存格式为 npy，您可以直接拖到 [netron](#) 中预览。

7.1.3.6 示例 6：运行时输入注入

如果排查到网络中有某个算子计算错误，并希望排查后续算子的错误情况，可以通过 **--inject_tensors** 将正确结果传给 ixRT，进行后续计算。

注意：传入的数据是 Linear 格式，并且没有任何其它的填充信息。

```
# 假设边 170 计算错误

# 从 ONNXRuntime 传入正确的边进行计算，不需指定路径
$ ixrtexec --onnx model.onnx --verify_acc --inject_tensors 170

# 从自定义路径传入此条边的 npy 数据，需要指定路径
$ ixrtexec --onnx model.onnx --verify_acc --inject_tensors 170:170.npy
```

7.2 性能调试工具

您可以直接使用 ixrtexec 工具分析一个模型的性能，例如：

```
$ ixrtexec --onnx model.onnx --precision fp16 --run_profiler --export_profiler result.csv
```

示例输出如下：

```
Layer Name  Kernel Name
↳ Time(ms)   Avg. Time(ms)   Median Time(ms)   Time(%)
-----
↳ -----
↳ -----
input_apply_reformat_0  nvinfer1::convert_format_nchw_nhwc_dtype<float, __half, false,
↳ false>           0.226           0.023           0.022           2.476
Conv_0_activation  filterPadKAlign32<__half>
↳ 0.118           0.012           0.012           1.294
Conv_0_activation  implConv2DFp16Algo1W16NoSmeKernelTemplate<256, 64, 32, 32, 32, false,
↳ 0.290           0.029           0.029           3.186
MaxPool_2  MaxPoolingForwardKernelNhwc<float, __half, float>
↳ 0.220           0.022           0.022           2.416
Conv_3_activation  implConv2DAlgo1W16SmeHReuseFP16KernelTemplate<16, 64, 32, 2, 32, 3, 3,
↳ 0.193           0.019           0.019           2.114
```

上述示例输出列解释如下表所列。

列名	描述
Layer Name	神经网络层的名称 (来自模型或 ixRT 自动生成的标识符)
Kernel Name	实际执行的底层计算内核 (GPU 核函数) 名称, 通常由框架或硬件优化实现
Time(ms)	该层的总执行时间 (毫秒)
Avg. Time(ms)	该层的平均执行时间 (多次运行的平均值, 毫秒)
Median Time(ms)	该层的中位数执行时间 (毫秒)
Time(%)	该层耗时占总推理时间的百分比

7.3 ixRT Hook 接口

ixRT 允许您注册 Hook 函数, 用于在运行时调用这些函数。通常情况下, 您不会用到该功能, 但也许您希望在推理过程中, 检查或调试网络中间层的计算结果, 或自定义特定层的输入, 那么 Hook 机制是您的好帮手。

7.3.1 C++ API

当实现一个 Hook 函数时, 这个函数只有一个参数 `nvinfer1::ExecutionContextInfo const* info`, 用于表示当前运行的网络层相关上下文信息。

您可以在 ixRT 安装包的 `{path_to_ixrt}/samples/sampleResNet/classification.cc` 文件的 `TensorRTAPIExecuteWithHook` 函数中学习使用 Hook 函数。

下面是一个简单的 Hook 函数, 用于将每一层的信息进行打印:

```
void MyHook(nvinfer1::ExecutionContextInfo const* info) {
    cudaDeviceSynchronize();
    std::cout << "Running callback function " << info->hookName << std::endl;
    std::cout << "Running op " << info->opName << std::endl;
    std::cout << "NbInputs " << info->nbInputs << std::endl;
    std::cout << "NbOutputs " << info->nbOutputs << std::endl;
}
```

定义 Hook 函数, 请注意第一行需要首先进行同步, 以确保获取到正确的内存数据。

定义好 Hook 函数后, 使用 `IEExecutionContext::registerHook(AsciiChar const* name, ExecutionHook hook, int32_t flag)` 注册这个函数:

```
context->registerHook("print_basic_info", MyHook,
↪ int32_t(nvinfer1::ExecutionHookFlag::kPRERUN));
```

参数说明如下：

- 第一个参数用于表示这个 Hook 函数的名字，每个 Hook 函数使用一个独立的字符串表示
- 第二个参数是刚才定义的 Hook 函数
- 第三个参数用于指示 Hook 函数的运行时刻，ExecutionHookFlag::kPRERUN 表示在层运行之前运行，ExecutionHookFlag::kPOSTRUN 表示在层之后运行

您可以参考 nvinfer1::ExecutionContextInfo 的定义，对 ixRT 暴露出的中间信息进行您的自定义操作，代码示例如下：

```
struct ExecutionContextInfo {
    /// The name of the op being executed
    AsciiChar const* opName;
    /// The type of the op being executed
    LayerType type;
    /// The op type defined in onnx fashion, optional
    AsciiChar const* op_type;
    /// The number of inputs of the layer
    size_t nbInputs;
    /// The number of outputs of the layer
    size_t nbOutputs;
    /// The input names of the executed op
    AsciiChar const* const* inputNames;
    /// The output names of the executed op
    AsciiChar const* const* outputNames;
    /// The input tensors of the executed op
    ExecutionContextTensorDesc const* inputTensors;
    /// The output tensors of the executed op
    ExecutionContextTensorDesc const* outputTensors;
    /// The hook being executed
    AsciiChar const* hookName;
};
```

其中，ExecutionContextTensorDesc 为 ixRT 的内部 Tensor 表示，定义为：

```
struct ExecutionContextTensorDesc {
    /// device ptr of the tensor
    void const* data;
    /// internal paddings, for better performance if used
    Dims paddings;
    /// dimension after internal paddings
    Dims dims;
    /// data type of the tensor
    DataType type;
    /// data format of the tensor
    TensorFormat format;
```

```
int32_t itemsize;
/// for int8, the scale factor
float const* scale;
/// number of scales, should be a non-negative number
int32_t nb_scales;
/// If the tensor is an initializer
bool is_initializer;
/// If the tensor is a shape tensor
bool is_shape_tensor;
};
```

其中, data 是运行时 GPU 内存地址, 您可以选择在运行时访问这一块内存, 并修改 (仅限于) 输入内存。**注意: 运行时修改输入内存内容仅限于调试阶段使用, 请谨慎使用该选项, 切勿在生产环境中使用。**如果您希望修改输入内存, 需要指定 Hook 函数标志包含 `int32_t(nvinfer1::ExecutionHookFlag::kPRERUN)`。

7.3.2 Python API

ixRT 也支持注册来自 Python 的回调函数, 从而可以方便利用 Python 丰富的第三方生态, 例如, numpy。

您可以在安装包的 `{path_to_ixrt}/samples/python/resnet18/sample_hook.py` 文件中学习使用 Hook 函数。

和 C++ 类似, 您需要首先定义 Hook 函数:

```
import tensorrt
import cuda.cudart as cudart
import cuda.cudart as cudart

def my_callback(info: tensorrt.ExecutionContextInfo):
    cudart.cudaDeviceSynchronize()
    print("Layer Type: ", info.layer_type)
    print("OpType: ", info.op_type)
    print("Op: ", info.op_name)
```

定义 Hook 函数, 请注意第一行需要首先进行同步, 以确保获取到正确的内存数据。

接着注册回调函数函数:

```
context.register_hook("simple_print", my_callback, tensorrt.ExecutionHookFlag.POSTRUN)
```

上面一行代码中, 将 `my_callback` 取名为 "simple_print" 注册, 并指定在每层运行完之后运行 Hook 函数。借助 Python 的动态语言特性, 您可以使用许多方便的特性, 例如, 在运行时打断点用于检查网络中间层结果:

```
def my_callback(info: tensorrt.ExecutionContextInfo):
    if info.op_name == "Conv_0":
        import pdb; pdb.set_trace()
```

Python 的 Hook 信息 ExecutionContextInfo 定义如下：

```
class ExecutionContextInfo:
    op_name: str
    type: LayerType
    op_type: str
    nb_inputs: int
    nb_outputs: int
    input_names: List[str]
    output_names: List[str]
    input_tensors: List[ExecutionContextTensorDesc]
    output_tensors: List[ExecutionContextTensorDesc]
    hook_name: str
```

其中，ExecutionContextTensorDesc 为 ixRT 的内部 Tensor 表示，定义为：

```
class ExecutionContextTensorDesc:
    data: int
    paddings: List[int]
    dims: List[int]
    type: Datatype
    format: TensorFormat
    itemsize: int
    scale: List[int]
    is_initializer: bool
    is_shape_tensor: bool
```

由于传递给 Hook 函数的是一个处于 GPU 的 ixRT Tensor，因此 ixRT 提供了一些方便的函数用于将数据拷贝 numpy array：

```
from tensorrt.hook.utils import copy_ixrt_tensor_as_np, copy_ixrt_io_tensors_as_np
def my_view_tensor_callback(info):
    # 直接得到一个 ixRT 内部表示的 tensor
    np_array = copy_ixrt_tensor_as_np(info.input_tensors[0], ort_style=False)
    # 转换成一个 kLINEAR、FP32 的标准 tensor
    np_array = copy_ixrt_tensor_as_np(info.input_tensors[0], ort_style=True)
    # 把所有的输入、输出都拷贝到 cpu，且使用 ixRT 内部表示格式
    np_array = copy_ixrt_io_tensors_as_np(info, ort_style=False)
    # 把所有的输入、输出都拷贝到 cpu，且使用 kLINEAR、FP32 的标准
    np_array = copy_ixrt_io_tensors_as_np(info, ort_style=True)
```

说明：

- ort_style 选取为 False 时，您可以更直接访问 ixRT 计算的中间结果
- ort_style 选取为 True 时，您可以更方便和第三方框架 (例如，ONNXRuntime, PyTorch) 进行比较

除此之外，ixRT 还内置了一些 Hook 函数供您使用，例如：

```
from tensorrt.hook import create_hook

print_info_hook = create_hook("print_info")
inject_input_hook = create_hook("inject_external_input", external_input="path/to/
    ↪ input.npy")

context.register_hook(...)
```

8 附录

本章节内容包括：

- ixRT 内置示例列表
- ixRT 内置插件列表

8.1 ixRT 内置插件列表

以下是 ixRT 提供的内置插件。

插件	名称	版本
AffineGrid	affinegrid	1
CustomFCPluginDynamic_IxRT	custom_fc	1, 2
RoiAlign	roialign	1
TransformerEncoderFp16	transformer_encoder	1
BertLnResidual	bert_ln_residual	1
CustomGeluPluginDynamic_IxRT	gelu	1
CustomRoPEPluginDynamic_IxRT	rope	1
CMVN	cmvn	1
EcapaAspAttn	ecapa_asp_attn	1
CustomSkipLayerNormPluginDynamic_IxRT	skip_layernorm	1, 3
ViterbiDecode	viterbi_decode	1
Conformer_Conv_Module	conformer_conv_module	1
EcapaPool	ecapa_pool	1
Random Normal Like	random_normal_like	1
SplitQKV	split_qkv	1
Conformer_Encoder_Beam_Search	conformer_encoder_beam_search	
EcapaScorePool	ecapa_score_pool	1
Inverse Sigmoid	inverse_sigmoid	1
RebuildPadding_IxRT	rebuild_padding	1
XSoftmax	xsoftmax	1
Conformer_Self_Attention	conformer_selfattention	1

插件	名称	版本
Efficient NMS Plugin	efficientNMSPlugin	1
L2_Normalization	l2_normalization	1
RemovePadding_IxRT	remove_padding	1
YolactDecoder	yolact_decoder	1
Conv2dSubsampling4	conv2d_subsampling4	1
CustomEmbLayerNormPluginDynamic_IxRT	emb_layernorm	1, 2
Reorg	reorg	1
YoloV3Decode	yolov3_decoder	1
CTCGreedySearch	ctc_greedy_search	1
FacenetNorm	facenet_norm	1
multiscaleDeformableAttn	multiscaledeformableattn	1
RetinaNet Decoder	retinanet_decoder	1
TemporalShift	temporal_shift	1
YoloV5Decoder	yolov5_decoder	1
CustomFFNPluginDynamic_IxRT	ffn	1
DetectionNMS_IxRT	NMS_IxRT	1
RMS_Norm	rms_norm	1
YoloV7Decoder	yolov7_decoder	1
Custom argmax	custom_argmax	1
Focus	focus	1
PointRendCalUncertaint	pointrend_cal_uncertaint	1
RNN	rnn	1
YoloxDecoder	yolox_decoder	1
fcosNMS	fcosNMSPlugin	1
PositionWiseFFN	positionwise_ffn	1
CustomQKVToContextPluginDynamic_IxRT	qkv_to_context	1, 2, 3

8.1.1 AffineGrid

算子描述

AffineGrid 算子通过给定的仿射变换参数 theta，生成 2D 或 3D 采样网格。

参数信息：属性

类型	属性名
string	name
bool	align_corners
int32	size
int32	size_len

参数信息：支持精度

输入	精度	输出	精度
theta	float16	score	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
theta	kLINEAR	score	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
theta for 2D	(N, 2, 3)	score for 2D	size_len=4 (size[0], size[2], size[3], 2)
theta for 3D	(N, 3, 4)	score for 3D	size_len=5 (size[0], size[2], size[3], size[4], 2)

更多参考

请参考 PyTorch [affine_grid](#) 了解更多关于该算子的信息。

8.1.2 CustomFCPluginDynamic_IxRT

算子描述

CustomFCPluginDynamic_IxRT 算子实现 fully_connected op 带 half_bias。

参数信息：属性

插件版本：

- 1: FP16
- 2: INT8
- 3: FP32

类型	属性名	版本	描述
int32_t	out_dims	1, 2	必选, fc 输出的维度
int32_t	type_id	1, 2	必选, DataType 类型
float weight	W	1, 2	必选, fc 的权重
float weight	B	1, 2	可选, fc 的偏置
float array	fc_amax	2	必选, 带 bias [qkv_wei_scale, qkv_out_scale], 不带 bias [qkv_wei_scale]

参数信息：支持精度

输入	精度	输出	精度
input	[half,int8]	output	[half,int8]

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR

参数信息：支持形状 1

输入	形状	输出	形状
input	[B, S, iE]	output	[B, S, oE]

参数信息：支持形状 2

输入	形状	输出	形状
input	[S, iE]	output	[S, oE]

更多参考

请参考 [BERT \[原文\]](#) 了解更多关于该算子的信息。

8.1.3 RoiAlign

算子描述

RoiAlign 算子实现了特征图的提取处理。该算子有 3 个输入和 1 个输出。

参数信息：属性

类型	属性名
int32	output_height
int32	output_width
int32	sampling_ratio
int32	mode
int32	coordinate_transformation_mode
float32	spatial_scale

参数信息：支持精度组合 1

输入	精度	输出	精度
input	float16	output	float16
rois	float16		
batch_indices	int32		

参数信息：支持精度组合 2

输入	精度	输出	精度
input	float32	output	float32
rois	float32		
batch_indices	int32		

参数信息：支持排布

输入	精度	输出	精度
input	kLINEAR	output	kLINEAR
rois	kLINEAR		
batch_indices	kLINEAR		

参数信息：支持形状

输入	形状	输出	形状
input	(BS, C, H, W)	output	(nb_rois, C, out_H, out_W)
rois	(nb_rois, 4)		
batch_indices	(nb_rois,)		

更多参考

请参考 PyTorch [Mask RCNN \[原文\]](#) 了解更多关于该算子的信息。

8.1.4 TransformerEncoderFp16

8.1.5 BertLnResidual

算子描述

BertLnResidual 算子融合 LayerNorm 和残差结构，融合后有 4 个输入和 4 个输出。

参数信息：属性

类型	属性名
float32	l0_qkv_in_amax
int32	hidden_size
int32	head_num
int32	inner_size

参数信息：支持精度

输入	精度	输出	精度
hidden_states	float16	q_0	int8

COREX01-BI430-UG11-00 127 V4.3.0 (x86)|V4.3.6 (ARM)

输入	精度	输出	精度
ln_weights_first	float16	residual_0	float16
ln_bias_first	float16	ffd1_0	int8
l0_self_attn_out_proj_bias	float16	ffd2_0	int8

参数信息：支持排布

输入	数据排布	输出	数据排布
hidden_states	kLINEAR	q_0	kLINEAR
ln_weights_first	kLINEAR	residual_0	kLINEAR
ln_bias_first	kLINEAR	ffd1_0	kLINEAR
l0_self_attn_out_proj_bias	kLINEAR	ffd2_0	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
hidden_states	(N, L, H)	q_0	(N, L, H)
ln_weights_first		residual_0	(N, L, H)
ln_bias_first		ffd1_0	(L1,)
l0_self_attn_out_proj_bias		ffd2_0	(L2,)

其中，L1 和 L2 的计算方式如下：

```
size1 = 3*N*L*H
size2 = inner_size*N*L
size3 = head_num*N*L*H

L1 = max(size1, size2)
L2 = max(L1, size3)
```

更多参考

请参考 [BERT \[原论文\]](#) 了解更多关于该算子的信息。

8.1.6 CustomGeluPluginDynamic_IxRT

算子描述

CustomGeluPluginDynamic_IxRT 算子融合 gelu 和前置的 bias 结构。

参数信息：属性

插件版本：

- 1: FP16
- 3: INT8

类型	属性名	版本	描述
int32_t	type_id	1, 3	必选, DataType 类型
float Weight	bias	1, 3	half 可选, int8 必选, gelu 的前置 fc 的 bias

参数信息：支持精度

输入	精度	输出	精度
input	[half,float,int8]	output	[half,float,int8]

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	[B, S, E]	output	[B, S, E]

更多参考

请参考 [BERT \[原论文\]](#) 了解更多关于该算子的信息。

8.1.7 CustomRoPEPluginDynamic_IxRT

算子描述

CustomRoPEPluginDynamic_IxRT, 旋转位置编码 (Rotary Position Embedding, ROPE) 是一种用于在 Transformer 模型中编码 token 相对位置信息的方法。

与传统的绝对位置编码相比, ROPE 更灵活, 能够更好地处理不同长度的序列, 并且在模型中保留相对位置信息。

参数信息: 属性

类型	属性名	描述
int32_t	type_id	必选, DataType 类型 (FP16)

参数信息: 支持精度

输入	精度	输出	精度
input	half	output	half
cos	float		
sin	float		

参数信息: 支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
cos	kLINEAR		
sin	kLINEAR		

参数信息: 支持形状

输入	形状	输出	形状
input	[b, s, h, d]	output	[b, s, h, d]
cos	[b, s, h, d]		
sin	[b, s, h, d]		

说明:

- b: batchsize
- s: sequence length
- h: head number

- d: dim of each head

8.1.8 CMVN

算子描述

CMVN 算子实现倒谱均值归一化 (Cepstral Mean Variance Normalization)，用于处理语音识别任务数据。该算子有 3 个输入和 1 个输出。

参数信息：属性

无

参数信息：支持精度

输入	精度	输出	精度
input	float16	output	float16
mean	float16		
variance	float16		

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
mean	kLINEAR		
variance	kLINEAR		

参数信息：支持形状

- class_number: 检测目标种类
- anchor_number: 每个 feature map 点关联的 anchor 数量

输入	形状	输出	形状
cls_map	(N, feat_len, feat_dim)	output	(N, feat_len, feat_dim)
mean	(feat_dim)		
variance	(feat_dim)		

更多参考

请参考 <https://github.com/wenet-e2e/wenet/blob/main/wenet/transformer/cmvn.py> 了解更多关于该算

子的信息。

8.1.9 EcapaAspAttn

算子描述

EcapaAspAttn 算子在 ECAPA-TDNN 模型中使用，是关于 attention 的融合算子。该算子有 2 个输入和 1 个输出。

参数信息：属性

无

参数信息：支持精度

输入	精度	输出	精度
input	float16	output	float16
length	float32		

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kHWC	output	kHWC
length	kHWC		

参数信息：支持形状

输入	形状	输出	形状
input	(N, H, W)	output	(N, 3*H, W)
length	(N,)		

更多参考

请参考参考 https://github.com/speechbrain/speechbrain/blob/develop/speechbrain/lobes/models/ECA_PA_TDNN.py 了解更多关于该算子的信息。

8.1.10 CustomSkipLayerNormPluginDynamic_IxRT

算子描述

CustomSkipLayerNormPluginDynamic_IxRT 算子融合 LayerNorm 和残差结构。

参数信息：属性

插件版本：

- 1: FP16
- 3: INT8

类型	属性名	版本	描述
int32_t	ld	1, 3	必选，隐藏层的大小，即 E
int32_t	type_id	1, 3	必选，DataType 类型
float Weight	beta	1, 3	必选，layernorm 的 beta 参数
float Weight	gamma	1, 3	必选，layernorm 的 gamma 参数
float bias	bias	1, 3	half 可选，int8 必选，layernorm 的前置 fc 的 bias

参数信息：支持精度

FP16

输入	精度	输出	精度
input	half	output	half
skip	half		

INT8

输入	精度	输出	精度
input	int8_t	output	int8_t
skip	int8_t	residual_out	float32
residual	float32		

参数信息：支持排布

FP16

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR

输入	数据排布	输出	数据排布
skip	kLINEAR		

INT8

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
skip	kLINEAR	residual_out	kLINEAR
residual	kLINEAR		

参数信息：支持形状

FP16

输入	形状	输出	形状
input	[B, S, E]	output	[B, S, E]
skip	[B, S, E]		

INT8

输入	形状	输出	形状
input	[B, S, E]	output	[B, S, E]
skip	[B, S, E]	residual_out	[B, S, E]
residual	[B, S, E]		

其中，L1 和 L2 的计算方式如下：

```
size1 = 3*N*L*H
size2 = inner_size*N*L
size3 = head_num*N*L*H

L1 = max(size1, size2)
L2 = max(L1, size3)
```

更多参考

请参考 [BERT \[原文\]](#) 了解更多关于该算子的信息。

8.1.11 ViterbiDecode

算子描述

ViterbiDecode 算子实现了维特比解码算法，常用于 CRF (Condition Random Field)，和 BERT 搭配使用，进行 NER (Name Entity Recognition) 任务。该算子有 5 个输入和 1 个输出。

参数信息：属性

无

参数信息：支持精度组合 1

输入	精度	输出	精度
crf_end_transitions	float32	output	int32
crf_start_transitions	float32		
crf_transitions	float32		
fc_output	float32		
i8_mask	int8		

参数信息：支持精度组合 2

输入	精度	输出	精度
fc_output	half	output	int32
mask	int32		
crf_start_transitions	half		
crf_transitions	half		
crf_end_transitions	half		

参数信息：支持排布

输入	数据排布	输出	数据排布
fc_output	kLINEAR	output	kLINEAR
i8_mask	kLINEAR		
crf_start_transitions	kLINEAR		
crf_transitions	kLINEAR		
crf_end_transitions	kLINEAR		

参数信息：支持形状

输入	形状	输出	形状
i8_mask	(N,L)	output	(N,L)
fc_output	(N, L, H)		
crf_transitions	(7, 7)		
crf_start_transitions	(7,)		
crf_end_transitions	(7,)		

更多参考

请参考 [BERT \[原论文\]](#) 了解更多关于该算子的信息。

8.1.12 Conformer_Conv_Module

算子描述

Conformer_Conv_Module 算子实现了 Transformer 中的卷积模块。

参数信息：属性

类型	属性名
int32	kernel_size_1
int32	kernel_size_2
int32	kernel_size_3
int32	stride_1
int32	stride_2
int32	stride_3
int32	odim_1
int32	odim_2
int32	odim_3

参数信息：支持精度

输入	精度	输出	精度
inputx	float16	output	float16

输入	精度	输出	精度
mask	int32		
ln1.weight	float16		
ln1.bias	float16		
pointwise_conv1.weight	float16		
pointwise_conv1.bias	float32		
depthwise_conv.weight	float16		
depthwise_conv.bias	float32		
ln2.weight	float16		
ln2.bias	float16		
pointwise_conv2.weight	float16		
pointwise_conv2.bias	float32		

参数信息：支持排布

输入	数据排布	输出	数据排布
inputx	kLINEAR	output	kLINEAR
mask	kLINEAR		
ln1.weight	kLINEAR		
ln1.bias	kLINEAR		
pointwise_conv1.weight	kLINEAR		
pointwise_conv1.bias	kLINEAR		
depthwise_conv.weight	kLINEAR		
depthwise_conv.bias	kLINEAR		
ln2.weight	kLINEAR		
ln2.bias	kLINEAR		
pointwise_conv2.weight	kLINEAR		
pointwise_conv2.bias	kLINEAR		

参数信息：支持形状

输入	形状	输出	形状
inputx	(BATCH_SIZE, IN_CHANNELS, IN_FEATURE)	output	(BATCH_SIZE, IN_CHANNELS, IN_FEATURE)
mask	(BATCH_SIZE, IN_CHANNELS, 1)		
ln1.weight	(IN_FEATURE)		
ln1.bias	(IN_FEATURE)		
pointwise_conv1.weight	(HIDDEN_CHANNELS, IN_FEATURE, 1)		
pointwise_conv1.bias	(HIDDEN_CHANNELS)		
depthwise_conv.weight	(IN_FEATURE, 1,)		
depthwise_conv.bias	(IN_FEATURE)		
ln2.weight	(IN_FEATURE)		
ln2.bias	(IN_FEATURE)		
pointwise_conv2.weight	(IN_FEATURE, IN_FEATURE, 1)		
pointwise_conv2.bias	(IN_FEATURE)		

更多参考

请参考 [Conformer](#) 了解更多关于该算子的信息。

8.1.13 EcapaPool

算子描述

EcapaPool 在 ECAPA-TDNN 模型中使用，是融合了 pool 之后的融合算子。该算子有 2 个输入和 1 个输出。

参数信息：属性

无

参数信息：支持精度

输入	精度	输出	精度
input	float16	output	float16
length	float32		

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kHWC	output	kHWC
length	kHWC		

参数信息：支持形状

输入	形状	输出	形状
input	(N, H, C)	output	(N, H, 1)
length	(N,)		

更多参考

请参考 https://github.com/speechbrain/speechbrain/blob/develop/speechbrain/lobes/models/ECAPA_T_DNN.py 了解更多关于该算子的信息。

8.1.14 Random Normal Like

算子描述

Random Normal Like 算子生成一个具有从正态分布中抽取的随机值的张量。输出张量的形状是从输入张量的形状复制的，正态分布的参数由 mean 和 scale 指定。

参数信息：属性

类型	属性名
int32	dtype
float	mean
float	scale
float	seed

参数信息：支持精度

输入	精度	输出	精度
input	float16 or float32	output	float16 or float32

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	(N, C, H, W)	output	(N, C, H, W)

更多参考

请参考 [Random Normal Like](#) 了解更多关于该算子的信息。

8.1.15 SplitQKV

算子描述

SplitQKV 算子用于从输出中分离出 query, key, value。

参数信息：属性

类型	属性名
string	name
float	atten_scale
bool	transpose
int32	head_num
int32	head_dim

参数信息：支持精度

输入	精度	输出	精度
input	float16	query	float16
		key	float16
		value	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	query	kLINEAR
		key	kLINEAR
		value	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	[dim0,dim1,3*dim2]	query	[dim0, dim1, :dim2] * atten_scale
		key	[dim0, dim1, dim2:2*dim2]
		value	[dim0, dim1, 2* dim2: 3*dim2]

8.1.16 Conformer_Encoder_Beam_Search

算子描述

Conformer_Encoder_Beam_Search 算子实现了 INT8 精度的 ConformerEncoder 和 BeamSearch。

参数信息：属性

类型	属性名
int32	model_defintion_params
float	quant_params_table

参数信息：支持精度

Conformer_Encoder_Beam_Search Plugin 共有 787 个输入和 1 个输出，其中：

- input[0] 的数据数据类型为 INT32，其余输入的数据类型均为 float16；
- output 的数据类型为 INT32。

参数信息：支持排布

Conformer_Encoder_Beam_Search Plugin 算子的全部输入和输出均为 kLINEAR 的数据排布方式。

更多参考

请参考 [Conformer](#) 了解更多关于该算子的信息。

8.1.17 EcapaScorePool

算子描述

EcapaScorePool 算子在 ECAPA-TDNN 模型中使用，是融合了 pool 之后的融合算子。该算子有 3 个输入和 1 个输出。

参数信息：属性

无

参数信息：支持精度

输入	精度	输出	精度
input	float16	output	float16
score	float16		
length	float32		

参数信息：支持排布

输入	精度	输出	精度
input	kHWC	output	kHWC
score	kHWC		
length	kHWC		

参数信息：支持形状

输入	精度	输出	精度
input	(N, H, W)	output	(N, 2*H, 1)
score	(N, H, W)		
length	(N,)		

更多参考

请参考 https://github.com/speechbrain/speechbrain/blob/develop/speechbrain/lobes/models/ECAPA_TDNN.py 了解更多关于该算子的信息。

8.1.18 Inverse Sigmoid

算子描述

Inverse Sigmoid 算子是 sigmoid 的反函数。

参数信息：属性

类型	属性名
float	eps

参数信息：支持精度

输入	精度	输出	精度
input	float16	output	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR or kHWC	output	kLINEAR or kHWC

参数信息：支持形状

输入	形状	输出	形状
input	(N, H, W, C) or (N, C, H, W)	output	(N, H, W, C) or (N, C, H, W)

更多参考

请参考 [Inverse Sigmoid](#) 了解更多关于该算子信息。

8.1.19 RebuildPadding_IxRT

算子描述

RebuildPadding_IxRT 算子用于给 NLP 模型增加 effective-padding，与 [RemovePadding_IxRT](#) 算子配合使用。RebuildPadding_IxRT 将 RemovePadding_IxRT 处理后的拼接成的输入还原成原始的输入。其中，下述的 input_id 输入为整个 NLP 模型的 token 输入。

参数信息：属性

插件版本：

- 1: FP16
- 2: INT8

类型	属性名	版本	描述
int32_t	type_id	1, 2	插件的运行精度
int32_t	hidden_size	1, 2	隐藏层的大小
int32_t	valid_num	1, 2	input 为 pad 的实际有效总长度

参数信息：支持精度

FP16

输入	精度	输出	精度
input	half	output	half
batch_id	int32_t		
token_id	int32_t		
input_id	int32_t		

INT8

输入	精度	输出	精度
input	int8_t	output	int8_t
batch_id	int32_t		
token_id	int32_t		
input_id	int32_t		

参数信息：支持排布

FP16

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
batch_id	kLINEAR		
token_id	kLINEAR		

输入	数据排布	输出	数据排布
input_id	kLINEAR		

INT8

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
batch_id	kLINEAR		
token_id	kLINEAR		
input_id	kLINEAR		

参数信息：支持形状

FP16

输入	形状	输出	形状
input	[mPadBatchSize, max_seq_len, E]	output	[B, S, E]
batch_id	[valid_num]		
token_id	[valid_num]		
input_id	[B, S]		

INT8

输入	形状	输出	形状
input	[mPadBatchSize, max_seq_len, E]	output	[B, S, E]
batch_id	[valid_num]		
token_id	[valid_num]		
input_id	[B, S]		

8.1.20 XSoftmax

算子描述

XSoftmax 算子用于标记节省内存优化的 softmax。

参数信息：属性

类型	属性名
string	name
DataType	type
int	dim
int	mIsConformer

参数信息：支持精度

输入	精度	输出	精度
input	float16	score	float16
mask	bool	score	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
mask	kLINEAR		

参数信息：支持形状

输入	形状	输出	形状
input	[bsz,head_dim,sequence_len,sequece_len]	output	[bsz,head_dim,sequence_len,sequece_len]
mask	[bsz, 1,sequence_len,sequece_len]		

更多参考

请参考 [xsoftmax](#) 了解更多关于该算子信息。

8.1.21 Conformer_Self_Attention

算子描述

Conformer_Self_Attention 算子实现了 Transformer 中的 MultiHeadSelfAttention。

参数信息：属性

类型	属性名
int32	bs
int32	seq_len
int32	n_head
int32	n_feat

参数信息：支持精度

输入	精度	输出	精度
inputx	float16	output	float16
mask	int32		
pos_emb	float16		
linear_q.weight	float16		
linear_q.bias	float16		
linear_k.weight	float16		
linear_k.bias	float16		
linear_v.weight	float16		
linear_v.bias	float16		
pos_bias_u	float16		
pos_bias_v	float16		
linear_out.weight	float16		
linear_out.bias	float16		

参数信息：支持排布

输入	数据排布	输出	数据排布
inputx	kLINEAR	output	kLINEAR

输入	数据排布	输出	数据排布
mask	kLINEAR		
pos_emb	kLINEAR		
linear_q.weight	kLINEAR		
linear_q.bias	kLINEAR		
linear_k.weight	kLINEAR		
linear_k.bias	kLINEAR		
linear_v.weight	kLINEAR		
linear_v.bias	kLINEAR		
pos_bias_u	kLINEAR		
pos_bias_v	kLINEAR		
linear_out.weight	kLINEAR		
linear_out.bias	kLINEAR		

参数信息：支持形状

输入	形状	输出	形状
inputx	(BATCH_SIZE, IN_CHANNELS, IN_FEATURE)	output	(BATCH_SIZE, IN_CHANNELS, IN_FEATURE)
mask	(BATCH_SIZE, IN_CHANNELS, 1)		
pos_emb	(1, IN_CHANNELS, IN_FEATURE)		
linear_q.weight	(IN_FEATURE, IN_FEATURE)		
linear_q.bias	(IN_FEATURE)		
linear_k.weight	(IN_FEATURE, IN_FEATURE)		
linear_k.bias	(IN_FEATURE)		
linear_v.weight	(IN_FEATURE, IN_FEATURE)		
linear_v.bias	(IN_FEATURE)		
pos_bias_u	(N_HEAD, IN_FEATURE//N_HEAD)		
pos_bias_v	(N_HEAD, IN_FEATURE//N_HEAD)		

输入	形状	输出	形状
linear_out.weight	(IN_FEATURE, IN_FEATURE)		
linear_out.bias	(IN_FEATURE)		

更多参考

参考 [Conformer](#) 了解更多关于该算子的信息。

8.1.22 Efficient NMS Plugin

8.1.23 L2_Normalization

算子描述

L2_Normalization 算子通过使用向量的 Euclidean norm 进行归一化。

参数信息：属性

类型	属性名
float	eps
int32	axes
int32	axes_length

参数信息：支持精度

输入	精度	输出	精度
input	float16	output	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	(N, C, H, W)	output	(N, C, H, W)

更多参考

参考 Pytorch [L2 Norm](#) 了解更多关于该算子的信息。

8.1.24 RemovePadding_IxRT

算子描述

RemovePadding_IxRT 算子用于给 NLP 模型增加 effective-padding，与 **RebuildPadding_IxRT** 算子配合使用。RemovePadding_IxRT 将输入拼接成设定的最大的 seq_len 长度的输入，可以减少 batchsize 的大小。其中下述的 input_id 输入为整个 NLP 模型的 token 输入。

参数信息：属性

插件版本：

- 1: FP16
- 2: INT8

类型	属性名	版本	描述
int32_t	type_id	1, 2	插件的运行精度
int32_t	hidden_size	1, 2	隐藏层的大小
int32_t	max_seq_len	1, 2	remove 后的 seq_len
int32_t	valid_num	1, 2	input 为 pad 的实际有效总长度

参数信息：支持精度

FP16

输入	精度	输出	精度
input	half	output	half
batch_id	int32_t		
token_id	int32_t		
input_id	int32_t		

INT8

输入	精度	输出	精度
input	int8_t	output	int8_t
batch_id	int32_t		
token_id	int32_t		
input_id	int32_t		

参数信息：支持排布

FP16

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
batch_id	kLINEAR		
token_id	kLINEAR		
input_id	kLINEAR		

INT8

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
batch_id	kLINEAR		
token_id	kLINEAR		
input_id	kLINEAR		

参数信息：支持形状

FP16

输入	形状	输出	形状
input	[B, S, E]	output	[mPadBatchSize, max_seq_len, E]
batch_id	[valid_num]		
token_id	[valid_num]		
input_id	[B, S]		

INT8

输入	形状	输出	形状
input	[B, S, E]	output	[mPadBatchSize, max_seq_len, E]
batch_id	[valid_num]		
token_id	[valid_num]		
input_id	[B, S]		

8.1.25 YolactDecoder

算子描述

YolactDecoder 是用于处理特征图并生成目标框的算子。该算子有 2 个输入和 1 个输出。

参数信息：属性

类型	属性名
float32*	anchor
int32	num_class
float32	stride

参数信息：支持精度组合 1

输入	精度	输出	精度
cls_map	int8	output	float32
bbox_map	int8		

参数信息：支持精度组合 2

输入	精度	输出	精度
cls_map	float16	output	float32
bbox_map	float16		

参数信息：支持排布

输入	数据排布	输出	数据排布
cls_map	kHWC	output	kLINEAR
bbox_map	kHWC		

参数信息：支持形状

输入	形状	输出	形状
cls_map	(N, 81*3, H, W)	output	(N, (H * W)*3, 6)
bbox_map	(N, 4*3, H, W)		

更多参考

请参考 https://github.com/feiyuhuahuo/Yolact_minimal 了解更多关于该算子的信息。

8.1.26 Conv2dSubsampling4

算子描述

Conv2dSubsampling4 算子通过两个 conv2d 算子和一个 linear 算子实现对网络的特征的 4 倍下采样，目前用于 wenet 中，进行语音识别任务。该算子有 8 个输入和 3 个输出。

参数信息：属性

类型	属性名	描述
int32	kernel_size_1	第 1 个 conv 的 kernel 尺寸
int32	kernel_size_2	第 2 个 conv 的 kernel 尺寸
int32	stride_1	第 1 个 conv 的 stride
int32	stride_2	第 2 个 conv 的 stride
int32	odim_1	第 1 个 conv 输出通道数
int32	odim_2	第 2 个 conv 输出通道数
int32	hidden_size	矩阵乘法输出 dim

参数信息：支持精度

输入	精度	输出	精度
input	float16	output	float16

输入	精度	输出	精度
mask	int32	pos_embed	float16
conv1_weight	float16	out_mask	int32
conv1_bias	float32		
conv2_weight	float16		
conv2_bias	float32		
linear_weight	float16		
linear_bias	float16		

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
mask	kLINEAR	pos_embed	kLINEAR
conv1_weight	kLINEAR	out_mask	kLINEAR
conv1_bias	kLINEAR		
conv2_weight	kLINEAR		
conv2_bias	kLINEAR		
linear_weight	kLINEAR		
linear_bias	kLINEAR		

参数信息：支持形状

$$\text{new_feat_len} = ((\text{feat_len} - (\text{kernel_size}_1 - 1) - 1) / \text{stride}_1 - (\text{kernel_size}_2 - 1)) / \text{stride}_2 + 1$$

$$\text{new_feat_dim} = ((\text{feat_dim} - (\text{kernel_size}_1 - 1) - 1) / \text{stride}_1 - (\text{kernel_size}_2 - 1)) / \text{stride}_2 + 1$$

输入	形状	输出	形状
input	(N, feat_len, feat_dim)	output	(N, new_feat_len, hidden_size)
mask	(N, 1, feat_dim)	pos_embed	(1, new_feat_len, hidden_size)
conv1_weight	(odim_1, 2, kernel_size_1, kernel_size_1)	out_mask	(N, new_feat_len)
conv1_bias	odim_1		

输入	形状	输出	形状
conv2_weight	(odim_2, odim_1, kernel_size_2, kernel_size_2)		
conv2_bias	odim_2		
linear_weight	(hidden_size, odim_2*new_feat_dim)		
linear_bias	hidden_size		

更多参考

请参考 <https://github.com/wenet-e2e/wenet/blob/main/wenet/transformer/subsampling.py> 了解更多关于该算子的信息。

8.1.27 CustomEmbLayerNormPluginDynamic_IxRT

算子描述

CustomEmbLayerNormPluginDynamic_IxRT 算子融合 embedding 和 layernorm 结构，用于 BERT 推理。

参数信息：属性

插件版本：

- 1: FP16
- 2: INT8

类型	属性名	版本	描述
float32 Weights	bert_embeddings_layernorm_beta	1, 2	layernorm 的偏置
float32 Weights	bert_embeddings_layernorm_gamma	1, 2	layernorm 的权重
float32 Weights	bert_embeddings_word_embeddings	1, 2	输入 input_token 对应的 embedding
float32 Weights	bert_embeddings_token_type_embeddings	1, 2	输入 input_token_type 对应的 embedding
float32 Weights	bert_embeddings_position_embeddings	1, 2	input_position 对应的 embedding
int32_t	output_fp16	1, 2	embedding_layernorm 的输出是否使用 FP16
int32_t	full_mask	1, 2	即将废弃

类型	属性名	版本	描述
int32_t	mha_type_id	1, 2	mha 的 type_id [1,2] 1: fp16 2: int8
int32_t	pad_id	1, 2	input_token 对应的需要 mask 掉的 padding 的值，默认为 0

参数信息：支持精度

FP16

输入	精度	输出	精度
inputIds	int32_t	output	half
segmentIds	int32_t	out_mask	int32_t
maskIds	int32_t		

INT8

输入	精度	输出	精度
inputIds	int32_t	output	int8_t
segmentIds	int32_t	out_mask	int8_t
maskIds	int32_t	residual_buffer	float32

参数信息：支持排布

FP16

输入	精度	输出	精度
inputIds	kLINEAR	output	kLINEAR
segmentIds	kLINEAR	out_mask	kLINEAR
maskIds	kLINEAR		

INT8

输入	精度	输出	精度
inputIds	kLINEAR	output	kLINEAR
segmentIds	kLINEAR	out_mask	kLINEAR
maskIds	kLINEAR	residual_buffer	kLINEAR

参数信息：支持形状

FP16

输入	形状	输出	形状
inputIds	[B, S]	output	[B, S, E]
segmentIds	[B, S]	out_mask	[B, S]
maskIds	[B, S]		

INT8

输入	形状	输出	形状
inputIds	[B, S]	output	[B, S, E]
segmentIds	[B, S]	out_mask	[B, S]
maskIds	[B, S]	residual_buffer	[B, S, E]

更多参考

请参考 [BERT \[原文\]](#) 了解更多关于该算子的信息。

8.1.28 Reorg

算子描述

Reorg 算子实现了对特征的重新排布，主要用于 YOLOv2 检测模型。该算子有 1 个输入和 1 个输出。

参数信息：属性

类型	属性名	描述
int32	stride	特征图尺寸缩小的步长

参数信息：支持精度组合 1

输入	精度	输出	精度
input	float16	output	float16

参数信息：支持精度组合 2

输入	精度	输出	精度
input	int8	output	int8

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	(N, input_channel, input_h, input_w)	output	(N, input_channel * stride * stride, input_h / stride, input_w / stride)

更多参考

请参考 https://github.com/pjreddie/darknet/blob/master/src/reorg_layer.c 了解更多关于该算子的信息。

8.1.29 YoloV3Decode

算子描述

YoloV3Decode 算子用于处理特征图并生成目标框。该算子有 1 个输入和 1 个输出。

参数信息：属性

类型	属性名	描述
int32	num_class	目标检测类别数量
int32	stride	特征图大小相对网络输入缩小的倍数

类型	属性名	描述
float*	anchor	目标检测使用锚点预设值
int32	faster_impl	目前仅支持 1: 检测目标仅输出分值最高的类对应的置信度

参数信息：支持精度组合 1

输入	精度	输出	精度
feature map	int8	output	float16

参数信息：支持精度组合 2

输入	精度	输出	精度
feature map	float16	output	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
feature map	kHWC	output	kLINEAR

参数信息：支持形状

- class_number: 检测目标种类
- anchor_number: 每个 feature map 点关联的 anchor 数量

输入	形状	输出	形状
feature map	$(N, (class_number+5)*anchor_number, H, W)$	output	$(N, (H*W)*anchor_number, 6)$

更多参考

请参考 <https://github.com/ultralytics/yolov3> 了解更多关于该算子的信息。

8.1.30 CTCGreedySearch

算子描述

CTCGreedySearch 算子实现贪心搜索，目前用于 wenet 中，用于处理语音识别任务数据。该算子有 1 个输入和 1 个输出。

参数信息：属性

类型	属性名	描述
int32	vocab_size	词汇表包含词汇数量
int32	eos_id	用于表示句子结尾的 id
int32	pad_id	取 top1 时会避开的 id

参数信息：支持精度

输入	精度	输出	精度
input	float16	output	int32

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	(N, feat_len, feat_dim)	output	(N, feat_len)

更多参考

请参考 <https://github.com/wenet-e2e/wenet/blob/main/wenet/transformer/search.py> 了解更多关于该算子的信息。

8.1.31 FacenetNorm

算子描述

Facenet Norm 是将最后一层的特征图做 L2 正则化的算子 (fixed outdim=512)。该算子有 1 个输入和 1 个输出。

参数信息：属性

类型	属性名	描述
int32	size	size=512

参数信息：支持精度组合 1

输入	精度	输出	精度
input	int8	output	float32

参数信息：支持精度组合 2

输入	精度	输出	精度
input	float16	output	float32

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kHWC	output	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	(N,1,1,512)	output	(N,512)

更多参考

请参考 <https://github.com/timesler/facenet-pytorch> 了解更多关于该算子的信息。

8.1.32 multiscaleDeformableAttn

算子描述

multiscaleDeformableAttnPlugin 算子用于对参考点周围的一小组关键采样点执行注意力计算，而不是查看所有可能的空间位置。该算子利用多尺度特征图来有效地表示不同尺度地对象，有助于实现更快的收敛和对小物体更好的性能。

结构

multiscaleDeformableAttnPlugin 算子有以下 5 个输入：

- value：不同尺度的输入特征图连接起来提供输入特征向量
形状：[N, S, M, D]，其中，
 - N 为 batch size
 - S 为特征图长度
 - M 为注意力头数量
 - D 为 hidden_dim/num_heads
- spatial_shapes：每个特征图的形状
形状：[L, 2]，其中，L 为特征图的数量
- level_start_index：当输入特征张量被扁平化时，该张量用于找到不同特征级别的采样位置
形状：[L,]
- sampling_locations：所有特征图像素中突出关键元素的预过滤器
形状：[N, Lq, M, L, P, 2]，其中，
 - P 为 points 的数量
 - Lq 为 feature maps (encoder) 的长度或 queries (decoder) 的长度
- attention_weights：包含 attention 权重
形状：[N, Lq, M, L, P]

multiscaleDeformableAttnPlugin 算子的输出是 attention，形状：[N, Lq, M, D]。

更多参考

请参考 [Deformable DETR](#) 了解更多关于该算子的信息。

8.1.33 RetinaNet Decoder

算子描述

RetinaNet Decoder 算子实现了 retinanet 检测模型获得检测结果的模块。

参数信息：属性

类型	属性名	
int32	num_class	
int32	stride	
float	anchors	
int32	nb_anchors	

参数信息：支持精度

输入	精度	输出	精度
class_data	float16 or int8	output	float16 or int8
box_data	float16 or int8		

参数信息：支持排布

输入	数据排布	输出	数据排布
class_data	kHWC	output	kLINEAR
box_data	kHWC		

参数信息：支持形状

输入	形状	输出	形状
class_data	(N, H, W, C)	output	(N, HWanchor_number, 6)
box_data	(N, anchor_number*4, H, W)		

更多参考

请参考 [RetinaNet](#) 了解更多关于该算子的信息。

8.1.34 TemporalShift

算子描述

TemporalShift (Temporal Shift Module) 算子用于视频理解领域，在 TSM 模型中被使用。TemporalShift 使得相邻帧之间包含了对方的信息，从而保证了时间上的信息的获取。

参数信息：属性

类型	属性名
int32	n_segment
int32	fold_div
int32	off_line

参数信息：支持精度

输入	精度	输出	精度
hidden_states	float16	q_0	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kHWC	output	kHWC

参数信息：支持形状

输入	数据排布	输出	数据排布
input	(N, H, W, C)	output	(N, H, W, C)

更多参考

请参考 [TSM 原论文](#) 了解更多关于该算子的信息。

8.1.35 YoloV5Decoder

算子描述

YoloV5Decoder 算子用于处理特征图，生成目标框。该算子有 1 个输入和 1 个输出。

参数信息：属性

类型	属性名	描述
int32	num_class	目标检测类别数量
int32	stride	特征图大小相对网络输入缩小的倍数
float*	anchor	目标检测使用锚点预设值
int32	faster_impl	目前仅支持 1: 检测目标仅输出分值最高的类对应的置信度

参数信息：支持精度组合 1

输入	精度	输出	精度
feature map	int8	output	float16

参数信息：支持精度组合 2

输入	精度	输出	精度
feature map	float16	output	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
feature map	kHWC	output	kLINEAR

参数信息：支持形状

- class_number: 检测目标种类
- anchor_number: 每个 feature map 点关联的 anchor 数量

输入	形状	输出	形状
feature map	$(N, (class_number+5)*anchor_number, H, W)$	output	$(N, (H*W)*anchor_number, 6)$

更多参考

请参考 <https://github.com/ultralytics/yolov5> 了解更多关于该算子的信息。

8.1.36 CustomFFNPluginDynamic_IxRT

算子描述

CustomFFNPluginDynamic_IxRT 算子是 BERT attention 后面的两个全连接层合在一起，用于 API 搭建 BERT FP16 模型中。

参数信息：属性

类型	属性名	描述
int32_t	out_dims	必选，输出的维度

类型	属性名	描述
int32_t	type_id	必选，DataType 类型 (FP16)
float Weight	W1	必选，fc1 的权重
float Weight	B1	可选，fc1 的偏置
float Weight	W2	必选，fc2 的权重
int32_t	act_type	必选，第一个 fc 后接的激活函数

参数信息：支持精度

输入	精度	输出	精度
input	half	output	half

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	[B, S, iE]	output	[B, S, oE]

更多参考

请参考 [BERT 原论文](#) 了解更多关于该算子的信息。

8.1.37 DetectionNMS_IxRT

算子描述

DetectionNMS_IxRT 用于检测模型 (YOLOv3, YOLOv4, YOLOv5, YOLOv7, YOLOX 等) NMS。该算子有 1 个输入，2 或 3 个输出，不支持 per class NMS。

参数信息：属性

类型	属性名	描述
float32	fIoUThresh	iou_thresh, 默认为 0.65
float32	fScoreThresh	score_thresh, 默认为 0.01
int32	nMaxKeep	最大输出 box 数量, 默认为 1000
int32	outputIndice	输出是否有索引 0: 表示不输出索引 1: 表示输出索引

参数信息：支持精度

输入	精度	输出	精度
input_box	float32	output_box	float32
		num_output_boxes	int32
		output_indices (可选, outputIndice = 1)	int32

参数信息：支持排布

输入	数据排布	输出	数据排布
input_box	kLINEAR	output_box	kLINEAR
		num_output_boxes	kLINEAR
		output_indices (可选, outputIndice = 1)	kLINEAR

参数信息：支持形状

class_number: 检测目标种类

输入	形状	输出	形状
input_box	(batch_size, num_input_box, 6)	output_box	(batch_size, nMaxKeep, 6) 6 表示 [x1,y1,x2,y2, class_id,score]
		num_output_boxes	(batch_size,)

输入	形状	输出	形状
		output_indices (可选, outputIndice = 1)	(batch_size, nMaxKeep)

更多参考

请参考 http://bitbucket.iluvatar.ai:7990/projects/SWAPP/repos/inferencesamples/browse/benchmarks/cv/detection/yolov5s/tensorrt/build_nms_engine.py#19 了解更多关于该算子的信息。

8.1.38 RMS_Norm

算子描述

RMS_Norm 算子用于对小批量输入应用均方根层归一化。

参数信息：属性

类型	属性名
string	name
int32	axis
float	epsilon
int32	stash_type
int32	hiddensize
weights	gamma

参数信息：支持精度

输入	精度	输出	精度
input	float16	output	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	(N, C, H, W)	output	(N, C, H, W)

更多参考

请参考 [RMS_Norm](#) 了解更多关于该算子的信息。

8.1.39 YoloV7Decoder

算子描述

YoloV7Decoder 算子用于处理特征图，生成目标框。该算子有 1 个输入和 1 个输出。

参数信息：属性

类型	属性名	描述
int32	num_class	目标检测类别数量
int32	stride	特征图大小相对网络输入缩小的倍数
float*	anchor	目标检测使用锚点预设值
int32	faster_impl	目前仅支持 1: 检测目标仅输出分值最高的类对应的置信度

参数信息：支持精度组合 1

输入	精度	输出	精度
feature map	int8	output	float16

参数信息：支持精度组合 2

输入	精度	输出	精度
feature map	float16	output	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
feature map	kHWC	output	kLINEAR

参数信息：支持形状

- class_number: 检测目标种类
- anchor_number: 每个 feature map 点关联的 anchor 数量

输入	形状	输出	形状
feature map	$(N, (class_number+5)*anchor_number, H, W)$	output	$(N, (H*W)*anchor_number, 6)$

更多参考

请参考 <https://github.com/WongKinYiu/yolov7> 了解更多关于该算子的信息。

8.1.40 Custom argmax

算子描述

Custom argmax 算子用于计算 top1 的 index。

参数信息：属性

类型	属性名
string	name
int32	pad_idx

参数信息：支持精度

输入	精度	输出	精度
input	float16	index	int32

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	index	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
input	(N, ..., ...)	index	(N, 1)

更多参考

请参考 [Numpy argmax](#) 了解更多关于该算子的信息。

8.1.41 Focus

算子描述

Focus 算子用于在内存中对数据做重新排布。该算子有 1 个输入和 1 个输出。

参数信息：属性

类型	属性名	描述
int32	in_c	默认值为 4
int32	out_c	默认值为 16
int32	size	默认值为 2

参数信息：支持精度组合 1

输入	精度	输出	精度
input	int8	output	int8

参数信息：支持精度组合 2

输入	精度	输出	精度
input	float16	output	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kHWC	output	kHWC

参数信息：支持形状

输入	形状	输出	形状
input	(N,C,H,W)	output	(N,C*4,H/2,W/2)

更多参考

请参考 <https://github.com/Megvii-BaseDetection/YOLOX> 了解更多关于该算子的信息。

8.1.42 PointRendCalUncertaint

算子描述

PointRendCalUncertaint 算子使用 0.0 与“logits”中对“classes”的前景类的 logit 预测之间的 L1 距离来估计不确定性。

参数信息：属性

类型	属性名
string	name

参数信息：支持精度

输入	精度	输出	精度
logits	float16 or float32	output	float16 or float32
classes	int32		

参数信息：支持排布

输入	数据排布	输出	数据排布
logits	kLINEAR	output	kLINEAR

输入	数据排布	输出	数据排布
classes	kLINEAR		

参数信息：支持形状

输入	形状	输出	形状
logits	[num_boxes, num_classes,] h, w	output	[num_boxes, 1, h, w]
classes	[num_boxes]		

更多参考

请参考 [PointRend](#) 了解更多关于该算子的信息。

8.1.43 RNN

算子描述

RNN 算子主要实现了包括普通 RNN (tanh, relu)、GRU 和 LSTM 在内的 RNN。

参数信息：属性

类型	属性名
int32	direction
int32	hidden_size
int32	linear_before_reset
int32	rnn_mode

参数信息：支持精度

输入	精度	输出	精度
inputx	float16	output	float16
initial_h	float16	output_h	
weight_ih	float16		
weight_hh	float16		

输入	精度	输出	精度
bias	float16		

参数信息：支持排布

输入	数据排布	输出	数据排布
inputx	kLINEAR	output	kLINEAR
initial_h	kLINEAR	output_h	kLINEAR
weight_ih	kLINEAR		
weight_hh	kLINEAR		
bias	kLINEAR		

参数信息：支持形状

输入	形状	输出	形状
inputx	(SEQ_LEN, BATCH_SIZE, IN_FEATURE)	output	(SEQ_LEN, BATCH_SIZE, direction, hidden_size)
initial_h	(DIRECTION, BATCH_SIZE, HIDDEN_SIZE)	output_h	(direction, BATCH_SIZE, hidden_size)
weight_ih	取决于 RNN 的类型		
weight_hh	取决于 RNN 的类型		
bias	取决于 RNN 的类型		

8.1.44 YoloxDecoder

算子描述

YoloxDecoder 算子用于处理特征图，生成目标框。该算子有 3 个输入和 1 个输出。

参数信息：属性

类型	属性名	描述
int32	num_class	目标检测类别数量

类型	属性名	描述
int32	stride	特征图大小相对网络输入缩小的倍数
int32	faster_impl	目前仅支持 1: 检测目标仅输出分值最高的类对应的置信度

参数信息：支持精度组合 1

输入	精度	输出	精度
location map	int8	output	float16
confidence map	int8		
classification map	int8		

参数信息：支持精度组合 2

输入	精度	输出	精度
location map	float16	output	float16
confidence map	float16		
classification map	float16		

参数信息：支持排布

输入	数据排布	输出	数据排布
location map	kHWC	output	kLINEAR
confidence map	kHWC		
classification map	kHWC		

参数信息：支持形状

- class_number: 检测目标种类

输入	形状	输出	形状
location map	(N, 4, H, W)	output	(N, (H*W), 6)

输入	形状	输出	形状
confidence map	(N, 1, H, W)		
classification map	(N, class_number, H, W)		

更多参考

请参考 <https://github.com/Megvii-BaseDetection/YOLOX> 了解更多关于该算子的信息。

8.1.45 fcosNMS

算子描述

fcosNMS 算子实现了 CenterMask2 模型的后处理部分，主要包括以下操作：

1. 计算不同 level 的 feature map 的预测 object 分数，并且根据阈值进行筛选；
2. 对 1 中的筛选结果进行排序，取出 pre_nms_topk；
3. 根据 2 中的排序结果取出 topk，并计算对应的 locations；
4. 进行 NMS，得到最终结果。

参数信息：属性

类型	属性名
int32	batch_size
float	nms_thresh
int32	plugin_version
int32	post_nms_topk
float	pre_nms_thresh
int32	pre_nms_topk
int32	thresh_with_ctr

参数信息：支持精度

输入	精度	输出	精度
default_anchors	float16	detection_boxes	float16
cls_score_0	float16	detection_scores	float16
anchors_3	float16	detection_classes	float16
scores_unsqueeze:0_2	float16	detection_locations	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
default_anchors	kLINEAR	detection_boxes	kLINEAR
cls_score_0	kLINEAR	detection_scores	kLINEAR
anchors_3	kLINEAR	detection_classes	kLINEAR
scores_unsqueeze:0_2	kLINEAR	detection_locations	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
default_anchors	(21824,2)	detection_boxes	(1,50,4)
cls_score_0	(1,21824,80)	detection_scores	(1,50)
anchors_3	(1,21824,4)	detection_classes	(1,50)
scores_unsqueeze:0_2	(1,21824,1)	detection_locations	(1,50,2)

更多参考

请参考 [centermask2](#) 和 [fcos](#) 了解更多关于该算子的信息。

8.1.46 PositionWiseFFN

算子描述

PositionWiseFFN 算子实现了 Transformer 结构中的前馈模块 (Feed Forward Module) 部分。Transformer 模型在 MHSA 层之后接了一个由两层线性变换和一个非线性激活函数组成的前馈模块。整个前馈模块中，在 normalization 之后增加了残差连结。

参数信息：属性

类型	属性名
int32	act_type
float	ff_scale
int32	hidden_size
int32	in_feature

参数信息：支持精度

输入	精度	输出	精度
Mul_output_1	float16	ff_macaron_0_output	float16

参数信息：支持排布

输入	数据排布	输出	数据排布
Mul_output_1	kLINEAR	ff_macaron_0_output	kLINEAR

参数信息：支持形状

输入	形状	输出	形状
Mul_output_1	(N, seq_len, in_feature)	ff_macaron_0_output	(N, seq_len, in_feature)

更多参考

请参考 [Paper transformer](#) 了解更多关于该算子的信息。

8.1.47 CustomQKVToContextPluginDynamic_IxRT

算子描述

CustomQKVToContextPluginDynamic_IxRT 算子融合 qkv Attention 结构，用于 BERT 推理。

参数信息：属性

插件版本：

- 1: FP16
- 2: deprecated
- 3: INT8

类型	属性名	版本	描述
int32_t	type_id	1	Integer encoding the DataType (1: FP16, 2: INT8)
int32_t	hidden_size	1, 3	隐藏层的大小，即 E
int32_t	num_heads	1, 3	self-attention 的头个数
int32_t	has_mask	1	是否使用 input_mask 输入

类型	属性名	版本	描述
float32	dq_probs	1, 3	QAT 之后 amax 的 list [arrange_qkv_amax, softmax_in_amax, softmax_out_amax]
int32_t	has_qk_bias	1	是否使用 relative_pos_bias
int32_t	is_t5_mode	1	是否使用 t5 的 attention 模式

约定：

- B: batch_size
- S: seq_len
- E: hidden_size
- n: num_heads

参数信息：支持精度

FP16:

输入	精度	输出	精度
input	half	output	half
(optional) input_mask	int or float		

INT8:

输入	精度	输出	精度
input	int8_t	output	int8_t
(optional) input_mask	int8_t		

参数信息：支持排布

输入	数据排布	输出	数据排布
input	kLINEAR	output	kLINEAR
(optional) input_mask	kLINEAR		

参数信息：支持形状

FP16:

输入	形状	输出	形状
input	[B, S, 3*E]	output	[B, S, E]
(optional) input_mask	int: [B, S] float: 维度需满足 [2,4], plugin 内部统一广播到 [B, n, S, S], 也支持 repeat, 如 [4,n,S,S]->[4*b, n, S, S]		

INT8:

输入	形状	输出	形状
input	[B, S, 3*E]	output	[B, S, E]
(optional) input_mask	[B, S]		

更多参考

请参考 [BERT 原论文](#) 了解更多关于该算子的信息。

8.2 ixRT 内置示例列表

编译 ixRT 内置示例前，您需要先安装 ixRT，详情请参考“ixRT 推理引擎安装指南”。

以下是编译方法：

```
//进入解压的 ixRT 目录
$ cd IXRT
$ cmake -B build -DIXRT_HOME=/path/to/IXRT
$ cmake --build build -j
```

下表是 ixRT 提供的内置示例及说明。

Note

ixRT 内置示例的使用说明，请参考对应的 README.md 文件。

示例	路径	说明
sampleAPIToInt8Resnet	samples/sampleAPIToInt8Resnet	演示 ixRT API 逐层搭建 ResNet 网络的过程，并执行量化、编译和推理
sampleAPIToResnet	samples/sampleAPIToResnet	演示 ixRT API 逐层搭建 ResNet 网络的过程，并执行编译和推理
APIToResnet	samples/python/APIToResnet	演示 ixRT Python API 逐层搭建 ResNet 网络的过程，并执行编译和推理
sampleBert	samples/bert	演示 ixRT 执行 INT8/FP16 等版本的 BERT 网络
sampleCondition	samples/sampleCondition	演示通过 ixRT 构建 ControlFlow 结构网络，并完成推理
sampleHideDataTransfer	samples/sampleHideDataTransfer	演示通过多线程/多 stream 等技术来加速 ixRT 推理
sampleResNet	samples/sampleResNet	演示 ixRT 执行 INT8/ FP16/ FP32 等各种版本的 ResNet 网络
sampleResNet18	samples/sampleResNet18	演示 ixRT 通过加载 onnx 模型执行 ResNet18 网络
resnet18	samples/python/resnet18	演示通过 ixRT Python API 加载 onnx 模型用于执行 ResNet18 网络
sampleTypedNetwork	samples/sampleTypedNetwork	演示 ixRT 执行强弱类型网络的过程以及区别
sampleYoloV3	samples/YoloV3	演示 ixRT 执行量化 YOLOv3 网络，可选的参数包括动态形状输入、是否使用 ixRT 插件等
yolov3	samples/python/Yolov3	演示通过 ixRT Python 接口执行量化 YOLOv3 网络
sampleYoloV5	samples/YoloV5	演示 ixRT 执行 YOLOv5 网络
sampleYoloX	samples/YoloX	演示 ixRT 执行 YOLOX 网络，可选择 INT8 或 FP16 版本

示例	路径	说明
quantization	samples/python/quantization	演示在天数智芯加速卡上使用 pytorch-quantization 工具量化网络并使用 ixRT 完成推理的过程
with_ppq_quant	samples/python/with_ppq_quant	演示使用 PPQ 工具量化网络并使用 ixRT 完成推理的过程

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。